

Amazon Redshift

Contents

1. Amazon Redshift Overview	5
2. Amazon Redshift Serverless	6
2.1 Compute capacity for Amazon Redshift Serverless	6
2.2 Billing for on-demand compute capacity.....	8
2.3 Getting started with IAM credentials for Amazon Redshift	11
2.4 Accessing Amazon Redshift Serverless database objects	12
2.5 Migrating a provisioned cluster to Amazon Redshift Serverless	13
2.6 Workgroups	14
2.6.1 Properties for a workgroup	14
2.6.2 Creating a workgroup with a namespace	15
2.7 Namespaces	16
2.8 Monitoring queries and workloads	18
2.9 Monitoring queries and workloads	19
2.9.1 Adding a query monitoring policy.....	20
2.9.2 Setting usage limits	21
2.10 Snapshots and recovery points	21
2.10.1 Snapshot.....	22
2.10.2 Recovery point.....	26
2.11 Copying backups to another AWS Region	26
2.12 Restoring a table	28
2.13 Data sharing in Amazon Redshift Serverless.....	29
2.13.1 Granting access to view datashares.....	29
2.13.2 Registering namespaces to the AWS Glue Data Catalog	30
3 Amazon Redshift provisioned clusters.....	31
3.1 Cluster operations	35
3.1.1 Creating Cluster	35
3.1.1 Resizing a cluster.....	36
3.1.2 Renaming a cluster	38
3.1.3 Pausing and resuming a cluster	39
3.1.4 Relocating a cluster	40
3.1.5 Shutting down and deleting a cluster	42
3.2 Multi-AZ deployment.....	42
3.2.1 Setting up Multi-AZ when creating a new cluster	42
3.2.2 Converting a Single-AZ data warehouse to a Multi-AZ data warehouse	44

3.3 Viewing performance data.....	45
3.3.1 Viewing cluster performance data	45
3.3.2 Viewing query history data	45
4 Zero-ETL integrations	47
4.1 Configure authorization for your Amazon Redshift data warehouse	50
4.2 Create a zero-ETL integration for Aurora	52
5 Interacting with Amazon Q generative SQL.....	61
6 Loading data from Amazon S3.....	62
7. Creating a notebook	64
8 Querying the AWS Glue Data Catalog	65
9 Scheduled queries with query editor v2	67
10 Amazon Redshift Python connector	70
11 Data warehouse system architecture	70
12 Amazon Redshift Performance	72
13 Columnar storage.....	73
11 Automatic table optimization	74
11.1 Data distribution for query optimization	76
11.2 Sort keys.....	82
12 Loading data in Amazon Redshift	83
12.1 Loading data from Amazon S3	84
12.2 Loading data from an Amazon DynamoDB table	86
12.3 S3 event integration to automatically copy files from Amazon S3 buckets	87
13 Unloading data into s3.....	94
14 Scalar SQL function.....	95
15 Materialized View	96
15.1 streaming ingestion from Amazon Kinesis Data Streams.....	99
16 federated queries to PostgreSQL	101
17 Redshift Spectrum.....	102
17.1 Steps for creating Redshift Spectrum from Glue data catalog.....	103
18 Cross Database Queries	104
19 Semi-structured data in Amazon Redshift.....	105
19.1 Using JSON_PARSE to insert data into SUPER columns.....	108
19.2 Using COPY to load data into SUPER columns	109
19.3 Example	115
20 SQL Commands in Redshift	126
20.1 SQL functions supported on the leader node.....	126

20.2 Names and Identifiers	132
20.3 Data Types	134
20.4 SQL Commands.....	135
20.5 SQL Function	153
20.5.1 Aggregate Function	153
20.5.2 Array Functions	155
20.5.3 Contitonal Expressions	156
20.5.4 Data type formatting functions	158
20.5.5 Date Function.....	160
20.5.6 JSON functions.....	162

1. Amazon Redshift Overview

- Fully managed, **petabyte-scale data warehouse** in the cloud.
- Supports **fast SQL-based querying** for large datasets.

◆ Amazon Redshift Serverless

- **No need for manual configuration** of data warehouse infrastructure.
- Automatically **provisions resources** and **scales capacity** based on workload.
- **Pay-as-you-go** pricing — no charges when idle.
- If we prefer to manage your Amazon Redshift resources manually, we can create provisioned clusters for your data querying needs.

◆ Ease of Use

- Start querying immediately using:
 - **Amazon Redshift Query Editor v2**
 - **Your favorite BI tools** (e.g., Tableau, Power BI)

◆ Performance & Cost Efficiency

- Delivers **fast performance** even for demanding or unpredictable workloads.
- Offers **best price-performance ratio** with familiar SQL features.
- Designed for a **zero-administration environment**.

Difference between Amazon Redshift Serverless and Provisioned:

✅ Amazon Redshift Serverless

- **No infrastructure management:** You don't need to set up clusters or nodes.
- **Automatic scaling:** Resources scale up or down based on workload.
- **Pay-per-use:** You pay only for the compute and storage used during queries.
- **Ideal for:** Unpredictable workloads, quick analytics without setup.
- **Zero administration:** No need to manage capacity, backups, or maintenance.

✅ Amazon Redshift Provisioned

- **Manual configuration:** You create and manage clusters with specific node types.
- **Fixed capacity:** Performance depends on the cluster size you choose.
- **Pay for uptime:** Charges apply even when the cluster is idle.
- **Ideal for:** Predictable, continuous workloads with consistent usage.
- **More control:** You can fine-tune performance and optimize resources.

2. Amazon Redshift Serverless

✓ Core Features

- **Automatic provisioning & scaling:** Adjusts capacity in seconds for high performance.
- **No cluster management:** No need to set up, tune, or manage provisioned clusters.
- **Superior SQL & integration:** Query across data warehouse, data lake (Amazon S3), and operational sources.
- **Consistent performance:** Handles demanding and volatile workloads with intelligent scaling.
- **Workgroups & namespaces:** Organize compute and data with granular cost controls.
- **Pay-per-use:** Charges only when the warehouse is in use.
- **Access options:** Use console interface or APIs for applications.
- **Data access:** Connect to Amazon Redshift managed storage and Amazon S3 data lake.

Create Redshift Serverless:

1. [Video link](#)
2. [Video Link](#)

2.1 Compute capacity for Amazon Redshift Serverless

✓ Compute Capacity Basics

- **Compute capacity** = Processing power + memory for workloads.
- **Scales automatically** up/down to match workload requirements.
- **Common use cases:** Peak traffic, complex analytics, large data processing.

✓ Key Terms

- **RPUs (Redshift Processing Units)**
 - Measure of capacity.
 - **1 RPU = 16 GB memory.**
- **Base Capacity**
 - Default = **128 RPU**s.
 - Adjustable: **4 RPU**s to **512 RPU**s (or up to **1024 RPU**s in some regions).
 - Higher base capacity = better performance for heavy queries.

✓ Base Capacity

- **Definition:** The minimum compute resources (in RPU) that your Redshift Serverless workgroup starts with to serve queries.
- **Measured in:** Redshift Processing Units (RPU).
- **Default:** 128 RPU.
- **Range:** 4 RPU to 512 RPU (up to 1024 RPU in select regions).

✓ Why It Matters

- Higher base capacity = **better query performance**, especially for:
 - Large datasets.
 - Complex queries.
 - High concurrency workloads.

✓ Configuration Rules

- **4 RPU**s → For warehouses <32 TB.
- **8–24 RPU**s → For workloads <128 TB.
- **≥32 RPU**s → Required for >128 TB or heavy workloads.
- **Max 1024 RPU**s → For very large, complex workloads (available in some regions).

✓ Performance Considerations

- Use higher RPU
s for:

 - Complex queries.
 - Tables with many columns.
 - High JOINS or aggregations.
 - Large external data scans (e.g., data lake).

✓ Limitations

- Cannot set <8 RPU
s if storage >32 TB.- Cannot set <32 RPU
s if storage >128 TB.- Changing base capacity may cancel running queries.
- Vacuum boost only supported for **≥8 RPU**s.

To edit the price-performance target for your workgroup

1. In the Amazon Redshift Serverless console, choose **Workgroup configuration**.
2. Choose the workgroup for which you want to edit the price-performance target. Choose the **Performance** tab, then choose **Edit**.
3. Choose **Price-performance** target, and adjust the slider to your desired setting.
4. Choose **Save changes**.
5. To update the maximum amount of RPU
s that Amazon Redshift Serverless can allocate to your workload, choose the **Limits** tab of the **Workgroup Configuration** section.

You can use the **Price-performance target** slider to set your desired balance between cost and performance. By moving the slider, you can choose one of the following options:

- **Optimizes for cost** — This setting prioritizes cost savings. Amazon Redshift Serverless attempts to automatically scale up compute capacity when doing so doesn't incur additional charges. Amazon Redshift Serverless also attempts to scale down compute resources for lower cost, possibly increasing query runtimes.
- **Balanced** — This setting creates a balance between performance and cost. Amazon Redshift Serverless scales for performance, and may result in a moderate cost increase or decrease. This is the recommended setting for most Amazon Redshift Serverless data warehouses.
- **Optimizes for performance** — This setting prioritizes performance. Amazon Redshift scales aggressively for high performance, potentially incurring higher costs.
- Intermediate positions: You can also set the slider to one of two intermediate positions between **Balanced** and **Optimizes for cost** or **Optimizes for performance**. Use these settings if full optimization for cost or performance is too extreme.

2.2 Billing for on-demand compute capacity

Base Capacity & Billing

- **Billing model:**
 - Compute charges = **RPU-hours**, billed per second while queries run.
 - No compute charges when idle.
 - Storage charges = **Redshift Managed Storage (RMS)** based on data stored.
- **Base Capacity:**
 - Sets **minimum compute resources** for your workgroup (in RPUs).
 - Higher base capacity = **better performance**, but can increase cost if queries run longer.
- **Automatic Scaling:**
 - Redshift Serverless scales beyond base capacity as queries increase to maintain performance.

Maximum RPU Hours Usage Limit

- A **cost control feature** in Amazon Redshift Serverless.
- Lets you set a **limit on total compute usage** (in RPU-hours) per **day, week, or month**.
- Applies to:
 - Queries accessing **data warehouse**.
 - Queries accessing **external data** (e.g., Amazon S3).

✓ Why Use It?

- Helps keep costs predictable.
- Does **NOT** limit performance or compute resources directly.
- Can be adjusted anytime without affecting query processing.

✓ Actions When Limit is Reached

- Write a **log entry** to system table.
- Send an **alert**.
- **Turn off user queries**.

✓ Example

- Weekly limit = **100 RPU-hours**.
- After reaching 100 hours, queries stop.

✓ Steps to Configure in AWS Console

1. **Sign in to AWS Management Console.**
2. Navigate to **Amazon Redshift → Serverless → Workgroups**.
3. Select your **workgroup**.
4. Go to **Limits tab**.
5. Click **Manage usage limits**.
6. Add a new limit:
 - **Frequency:** Daily, Weekly, or Monthly.
 - **Duration:** e.g., 100 hours.
 - **Action:** Choose one (Log entry, Alert, Turn off queries).
7. Save the configuration.

✓ Additional Cost Control

- Use **AWS Cost Anomaly Detection** to avoid billing surprises.
- Use **Amazon Redshift Pricing Calculator** for estimates.

✓ Max Capacity

- **Definition:** The **upper limit of RPUs** that Redshift Serverless can scale up to for a workgroup.
- **Purpose:** Controls **compute cost** by preventing scaling beyond a set RPU ceiling.
- **Top limit:** **5632 RPUs**.

✓ Difference from RPU Hour Usage Limits

- **Max Capacity:**
 - Sets the **maximum RPUs** for scaling.
 - Stops scaling beyond this limit.
 - Does **not stop queries**; only limits compute resources.
- **RPU Hour Usage Limit:**
 - Does **not cap RPUs**.
 - Adds logs, sends notifications, or stops queries to control cost.

✓ Use Case Example

- Base capacity = **256 RPUs**.
- Max capacity = **512 RPUs**.
- During peak demand, scaling stops at 512 RPUs → prevents unexpected cost spikes.

✓ Important Notes

- Each workgroup can have its own Max capacity.
- Setting Max capacity too low can **delay workloads**.
- Does not interfere with running queries.
- Can be turned off by setting **-1**.

✓ How to Set

- **AWS Console** → Workgroup configuration.
- **AWS CLI:**

```
aws redshift-serverless update-workgroup --workgroup-name myworkgroup --max-capacity 512
```

- To disable: `aws redshift-serverless update-workgroup --workgroup-name myworkgroup --max-capacity -1`

Query the `SYS_SERVERLESS_USAGE` system table to track usage and get the charges for queries:

```
select trunc(start_time) "Day",  
(sum(charged_seconds)/3600)::double
```

```
precision) * <Price for 1 RPU> as cost_incurred

from sys_serverless_usage

group by 1

order by 1
```

This query provides the cost per day incurred for Amazon Redshift Serverless, based on usage.

2.3 Getting started with IAM credentials for Amazon Redshift

✓ Step 1: Sign in with IAM Role

1. Go to **AWS Management Console** → **Amazon Redshift**.
2. Sign in using an **IAM role** that has:
 - AdministratorAccess **OR**
 - AmazonRedshiftFullAccess.

✓ Step 2: Create Redshift Serverless Instance

1. In the **Amazon Redshift console**, choose **Create workgroup**.
2. Select **Amazon Redshift Serverless**.
3. Configure:
 - **Workgroup name**.
 - **Namespace name**.
 - **Base capacity** (default is 128 RPU, can adjust later).

✓ Step 3: Database Setup

- During instance creation:
 - **Option 1: Use default settings** → IAM identity becomes **database superuser**.
 - **Option 2: Customize settings without admin credentials** → IAM credentials act as admin.
 - **Option 3: Customize settings with admin username/password** → Creates a **persistent superuser login**.

Mostly everyone choose option 3.

✓ Step 4: Connect to Database

- Use **Query Editor v2** in the console.
- Choose **Temporary credentials** or the **admin username/password** you created.

✔ Step 5: Verify IAM Role

- Ensure the IAM role is attached to the workgroup.
- Check permissions in **IAM console**.

2.4 Accessing Amazon Redshift Serverless database objects

✔ Step 1: Create IAM User

1. Go to **AWS IAM Console**.
2. Create a new **IAM user** (this user will later query the database).

✔ Step 2: Create Database Role in Redshift

1. Open **Query Editor v2** or any SQL client connected to your Redshift Serverless database.
2. Run: `CREATE ROLE urban_planning;`
3. Verify role creation: `SELECT * FROM SVV_ROLES;`

✔ Step 3: Grant Role Permissions

- Grant SELECT permission on a table (example: `cities`): `GRANT SELECT ON TABLE cities TO ROLE urban_planning;`

✔ Step 4: Create IAM Role for Query Editor Access

1. In **IAM Console**, create a new **IAM role**.
2. Trusted entity: **AWS account** → **This account**.
3. Attach policies:
 - `AmazonRedshiftReadOnlyAccess`
 - `tag:GetResources`
 - `tag:GetTagKeys`
 - All actions for `sqlworkbench` (e.g., `sqlworkbench:ListDatabases`, `sqlworkbench:UpdateConnection`).

✔ Step 5: Add Tag to IAM User

- Add tag:
 - **Key:** `RedshiftDbRoles`
 - **Value:** `urban_planning` (the database role name).

✔ Step 6: Assign IAM Role to IAM User

- Attach the IAM role created in Step 4 to the IAM user.

✔ Step 7: Sign In and Query

- When the IAM user signs in to **Query Editor v2**, the tag passes the database role to Redshift.
 - The user can now query tables using the assigned role (e.g., `SELECT * FROM cities;`).
-

2.5 Migrating a provisioned cluster to Amazon Redshift Serverless

We can migrate your existing provisioned clusters to Amazon Redshift Serverless, enabling on-demand and automatic scaling of compute resources. Migrating a provisioned cluster to Amazon Redshift Serverless allows you to optimize costs by paying only for the resources you use and automatically scaling capacity based on workload demands. Common use cases for the migration include running ad-hoc queries, periodic data processing jobs, or handling unpredictable workloads without over-provisioning resources. Perform the following set of tasks to migrate your provisioned Amazon Redshift cluster to the serverless deployment option.

Creating a snapshot of your provisioned cluster

To create a snapshot of your provisioned cluster

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. On the navigation menu, choose **Clusters**, **Snapshots**, and then choose **Create snapshot**.
3. Enter the properties of the snapshot definition, then choose **Create snapshot**. It might take some time for the snapshot to be available.

To restore a provisioned cluster snapshot to a serverless namespace:

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. Start on the Amazon Redshift provisioned cluster console and navigate to the **Clusters**, **Snapshots** page.
3. Choose a snapshot to use.
4. Choose **Restore snapshot**, **Restore to serverless namespace**.
5. Choose a namespace to restore your snapshot to.
6. Confirm you want to restore from your snapshot. This action replaces all the databases in your serverless endpoint with the data from your provisioned cluster. Choose **Restore**.

2.6 Workgroups

- A **workgroup** is a collection of **compute resources**.
- It includes:
 - RPU (Redshift Processing Units)
 - VPC subnet groups
 - Security groups
 - Access and usage limits
- Properties include:
 - Network and security settings
- Can be configured via:
 - Redshift Serverless Console
 - AWS CLI
 - Redshift Serverless APIs

◆ Relationship Between Namespace and Workgroup

- You can create **multiple namespaces and workgroups**.
- **One-to-one relationship:**
 - Each **namespace** can be associated with **only one workgroup**.
 - Each **workgroup** can be associated with **only one namespace**.

2.6.1 Properties for a workgroup

- **Workgroup:** Name of the workgroup.
- **Namespace:** Associated namespace (one-to-one relationship).
- **Date Created:** Timestamp of creation.
- **Status:** Indicates availability of resources.
- **Endpoint:** URL to connect to the instance.
- **JDBC URL:** For JDBC client connections.
- **ODBC URL:** For ODBC client connections (includes DB and user ID).
- **Workgroup Version & Patch Version:** Tracks updates and patches.

Catalog ARN - The Amazon Resource Name (ARN) for the managed workgroup in the AWS Glue Data Catalog.

- **Network and Security:**
 - VPC ID, Security Groups
 - Enhanced VPC Routing (for secure traffic)
 - you can select **Turn on enhanced VPC routing**, which routes network traffic between your serverless database and data repositories through a VPC
 - Turn on Public Access setting
 - IP Address Type: Can be set to **dual-stack** (IPv4 + IPv6)

- **Redshift Managed VPC Endpoints:**
 - Allows access from another VPC
- **Base Capacity (RPU):**
 - Set compute capacity for workloads
- **Usage Limits:**
 - Up to 4 limits (e.g., 500 RPU hours alert, 900 RPU hours block queries)
 - Helps control and predict costs
 - For example, you can set your workgroup to have two limits, one of 500 RPU hours and one of 900 RPU hours. You can have Amazon Redshift Serverless send you an alert when it reaches the first limit of 500 RPU hours, then turn off user queries when it reaches the second limit of 900 hours.
- **Query Limits:**
 - Set query timeouts and other limits
 - Optimizes performance and cost

2.6.2 Creating a workgroup with a namespace

- **Open Serverless Dashboard**
 - Go to the **Serverless dashboard**.
 - Click **Create workgroup**.
- **Enter Workgroup Details**
 - **Workgroup Name:** Provide a name.
- **Choose IP Address Type**
 - **IPv4:** Only IPv4 communication.
 - **Dual-stack mode:** Supports both IPv4 and IPv6.
 - Requires IPv6 CIDR block in VPC and subnets.
 - Use Amazon VPC console to configure IPv6.
- **Select Network Settings**
 - **VPC:** Choose a VPC (must support IPv6 for dual-stack).
 - **Security Groups:** Select one or more.
 - **Subnets:** Choose subnets in **three distinct Availability Zones**.
- **Set Compute Capacity**
 - Choose **Base RPU capacity** as per workload needs.

- **Choose Namespace**
 - **Create new namespace:** Enter name.
 - **Add to existing namespace:** Select from list.
- **Configure Database Settings**
 - **Database Name & Password:** Set initial DB name and admin credentials.
 - Optionally, change **admin user**.
- **Set Permissions**
 - **Associate IAM Role:** Link IAM roles to namespace/workgroup.
- **Encryption & Logging**
 - Choose or create **KMS key** for encryption.
 - Enable **Audit Logging**: Select log types to export.
- **Review & Save**
 - Review all settings under **Review and create**.
 - Click **Save**.
- **Workgroup Created**
 - The new workgroup appears in the **Workgroups list**.

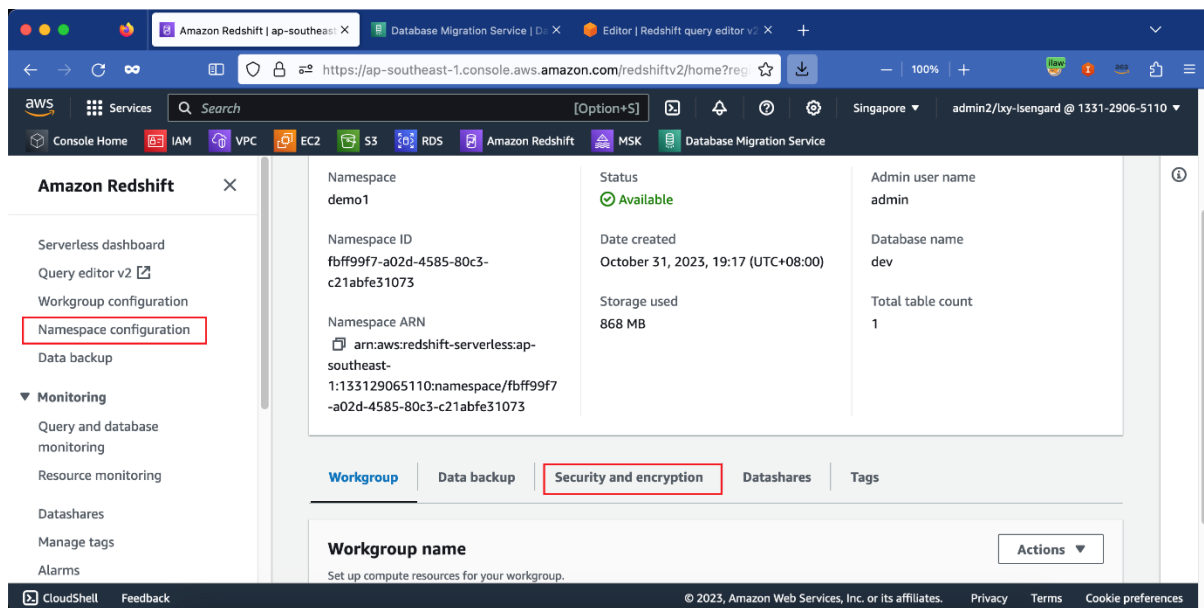
2.7 Namespaces

- A **namespace** is a collection of **database objects and users**.
- It groups:
 - Schemas, tables, users
 - AWS KMS keys (for encryption)
 - Datashares, recovery points, usage limits
- Storage properties include:

- Database name, admin password
- Permissions, encryption, and security settings
- Can be configured via:
 - Redshift Serverless Console
 - AWS CLI
 - Redshift Serverless APIs
- **Namespace:** Name of the namespace.
- **Namespace ID:** Unique identifier.
- **ARN:** Amazon Resource Name (includes region and service info).
- **Status:** Indicates availability (e.g., Available).
- **Date Created:** UTC timestamp of creation.
- **Storage Used:** Total storage consumed by namespace objects.
- **Admin User Name:** Typically the creator of the namespace.
- **Database Name:** Name of the database within the namespace.
- **Total Table Count:** Number of tables across all schemas.

Additional Tabs & Settings

1. **Workgroup**
 - Displays the associated workgroup (one-to-one relationship).
2. **Data Backup**
 - Configure **snapshots** and **recovery points**.
3. **Security and Encryption**
 - Manage **IAM roles** and **encryption settings**.
 - View encryption key status.
 - Enable **audit logging**.
4. **Datashares**
 - View and manage **datashares**.
 - Allows **data sharing** without copying or moving data.



2.8 Monitoring queries and workloads

- **Workload Monitoring**

- Track query activity over time.
- Understand **baseline patterns** and **SLA compliance**.
- Detect **deviations** or anomalies quickly.

- **Data Load & Unload Monitoring**

- Monitor **COPY** and **UNLOAD** operations.
- Track:
 - Bytes/rows transferred
 - Files completed
- Run frequent queries (e.g., every minute) to detect issues.

- **Failure & Problem Diagnostics**

- Diagnose **query/runtime failures**.
- Developers use system views to identify and fix issues.

- **Performance Tuning**

- Tune queries that violate SLAs or degrade over time.
- Requires:
 - Runtime details
 - Execution plans
 - Statistics
 - Resource usage
- Compare with **baseline data** to guide improvements.

- **User Object Event Monitoring**

- Monitor actions on user objects:
 - Refreshing materialized views
 - VACUUM
 - ANALYZE
- Includes **system-managed events** like auto-refresh.

- **Usage Tracking for Billing**

- Track usage trends to:
 - Plan budgets
 - Estimate expansion
 - Identify cost-saving opportunities (e.g., removing cold data)

2.9 Monitoring queries and workloads

- ◆ System Views Overview

- Located in the `pg_catalog` schema.
- Use **SQL SELECT statements** to query them.
- Designed specifically for **Amazon Redshift Serverless** (simpler than provisioned clusters).
- **SYS views** are optimized for Serverless monitoring.

- ◆ Monitoring Objectives

1. **Workload Monitoring**

- Track query activity over time.
- Understand **baseline patterns** and **SLA compliance**.
- Detect **deviations** or anomalies quickly.

2. **Data Load & Unload Monitoring**

- Monitor **COPY** and **UNLOAD** operations.
- Track:
 - Bytes/rows transferred
 - Files completed
- Run frequent queries (e.g., every minute) to detect issues.

3. **Failure & Problem Diagnostics**

- Diagnose **query/runtime failures**.
- Developers use system views to identify and fix issues.

4. **Performance Tuning**

- Tune queries that violate SLAs or degrade over time.
- Requires:
 - Runtime details
 - Execution plans
 - Statistics
 - Resource usage

- Compare with **baseline data** to guide improvements.
- 5. **User Object Event Monitoring**
 - Monitor actions on user objects:
 - Refreshing materialized views
 - VACUUM
 - ANALYZE
 - Includes **system-managed events** like auto-refresh.
- 6. **Usage Tracking for Billing**
 - Track usage trends to:
 - Plan budgets
 - Estimate expansion
 - Identify cost-saving opportunities (e.g., removing cold data)

2.9.1 Adding a query monitoring policy

To add the query monitoring policy

1. Choose <https://console.aws.amazon.com/iam/>.
2. Under **Access management**, choose **Policies**.
3. Choose **Create Policy**.
4. Choose **JSON** and paste the following policy definition.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "redshift-data:ExecuteStatement",
        "redshift-data:DescribeStatement",
        "redshift-data:GetStatementResult",
        "redshift-data:ListDatabases"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "redshift-serverless:GetCredentials",
      "Resource": "*"
    }
  ]
}
```

1. Choose **Review policy**.
2. For **Name**, enter a name for the policy, such as `query-monitoring`.
3. Choose **Create policy**.

2.9.2 Setting usage limits

1. Choose **Manage usage limits**. The limits section appears at the bottom of the **Compute usage by period** panel.
2. Set a usage limit in number of RPU hours.
3. Choose a **Frequency**, which is either **Daily**, **Weekly**, or **Monthly**. This sets the time period for the usage limit. Choosing **Daily** in this instance gives you more detailed control.
4. Set a usage limit, in number of hours.
5. Set the action. These are the following:
 - **Log to system table** - Adds a record to the system view [SYS_QUERY_HISTORY](#). You can query the `usage_limit` column in this view to determine if a query exceeded the limit.
 - **Alert** - Uses Amazon SNS to set up notification subscriptions and send notifications if a limit is breached. You can choose an existing Amazon SNS topic or create a new one.
 - **Turn off user queries** - Disables queries to stop use of Amazon Redshift Serverless. It also sends a notification.

The first two actions are informational, but the last turns off query processing.

6. Optionally, you can set a **Cross-Region data sharing usage limit**, which limits how much data transferred from producer Region to consumer Region consumers can query. To do this, choose **Add limit** and follow the steps.
7. Choose **Save changes** at the bottom of the page to save the limit.
8. Set up to 3 more limits as necessary.

For Query Limits:

1. Choose **Manage query limits**. Choose **Add new limit** on the **Manage query limits** dialogue.
2. Choose the limit type you want to set and enter a value for its corresponding limit.
3. Choose **Save changes** to save the limit.

When you change your query limit and configuration parameters, your database will restart.

2.10 Snapshots and recovery points

◆ Types of Backups

1. **Snapshots** – Manually created by users.

2. **Recovery Points** – Automatically created by Amazon Redshift Serverless.

◆ Recovery Point Details

- Created **every 30 minutes** or after **5 GB of data changes per node**, whichever is earlier.
- For large datasets (> 5 GB × number of nodes), minimum interval is **15 minutes**.
- **Retention:** Recovery points are kept for **24 hours**.
- **Note:** You **cannot schedule** recovery points manually.

◆ Snapshot Storage

- Stored in **Redshift Managed Storage (RMS)**.

◆ No-Backup Tables

- **Not supported** in RA3 clusters or Serverless workgroups.
- Marked no-backup tables are treated as **permanent** and **always backed up**.
- To avoid snapshot costs: **truncate** no-backup tables before taking a snapshot.

◆ Sharing

- Snapshots can be **shared with other AWS accounts** for data access and querying.

2.10.1 Snapshot

To create a snapshot, perform the steps in the following procedure.

To create a snapshot

1. On the Amazon Redshift Serverless console, choose **Data backup**.
2. Choose **Create snapshot**.
3. Choose a namespace to create a snapshot of.
4. Enter a snapshot identifier.
5. (Optional) Choose a retention period. If you choose **Custom value**, choose the number of days. The amount you choose must be between 1-3653 days, inclusive. The default is retain indefinitely.
6. Choose **Create**.

To create a snapshot from namespace configuration

1. On the Amazon Redshift Serverless console, choose **Namespace configuration**.

2. Choose the namespace to create a snapshot of. You can only create snapshots of namespaces that are associated with a workgroup and whose statuses are Available.
3. Choose the **Data backup** tab.
4. Choose **Create snapshot**.
5. Enter a snapshot identifier.
6. (Optional) Choose a retention period. If you choose **Custom value**, choose the number of days. The amount you choose must be between 1-3653 days, inclusive.
7. Choose **Create**.

To create a final snapshot

1. On the Amazon Redshift Serverless console, choose **Namespace configuration**.
2. Choose the namespace to delete.
3. Choose **Actions, Delete**.
4. Choose **Create final snapshot**.
5. Enter a name for the snapshot.
6. Enter delete.
7. Choose **Delete**.

To share or remove access to a snapshot

1. On the Amazon Redshift Serverless console, choose **Data backup**.
2. Choose a snapshot to share.
3. Choose **Actions, Manage access**.
4. To share a snapshot with another account, enter an **AWS account ID**. To remove access from an account, choose **Remove**.
5. Choose **Save changes**.

To schedule a snapshot

1. On the Amazon Redshift Serverless console, choose **Data backup**.
2. Choose **Snapshot schedules**.
3. Choose **Create schedule**.
4. Enter a name for the snapshot schedule.
5. Select the namespace to create snapshots for.
6. Enter a cron expression for the schedule or use the schedule builder to create one.
7. (Optional) Choose a retention period. If you choose **Custom value**, specify the number of days.
8. Choose **Create schedule**.

To precisely control when to take a snapshot, you can create a snapshot schedule for specific namespaces. When scheduling snapshot creation, you can create either

a one-time event or use Unix cron expressions to create a recurring schedule. Cron expressions support three fields and are separated by white space.

cron(*Minutes Hours Day-of-month Month Day-of-week Year*)

Fields	Values	Wildcards
Minutes	0–59	, - * /
Hours	0–23	, - * /
Day-of-month	1–31	, - * ? / L W
Month	1–12 or JAN-DEC	, - * /
Day-of-week	1–7 or SUN-SAT	, - * ? L #
Year	1970–2199	, - * /

Wildcards

- The , (comma) wildcard includes additional values. In the Day-of-week field, MON,WED,FRI would include Monday, Wednesday, and Friday. Total values are limited to 24 per field.
- The - (dash) wildcard specifies ranges. In the Hour field, 1–15 would include hours 1 through 15 of the specified day.
- The * (asterisk) wildcard includes all values in the field. In the Hours field, * would include every hour.
- The / (forward slash) wildcard specifies increments. In the Hours field, you could enter 1/10 to specify every 10th hour, starting from the first hour of the day (for example, the 01:00, 11:00, and 21:00).
- The ? (question mark) wildcard specifies one or another. In the Day-of-month field you could enter 7, and if you didn't care what day of the week the seventh was, you could enter ? in the Day-of-week field.
- The L wildcard in the Day-of-month or Day-of-week fields specifies the last day of the month or week.
- The W wildcard in the Day-of-month field specifies a weekday. In the Day-of-month field, 3W specifies the day closest to the third weekday of the month.
- The # wildcard in the Day-of-week field specifies a certain instance of the specified day of the week within a month. For example, 3#2 would be the second Tuesday of the month: the 3 refers to Tuesday because it is the third

day of each week, and the 2 refers to the second day of that type within the month.

Limits

- You can't specify the `Day-of-month` and `Day-of-week` fields in the same cron expression. If you specify a value in one of the fields, you must use a `?` (question mark) in the other.
- Snapshot schedules don't support the following frequencies:
 - Snapshots scheduled more frequently than 1 per hour.
 - Snapshots scheduled less frequently than 1 per day (24 hours).

If you have overlapping schedules that result in scheduling snapshots within a 1 hour window, a validation error results.

To update a snapshot retention period

1. On the Amazon Redshift Serverless console, choose **Data backup**.
2. Choose a snapshot to update.
3. Choose **Actions, Set manual snapshot settings**.
4. Choose a retention period. If you choose **Custom value**, choose the number of days.
5. Choose **Save changes**.

Restoration Behavior

- **Restoring a snapshot replaces** the current database in the namespace with the one from the snapshot.

◆ Two-Phase Restoration Process

1. **Phase 1: Initial Restore**
 - Completes in **a few minutes**.
 - Restores data and makes it **available for queries**.
2. **Phase 2: Database Tuning**
 - Can last from **a few hours to several days**, or even **a couple of weeks** for large datasets.
 - Minor **performance issues** may occur during this phase.
 - Performance **gradually improves** as tuning progresses.
 - After completion, the namespace is **fully optimized** for queries.

To restore a snapshot to a serverless namespace

1. On the Amazon Redshift Serverless console, choose **Data backup**.
2. Choose the snapshot to restore. You can only restore one snapshot at a time.
3. Choose **Actions, Restore to serverless namespace**.
4. Choose an available namespace to restore to. You can only restore to namespaces whose statuses are Available.

5. Choose **Restore**.

To restore a snapshot to a provisioned cluster

1. On the Amazon Redshift Serverless console, choose **Data backup**.
 2. Choose a snapshot to restore.
 3. Choose **Action, Restore to provisioned cluster**.
 4. Enter a cluster identifier.
 5. Choose a **Node type**. The number of nodes depends on the node type.
-

2.10.2 Recovery point

- **Automatic Creation:**
 - Every **30 minutes**, or
 - After **5 GB of data changes per node**, whichever comes first.
 - For large datasets, the minimum interval can be **15 minutes**.
- **Retention:**
 - Recovery points are stored for **24 hours**.
- **Purpose:**
 - Enables **quick recovery** in case of data loss or corruption.
 - Helps maintain **business continuity** without manual intervention.
- **Limitations:**
 - You **cannot schedule** recovery points manually.
 - They are **not customizable** like manual snapshots.
- **Storage:**
 - Stored in **Redshift Managed Storage (RMS)**.

To convert a recovery point to a snapshot

1. On the Amazon Redshift Serverless console, choose **Data backup**.
2. Under **Recovery points**, choose the **Creation time** of the recovery point that you want to convert to a snapshot.
3. Choose **Create snapshot from recovery point**.
4. Enter a **Snapshot identifier**.
5. Choose **Create**.

To restore a recovery point to a serverless namespace

1. On the Amazon Redshift Serverless console, choose **Data backup**.
 2. Under **Recovery points**, choose the **Creation time** of the recovery point that you want to restore.
 3. Choose **Restore**. You can only restore to namespaces whose statuses are Available.
 4. Enter **restore** in the text input field and choose **Restore**.
-

2.11 Copying backups to another AWS Region

- Automatically **copies snapshots and recovery points** to another **AWS Region**.

- Only **one destination Region** can be configured at a time per namespace.
 - Once copied, you can **restore** the snapshot/recovery point in the destination Region.
-

◆ Required IAM Permissions

To configure and manage cross-Region backup copying:

Redshift Serverless Permissions:

- `redshift-serverless:CreateSnapshotCopyConfiguration`
- `redshift-serverless:UpdateSnapshotCopyConfiguration`
- `redshift-serverless:ListSnapshotCopyConfigurations`
- `redshift-serverless>DeleteSnapshotCopyConfiguration`

KMS Permissions (if using custom key):

- `kms:CreateGrant`
- `kms:DescribeKey`

◆ Encryption

- By default, AWS uses a **managed KMS key**.
 - You can choose a **custom KMS key**.
 - Amazon Redshift Serverless will create a **grant** to enable encryption in the destination Region.
-

◆ Changing Destination Region

- To change the destination Region:
 1. **Disable** backup copying.
 2. **Re-enable** it with the **new Region**.
-

◆ Steps to Configure Cross-Region Backup

1. Go to **Amazon Redshift Serverless console**.
 2. Choose the **namespace**.
 3. Click **Actions > Configure cross-Region backup**.
 4. Select the **destination AWS Region**.
 5. (Optional) Set **snapshot retention period** (1–3653 days).
 6. (Optional) Choose a **custom KMS key**.
 7. Click **Save configuration**.
-

2.12 Restoring a table

◆ What Table Restore Does

- Restores a **single table** from a snapshot or recovery point.
- You must specify:
 - Source: snapshot/recovery point, database, schema, table
 - Target: database, schema, **new table name**
- The **new table name must be unique** (cannot match an existing table).

◆ Limitations

- **Only one table** can be restored at a time.
- **No-backup tables** are always backed up and restored, but **cannot be selectively restored**.
- **Foreign keys are not restored** to avoid dependency conflicts.
- **Views, permissions, and other dependencies** are not applied to the restored table.

◆ Ownership Rules

- If the **original owner exists** and has permission, they own the restored table.
- Otherwise, the **admin user** becomes the owner.

◆ Security Settings

- If **row-level security** was enabled on the original table, it remains enabled on the restored table.

◆ Restore Behavior

- The restored table reflects the **state at the time of backup**.
- Follows **Amazon Redshift's serializable isolation rules**, meaning restored data is immediately visible to new transactions.

◆ Steps to Restore a Table

1. Go to the **Amazon Redshift Serverless console**.
2. Choose **Data backup**.
3. Select the **snapshot or recovery point**.
4. Click **Actions > Restore table from snapshot/recovery point**.
5. Enter source and target details.
6. Click **Restore table**.

2.13 Data sharing in Amazon Redshift Serverless

- **Live Data Access:** Users get real-time access to the most up-to-date and consistent data as it's updated in Amazon Redshift Serverless.
- **Cross-Instance Sharing:** Data can be shared for **read-only purposes** across:
 - Different Redshift Serverless instances
 - Across AWS accounts
 - Across AWS Regions
- **Getting Started:**
 - Use **SQL interface** or **Amazon Redshift Console**
 - Navigate to **Namespace configuration > Datashares** in the console
- **Querying Shared Data:**
 - Create a **database** in a namespace with an associated **workgroup**
 - Select a **datashare** and associate it with a namespace to start querying
- **Supported Infrastructure:**
 - Only **RA3 instance types** (`ra3.16xlarge`, `ra3.4xlarge`, `ra3.xlplus`) and **Serverless endpoints** are supported for data sharing
- **Encryption:**
 - Amazon Redshift Serverless is **encrypted by default**

2.13.1 Granting access to view datashares

1. **Superuser Privileges:**
 - A **superuser** can grant access to **non-superusers** to view datashares created by any user.
2. **Granting Access to Individual Users:**
 - Use the following SQL command:

```
GRANT SHARE ON DATASHARE datasharename TO "IAM:testuser";
```
 - Replace `datashare_name` with the actual datashare name and `"IAM:test_user"` with the target user's IAM name.
3. **Granting Access to User Groups:**
 - First, **create a user group** using the `CREATE GROUP` command.
 - Then, grant access using:

```
GRANT SHARE ON DATASHARE datasharename TO GROUP usergroup;
```

- Replace `datashare_name` and `user_group` with appropriate values.

```
-- Team A creates a datashare
CREATE DATASHARE sales_data_share;

-- Team A adds a schema/table to the datashare
ALTER DATASHARE sales_data_share ADD SCHEMA public;
ALTER DATASHARE sales_data_share ADD TABLE public.sales;

-- Team A grants access to Team B's Redshift namespace
GRANT USAGE ON DATASHARE sales_data_share TO NAMESPACE 'team_b_namespace';

-- Team B creates a database from the datashare
CREATE DATABASE shared_sales_data FROM DATASHARE sales_data_share OF
NAMESPACE 'team_a_namespace';
``
```

2.13.2 Registering namespaces to the AWS Glue Data Catalog

- **Purpose:** You can register entire namespaces to the AWS Glue Data Catalog to manage catalogs compatible with Apache Iceberg.

- **Access:** These catalogs can be accessed using any SQL engine that supports the **Apache Iceberg REST API**.

- **Redshift Console:**

- Sign in to the AWS Management Console.
- Open the Amazon Redshift console: <https://console.aws.amazon.com/redshiftv2/>.

- **Navigation:**

- Go to **Redshift Serverless** in the navigation menu.
- In the **Namespaces/Workgroups** section, view or create namespaces.

- **Registering Namespace:**

- Select the namespace you want to register.
- Choose **Actions > Register to AWS Glue Data Catalog**.

- **Registration Details:**

- Enter the **Destination AWS Account ID** (where the catalog will reside).
- Provide a **name** for the namespace in the Data Catalog.

- **Final Steps:**

- Click **Register**.
 - You'll be redirected to the **AWS Lake Formation console**.
 - Follow the catalog creation process there.
-

3 Amazon Redshift provisioned clusters

An Amazon Redshift data warehouse is a collection of computing resources called *nodes*, which are organized into a group called a *cluster*. Each cluster runs an Amazon Redshift engine and contains one or more databases.

◆ Cluster Architecture

- A **Redshift cluster** consists of **nodes**: one **leader node** and one or more **compute nodes**.
- The **leader node**:
 - Receives and parses queries.
 - Develops execution plans.
 - Coordinates parallel execution.
 - Aggregates results and returns them to the client.
- **Compute nodes**:
 - Execute query plans.
 - Share data among themselves.
 - Send intermediate results to the leader node.

◆ Node Types

Amazon Redshift offers two main node types:

1. RA3 Nodes (Recommended for scalability)

- **Managed storage**: Separates compute and storage costs.
- Uses **SSD for fast access** and **Amazon S3 for overflow storage**.
- Ideal for **growing datasets** and **daily data processing**.
- You pay only for the **storage used**, regardless of SSD or S3.
- Launched in a **VPC**.

2. DC2 Nodes (Recommended for small, compute-intensive workloads)

- **Local SSD storage** for high performance.
- Best for **datasets < 1 TB (compressed)**.
- As data grows, add more nodes to scale.
- Less flexible than RA3 for storage scaling.
- Also launched in a **VPC**.

◆ Cluster Configuration

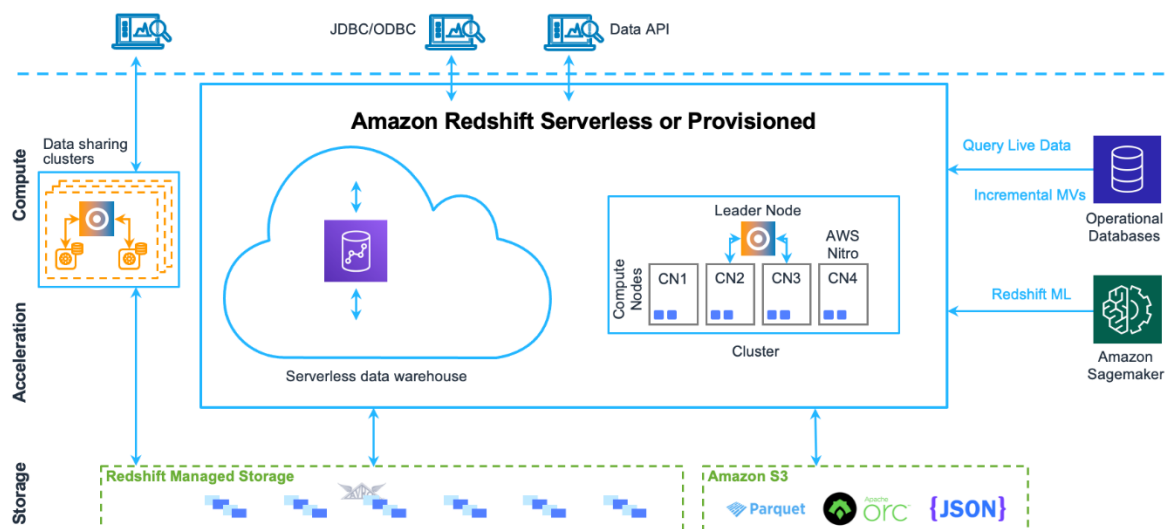
- Use the **“Help me choose”** tool in the Redshift console to get cluster recommendations.
- Node type affects **CPU, RAM, storage capacity, and drive type**.
- Node types come in **different sizes**; size + number of nodes = total cluster storage.

◆ Single-node vs Multi-node Clusters

- **Single-node**: Combines leader and compute roles; not recommended for production.
- **Multi-node**: Separate leader and compute nodes; leader node is same type as compute nodes.
- You **only pay for compute nodes**.

◆ Quotas and Pricing

- AWS applies **quotas** per region (e.g., number of nodes, snapshots).
- Cluster cost depends on:
 - **Region**
 - **Node type**
 - **Number of nodes**
 - **Reservation status**



Node type	vC PU	RA M (Gi B)	Defa ult slice s per nod e	Mana ged stora ge limit per node ¹	Nod e rang e with crea te clus ter	Total manag ed storag e capaci ty ²
ra3.larg e (single- node)	2	16	2	1 TB	1	1 TB ³
ra3.larg e (multi- node)	2	16	2	8 TB	2-16	128 TB
ra3.xlplu s (single- node)	4	32	2	4 TB	1	4 TB ³
ra3.xlplu s (multi- node)	4	32	2	32 TB	2– 32 ⁴	1024 TB ⁴
ra3.4xlar ge	12	96	4	128 TB	2– 32 ⁵	8192 TB ⁵
ra3.16xl arge	48	384	16	128 TB	2– 128	16,384 TB

Node type	vCPU	RAM (GiB)	Default slices per node	Storage per node	Node range	Total capacity
dc2.large	2	15	2	160 GB NVMe -SSD	1–32	5.12 TB
dc2.8xlarge	32	244	16	2.56 TB NVMe -SSD	2–128	326 TB

◆ Upgrade Options to RA3

1. Restore from Snapshot

- Uses the latest snapshot to create a new RA3 cluster.
- Preserves hot block data from DC2 to optimize cache performance.
- You can **rename the new cluster** to match the original for seamless endpoint continuity.

2. Elastic Resize

- Automatically creates a snapshot, provisions a new RA3 cluster, deletes the old one, and renames the new cluster.
- Can be run **on-demand or scheduled**.
- Fast and convenient for upgrading DC2 clusters.

◆ Networking Features of RA3 Nodes

- **Provisioned-cluster VPC endpoint:** Uses ports 5431–5455 or 8191–8215; creates private or elastic IP endpoints.
 - **Single-subnet clusters:** Supported but limited in disaster recovery unless relocation is enabled.
 - **Multi-subnet clusters:** Use subnet groups for better availability and flexibility.
 - **Cross-account/VPC access:** Use Redshift-managed VPC endpoints for secure, private access.
 - **Cluster relocation:** Move clusters across AZs without data loss.
 - **Custom domain name:** Create readable DNS URLs for cluster endpoints.
-

3.1 Cluster operations

With Amazon Redshift, you can create a provisioned cluster to launch a new data warehouse. A provisioned cluster is a collection of computing resources called nodes, which are organized into a single, massively parallel processing (MPP) system.

3.1.1 Creating Cluster

To create a cluster

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. On the navigation menu, choose **Clusters**. The clusters for your account in the current AWS Region are listed. A subset of properties of each cluster is displayed in columns in the list.
3. Choose **Create cluster** to create a cluster.
4. Follow the instructions on the console page to enter the properties for **Cluster configuration**.

If you don't know how large to size your cluster, choose **Help me choose**.

Doing this starts a sizing calculator that asks you questions about the size and query characteristics of the data that you plan to store in your data warehouse. If you know the required size of your cluster (that is, the node type and number of nodes), choose **I'll choose**. Then choose the **Node type** and number of **Nodes** to size your cluster for the proof of concept.

5. In the **Database configuration** section, specify a value for **Admin user name**. For **Admin password**, you can choose from the following options:
 - **Generate a password** – Use a password generated by Amazon Redshift.
 - **Manually add an admin password** – Use your own password.
 - **Manage admin credentials in AWS Secrets Manager** – Amazon Redshift uses AWS Secrets Manager to generate and manage your admin password. Using AWS Secrets Manager to generate and manage your password's secret incurs a fee. For information on AWS Secrets Manager pricing, see [AWS Secrets Manager Pricing](#).
6. (Optional) Follow the instructions on the console page to enter properties for **Cluster permissions**. Provide cluster permissions if your cluster needs to access other AWS services for you, for example to load data from Amazon S3.
7. Choose **Create cluster** to create the cluster. The cluster might take several minutes to be ready to use.

IP address type

Choose the IP address type for your cluster. You can choose to have your resources communicate only over the IPv4 addressing protocol, or choose dual-stack mode, which lets your resources communicate over both IPv4 and IPv6. This feature is only available in the AWS GovCloud (US-East) and AWS GovCloud (US-West) Regions. For more information on AWS Regions, see [Regions and Availability Zones](#).

Virtual private cloud (VPC)

Choose a VPC that has a cluster subnet group. After the cluster is created, the cluster subnet group can't be changed.

Parameter groups

Choose a cluster parameter group to associate with the cluster. If you don't choose one, the cluster uses the default parameter group.

Maintenance track

You can choose whether the cluster version used is the **Current**, **Trailing**, or sometimes **Preview** track.

Monitoring

You can choose whether to create CloudWatch alarms.

Configure cross-region snapshot

You can choose whether to enable cross-Region snapshots.

Automated snapshot retention period

You can choose the number of days to retain these snapshots within 35 days. If the node type is DC2, you can choose zero (0) days to not create automated snapshots.

Manual snapshot retention period

You can choose the number of days or **Indefinitely** to retain these snapshots.

3.1.1 Resizing a cluster

◆ Cluster Resizing Overview

- You can **resize a cluster** to adjust compute and storage based on changing data warehousing needs.
- Resizing involves changing the **number of nodes** or **node type**.
- During resizing:
 - **Only read queries** are allowed.
 - **Write and read/write queries are blocked** until the process completes.

◆ How to Resize a Cluster

1. Sign in to the Amazon Redshift Console.
2. Go to **Clusters** in the navigation menu.
3. Select the cluster you want to resize.
4. Choose **Actions > Resize**.
5. Follow the instructions to:
 - Resize **immediately**.
 - Schedule a **future resize**.
 - Set up **recurring resize operations**.

◆ Upgrading to RA3 Reserved Nodes

- You can upgrade to **RA3 reserved nodes** via:
 - **Snapshot restore**
 - **Elastic resize**
- The console guides you through this process.

◆ Sort Key Conversion During Upgrade

- When upgrading from **DC2.large** to **RA3.large**:
 - **Interleaved sort keys** are automatically converted to **compound sort keys**.
 - This enables **concurrency scaling**, which doesn't support interleaved sort keys.
 - **Impact:** May affect query performance patterns.

◆ Elastic Resize

Elastic Resize allows you to **quickly change the number of nodes** in your cluster. It's ideal for **scaling up or down** based on performance needs.

✓ Key Features

- **Fast:** Typically completes in minutes.
- **Minimal disruption:** Only a brief downtime during the resize.
- **Automated:** Redshift handles snapshot creation, cluster provisioning, and renaming.

Steps:

1. Go to the Amazon Redshift Console.
2. Navigate to **Clusters**.
3. Select your cluster.
4. Choose **Actions > Resize**.
5. On the **Resize cluster** page:
 - Choose **Elastic Resize**.
 - Specify the **new number of nodes**.

- Choose **Resize now** or **Schedule resize**.

⚠ Limitations

- You **cannot change node type** (e.g., DC2 to RA3) using Elastic Resize unless upgrading to RA3.
- Only works for clusters with **uniform node types**.
- **Not suitable for major architectural changes**.

◆ Classic Resize

Classic Resize is used for **changing node types** or **redistributing data** across nodes. It's more flexible but **slower** than Elastic Resize.

✅ Key Features

- Allows **changing node type** (e.g., DC2 to RA3).
- Redistributes data across new nodes.
- Can be used to **increase or decrease** node count.

Steps:

1. Go to the Amazon Redshift Console.
2. Navigate to **Clusters**.
3. Select your cluster.
4. Choose **Actions > Resize**.
5. On the **Resize cluster** page:
 - Choose **Classic Resize**.
 - Select the **new node type** and **number of nodes**.
 - Choose **Resize now** or **Schedule resize**.

⚠ Limitations

- **Longer downtime:** Can take hours depending on data size.
- **No queries allowed** during resizing except **read-only**.
- Requires **manual planning** for performance and cost optimization.

3.1.2 Renaming a cluster

◆ Purpose of Renaming a Cluster

- You can rename a cluster to **change its identifier** (name).
- The **endpoint URL** changes to reflect the new name.
- Applications using the cluster must be updated with the **new endpoint**.

◆ Use Case: Endpoint Reuse

- To **reuse the original cluster name** for a new cluster:
 1. Rename the **original cluster** to a different name.
 2. Rename the **new cluster** to the original name.
- This avoids changing connection settings in applications.

◆ DNS and Availability Impact

- The **old DNS name is deleted** immediately (may remain cached briefly).
- The **new DNS name becomes active within ~10 minutes**.
- The cluster is **unavailable during renaming**.
- The cluster is **rebooted**, and **existing connections are dropped**.
- Stop running queries **before renaming** and restart them **after**.

◆ Snapshots and Monitoring

- **Snapshots are retained** and remain associated with the renamed cluster.
- If you delete the original cluster, you must **manually delete unwanted snapshots**.
- **Amazon CloudWatch alarms** and **Amazon SNS notifications** are tied to the cluster name:
 - Must be **updated manually** after renaming.
 - Update CloudWatch in its console.
 - Update SNS in the Redshift console under **Events**.

◆ Performance Data

- **Load and query history** remains intact.
- **Performance metrics are reset** after renaming.

3.1.3 Pausing and resuming a cluster

- Useful when the cluster is only needed at **specific times**.
- **On-demand billing is suspended** while paused.
- Only **storage charges** apply during pause.

◆ Limitations: You Cannot Pause

- **EC2-Classic clusters**
- Clusters that are **modifying**
- **HSM-enabled clusters**
- Clusters with **automated snapshots turned off**

◆ During Pause

- A **snapshot is created**.
- Queries are **terminated**.
- Cluster enters **pausing state** (takes up to 15 minutes).
- **No queries or monitoring** available.
- **No modifications** allowed.
- **Scheduled actions** (e.g., maintenance, resizing) are skipped.
- **CloudWatch metrics** stop; update alarms accordingly.
- **Audit logs are archived** and not restored.
- **No-backup tables**:
 - Restored on resume for **RA3** nodes.
 - **Not restored** for **DC2** nodes.

◆ During Resume

- Cluster is **updated to the latest maintenance version**.
- **Elastic IP or subnet changes** may occur:
 - May need to update **VPC settings**.
 - New IPs may affect features like `COPY from SSH` or `COPY from EMR`.
- Resume errors occur if the cluster is **not paused**.
- Resume may take **several minutes**.
- **Query performance may be temporarily impacted** due to rehydration.

◆ Other Considerations

- If you **delete a paused cluster** without a final snapshot, it **cannot be restored**.
- **AWS Secrets Manager** secrets are **not deleted** during pause; billing continues.
- **Traces and logs** may not be available for issues before the pause.

3.1.4 Relocating a cluster

◆ Cluster Relocation

- Allows Amazon Redshift to **move a cluster to another Availability Zone (AZ)**.
- Ensures **no data loss** and **no changes to application endpoints**.
- Helps maintain operations during **AZ disruptions** or **resource constraints**.

◆ When and Why Relocation Happens

- Automatically triggered by Amazon Redshift in case of:
 - **Service interruptions** in the current AZ.
 - **Resource constraints** affecting operations like resize or resume.
- Can also be **manually invoked** by users.

- Offered **at no extra cost**.

◆ Supported Instance Types

- **Only supported for RA3 node types.**
- RA3 uses **Redshift Managed Storage (RMS)**, which keeps data **durable and available across AZs**.

◆ Impact on Applications

- **Cluster endpoint remains unchanged** after relocation.
- Applications can **reconnect without modification**.
- A **temporary outage** occurs during relocation and reboot.

◆ How Relocation Works

- Cluster is migrated **behind a proxy** for location-independent access.
- **Reboot occurs** during migration.
- **Enabled by default** for new/restored RA3 clusters with **multi-AZ subnet groups**.

◆ Networking Considerations

- Default port is **5439**, but can be changed to:
 - **5431–5455** or **8191–8215**
- **Do not use ports outside these ranges**—it causes errors.
- If using **leader node IP** or **Enhanced VPC Routing**, switch to using the **VPC endpoint IP**.
 - Find this in the **Network and security** section of the cluster details.
 - Use AWS CLI commands:
 - `describe-vpc-endpoints`
 - `describe-network-interfaces`

◆ Best Practices

- Ensure your **subnet group includes multiple AZs** to support relocation.
- Update **VPC settings** if IP addresses change post-relocation.
- Monitor **cluster health and availability** using CloudWatch and Redshift console.

◆ Managing Relocation via Console

✓ Turn Off Relocation When Creating a Cluster

1. Go to Amazon Redshift Console.
2. Choose **Clusters > Create cluster**.
3. Under **Backup**, set **Cluster relocation** to **Disabled**.
4. Click **Create cluster**.

✓ Modify Relocation for an Existing Cluster

1. Go to **Clusters** in the Redshift console.
2. Select the cluster to modify.
3. Go to the **Maintenance tab**.
4. In **Backup details**, click **Edit**.
5. Set **Cluster relocation** to **Disabled**.
6. Click **Modify cluster**.

✓ Manually Relocate a Cluster

1. Go to **Clusters** in the Redshift console.
2. Select the cluster to relocate.
3. Click **Actions > Relocate**.
4. (Optional) Choose a specific **Availability Zone**.
5. Redshift starts relocation; status changes to **relocating**, then **available** once complete.

3.1.5 Shutting down and deleting a cluster

◆ Why Shut Down a Cluster?

- To **stop incurring charges** when the cluster is no longer needed.
- You can **optionally create a final manual snapshot** before deletion to preserve data.

◆ Snapshot Options

- **Final manual snapshot:**
 - Useful if you plan to **restore the cluster later**.
 - Must be created **before deletion**.
- **No snapshot:**
 - Cluster and data are **permanently deleted**.
- **Automated snapshots** are **deleted** after shutdown.
- **Manual snapshots** are **retained** and billed at **Amazon S3 storage rates** if:
 - No other clusters are running.
 - You exceed the free snapshot storage quota.

3.2 Multi-AZ deployment

3.2.1 Setting up Multi-AZ when creating a new cluster

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.

2. On the navigation menu, choose **Provisioned clusters dashboard**, and choose **Clusters**. The clusters for your account in the current AWS Region are listed. A subset of properties of each cluster is displayed in columns in the list.
 3. Choose the button **Create cluster** to open the create cluster page.
 4. Enter properties for your cluster. For general information about creating clusters, see [Creating a cluster](#).
 5. Choose one of the RA3 node types from the **Node type** drop-down list. The AZ configuration option becomes available only when you chose an RA3 node type.
 6. Under **AZ configuration**, choose **Multi-AZ**.
 7. Under **Number of nodes per AZ**, enter at least two nodes for your cluster.
 8. You have the option to load sample data or bring your own data:
 - In **Sample data**, choose **Load sample data** to load the sample dataset into your Amazon Redshift cluster. Amazon Redshift loads the sample dataset Tackit into the default dev database and public schema. Amazon Redshift automatically loads the sample dataset into your Amazon Redshift cluster. You can start using the query editor v2 to query data.
 - To bring your own data to your Amazon Redshift cluster, follow the steps in [Bringing your own data to Amazon Redshift](#).
 9. Scroll down to **Additional configurations**, expand **Network and security**, and make sure that you either accept the default **Cluster subnet group** or choose another one. If you choose another cluster subnet group, make sure that there are 3 Availability Zones in the subnet group you selected.
 10. Under **Additional configurations**, expand **Database configurations**.
 11. To use a custom AWS KMS key instead of the default AWS-owned key, click **Customize encryption settings** under **Database encryption**.
 12. Under **Choose an KMS key**, you can either choose an AWS Key Management Service key or enter an ARN. Or, you can click **Create an AWS Key Management Service key** in the AWS Key Management Service console. For more information about creating KMS keys, see [Creating Keys](#) in the *AWS Key Management Service Developer Guide*.
 13. Click **Create cluster**. When the cluster creation succeeds, you can view the details in the cluster details page. You can use your SQL client to load and query data.
-

3.2.2 Converting a Single-AZ data warehouse to a Multi-AZ data warehouse

To convert a Single-AZ cluster to a Multi-AZ data warehouse using the console

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. On the navigation menu, choose **Provisioned clusters dashboard**, and choose **Clusters**. The clusters for your account in the current AWS Region are listed. A subset of properties of each cluster is displayed in columns in the list.
3. Choose the cluster you want to convert to a Multi-AZ deployment. The cluster details page appears.
4. For **Actions**, choose **Activate Multi-AZ**. The modification summary appears. Click **Activate Multi-AZ**.
5. When there is an error, do one of the following, then click **Activate Multi-AZ**.
 - Cluster encryption — Choose **Properties** to edit the encryption settings in the Database configuration section under the Properties tab of the cluster details page.
 - Subnet group — Choose **Subnet group** to edit the cluster subnet group settings by clicking the subnet group link. If you choose another cluster subnet group, make sure that there are 3 Availability Zones in the subnet group you selected.
 - Port settings — Choose **Properties** to edit the port setting in the Database configuration section under the Properties tab of the cluster details page.
6. You can use your SQL client to load and query data.

For Convert Multi-Az to single-Az

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. On the navigation menu, choose **Provisioned clusters dashboard**, and choose **Clusters**. The clusters for your account in the current AWS Region are listed. A subset of properties of each cluster is displayed in columns in the list.
3. Choose the cluster you want to convert to a Multi-AZ deployment. The cluster details page appears.
4. For **Actions**, choose **Deactivate Multi-AZ**. The modification summary appears. Click **Deactivate Multi-AZ**.

3.3 Viewing performance data

3.3.1 Viewing cluster performance data

To view performance data

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. On the navigation menu, choose **Clusters**, then choose the name of a cluster from the list to open its details. The details of the cluster are displayed, which can include **Cluster performance**, **Query monitoring**, **Databases**, **Datashares**, **Schedules**, **Maintenance**, and **Properties** tabs.
3. Choose the **Cluster performance** tab for performance information including the following:
 - **CPU utilization**
 - **Percentage disk space used**
 - **Database connections**
 - **Health status**
 - **Query duration**
 - **Query throughput**
 - **Concurrency scaling activity**

Many more metrics are available. To see the available metrics and choose which are displayed, choose the **Preferences** icon.

3.3.2 Viewing query history data

 Steps to Access Query History:

1. **Sign in** to the AWS Management Console.
2. Open the **Amazon Redshift console**: <https://console.aws.amazon.com/redshiftv2/>
3. In the **navigation menu**, choose **Clusters**.
4. Select your **cluster name** to view its details.
5. Go to the **Query monitoring** tab.
6. Click on the **Query history** tab.

 Query Monitoring Views:

Query List View:

- **Query runtime graph:**
 - Shows query activity over time.
 - Each bar = one query; length = runtime.
 - Filter by status: running, completed, loads, etc.

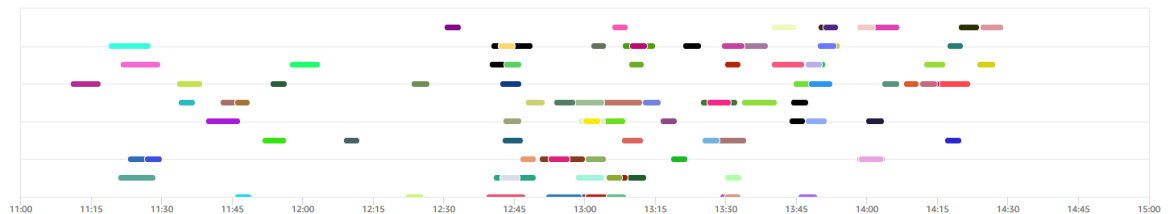
- Top 100 longest-running queries shown by default.
- **Queries and loads list:**
 - Displays all queries and loads.
 - Option to **terminate** running queries.

Tips for Using Query History Graphs:

- Click a **bar** in the Query runtime chart to view query details.
- Click a **query ID** in the list for more info.
- **Swipe** to zoom into a specific time period.
- **Page forward** through all pages to see complete data.
- Use the **settings gear icon** to customize columns and rows.
- You can also access **Queries and loads** from the left navigation pane.
- **Query runtime**

Query runtime

The query activity on a timeline. Use this graph to see which queries are running in the same timeframe. Choose a query to view more query execution details.



● **Queries and loads**

Queries and loads(100)

Filter queries

↺

1

2

↻

⌵

Start time

⌵

Query

⌵

Status

⌵

Duration

⌵

SQL

⌵

Copy SQL

User

⌵

Transaction ID

⌵

⌵

Apr 13th, 2020 01:00:55 PM
8 days ago

69248

✔ Completed

11 min

with /* query_templates/query67.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_10cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

105501

⌵

Apr 13th, 2020 12:58:07 PM
8 days ago

69199

✔ Completed

11 min

with /* query_templates/query67.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_10cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

105414

⌵

Apr 13th, 2020 12:54:15 PM
8 days ago

69111,69265,69253

✔ Completed

10 min

with /* query_templates/query22.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_10cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

105283

⌵

Apr 13th, 2020 12:50:17 PM
8 days ago

68976

✔ Completed

10 min

with /* query_templates/query67.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_10cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

105128

⌵

Apr 13th, 2020 01:29:23 PM
8 days ago

70089

✔ Completed

10 min

with /* query_templates/query67.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_10cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

106659

⌵

Apr 13th, 2020 11:18:35 AM
8 days ago

65543

✔ Completed

9 min

with /* query_templates/query67.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_05cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

101092

⌵

Apr 13th, 2020 12:40:30 PM
8 days ago

68729

✔ Completed

9 min

with /* query_templates/query67.tpl0 ICF:IR-09c6a4cc-6ec8-11e
a-8047-06872b3fecc8.stream_10cu_run01_nocache.stream-quer ...

📄

Copy

rsperf

104789

To display query performance data

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. On the navigation menu, choose **Queries and loads** to display the list of queries for your account.

By default, the list displays queries for all your clusters over the past 24 hours. You can change the scope of the displayed date in the console.

4 Zero-ETL integrations

Zero-ETL integration in Amazon Redshift is a **fully managed solution** that allows you to bring **operational and transactional data** into Redshift **without building or maintaining traditional ETL pipelines**. Here's a concise breakdown of its key features and benefits:

Zero-ETL Integration

- **Zero-ETL** means **no need for manual Extract, Transform, Load (ETL)** processes.
 - Data is **automatically replicated** from supported sources to Amazon Redshift.
 - Enables **real-time or near real-time analytics** on fresh data.
-

Key Benefits:

- **No pipeline maintenance** – AWS handles data replication and transformation.
 - **Fresher data** for analytics, dashboards, and AI/ML workloads.
 - **Faster insights** – reduced latency compared to traditional ETL.
 - **Improved data quality monitoring** and customer experience.
 - **Secure and scalable** – supports encryption and large-scale data processing.
-

Supported Data Sources:

- **Amazon Aurora MySQL & PostgreSQL**
 - **Amazon RDS (MySQL, PostgreSQL, Oracle)**
 - **Amazon DynamoDB**
 - **Oracle Database@AWS**
 - **Applications** like Salesforce, SAP, ServiceNow, Meta Ads, Zendesk, etc.
-

How It Works:

1. **Configure source** (e.g., Aurora, RDS).
2. **Specify target** – Amazon Redshift cluster or Serverless namespace.
3. **Initial data load** happens.
4. **Ongoing replication** keeps Redshift updated.
5. **Query data directly** in Redshift using SQL.

✓ Prerequisites for Target Redshift Warehouse:

- Must be **Amazon Redshift Serverless** or use **RA3 node type**.
- Must be **encrypted** (for provisioned clusters).
- Must have **case sensitivity enabled**.

```
aws redshift-serverless update-workgroup \  
    --workgroup-name target-workgroup \  
    --config-parameters  
parameterKey=enable_case_sensitive_identifier,parameterValue=true
```

In Console:

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. In the left navigation pane, choose **Provisioned clusters dashboard**.
3. Choose the provisioned cluster that you want to replicate data into.
4. In the left navigation pane, choose **Configurations > Workload management**.
5. On the workload management page, choose the parameter group.
6. Choose the **Parameters** tab.
7. Choose **Edit parameters**, then
change **enable_case_sensitive_identifier** to **true**.
8. Then, choose **Save**.

For provisioned Cluster:

Turn on case sensitivity for Amazon Redshift provisioned clusters using the AWS CLI

1. Because you can't edit the default parameter group, from your terminal program, run the following AWS CLI command to create a custom parameter group. Later, you will associate it with the provisioned cluster.

```
2. aws redshift create-cluster-parameter-group \  
3.     --parameter-group-name zero-etl-params \  
4.     --parameter-group-family redshift-2.0 \  
    --description "Param group for zero-ETL integrations"
```


5. Run the following AWS CLI command to turn on case sensitivity for the parameter group.

```
6. aws redshift modify-cluster-parameter-group \
```

```
7.     --parameter-group-name zero-etl-params \
```

```
    --parameters  
    ParameterName=enable_case_sensitive_identifier,ParameterValue=  
    true
```

8. Run the following command to associate the parameter group with the cluster.

```
9. aws redshift modify-cluster \
```

```
10.    --cluster-identifier target-cluster \
```

```
    --cluster-parameter-group-name zero-etl-params
```

11. Wait for the provisioned cluster to be available. You can check the status of the cluster by using the `describe-cluster` command. Then, run the following command to reboot the cluster.

```
12. aws redshift reboot-cluster \
```

```
    --cluster-identifier target-cluster
```

 Integration Behavior & Limitations:

- **Deleting a source** causes integrations to enter **FAILED** state, but previously replicated data remains queryable.
- **Destination database is read-only:**
 - You **cannot create** tables, views, or materialized views in it.
 - You **can use materialized views** on other tables or via cross-database queries.
- Only **Synced state** tables are queryable by default.
 - To query other states, set `QUERY_ALL_STATES = TRUE`.

 Technical Constraints:

- Redshift only supports **UTF-8 characters**.
 - Collation rules may differ from source DB.
- **Max 50 integrations** per Redshift data warehouse.
- **Source tables must have a primary key**.
 - No replication without primary keys.

- **No transformations** during replication.
 - You can transform data **after replication** in Redshift.
- **Parallel connections** are used for syncing.
 - **Concurrency scaling does not apply** to sync writes.
 - It **does apply** to reads from synced objects.

Replication Control:

- Use `REFRESH_INTERVAL` to control replication frequency.
- For **DynamoDB integrations**, tables are created under the `public` schema in the destination DB (e.g., `ddb_rs_customerprofiles_zetl_db`).

Advanced Features:

- **Data filtering** is supported for **Aurora integrations**.
 - You can include/exclude specific tables.
- **Aurora PostgreSQL** supports up to **100 databases** replicated independently.

4.1 Configure authorization for your Amazon Redshift data warehouse

To replicate data from your integration source into your Amazon Redshift data warehouse, you must initially add the following two entities:

- *Authorized principal* – identifies the user or role that can create zero-ETL integrations into the data warehouse.
- *Authorized integration source* – identifies the source database that can update the data warehouse.

Add authorized principals

To create a zero-ETL integration into your Redshift Serverless workgroup or provisioned cluster, authorize access to the associated namespace or provisioned cluster.

You can skip this step if both of the following conditions are true:

- The AWS account that owns the Redshift Serverless workgroup or provisioned cluster also owns the source database.
- That principal is associated with an identity-based IAM policy with permissions to create zero-ETL integrations into this Redshift Serverless namespace or provisioned cluster.

Add authorized principals to an Amazon Redshift Serverless namespace

1. In the Amazon Redshift console, in the left navigation pane, choose **Redshift Serverless**.
2. Choose **Namespace configuration**, then choose your namespace, and go to the **Resource Policy** tab.
3. Choose **Add authorized principals**.
4. For each authorized principal that you want to add, enter into the namespace either the ARN of the AWS user or role, or the ID of the AWS account that you want to grant access to create zero-ETL integrations. An account ID is stored as an ARN.
5. Choose **Save changes**.

Add authorized principals to an Amazon Redshift provisioned cluster

1. In the Amazon Redshift console, in the left navigation pane, choose **Provisioned clusters dashboard**.
2. Choose **Clusters**, then choose the cluster, and go to the **Resource Policy** tab.
3. Choose **Add authorized principals**.
4. For each authorized principal that you want to add, enter into the cluster either the ARN of the AWS user or role, or the ID of the AWS account that you want to grant access to create zero-ETL integrations. An account ID is stored as an ARN.
5. Choose **Save changes**.

Add authorized integration sources

To allow your source to update your Amazon Redshift data warehouse, you must add it as an authorized integration source to the namespace.

Add an authorized integration source to an Amazon Redshift Serverless namespace

1. In the Amazon Redshift console, go to **Serverless dashboard**.
2. Choose the name of the namespace.
3. Go to the **Resource Policy** tab.
4. Choose **Add authorized integration source**.
5. Specify the ARN of the source for the zero-ETL integration.

Add an authorized integration source to an Amazon Redshift provisioned cluster

1. In the Amazon Redshift console, go to **Provisioned clusters dashboard**.
 2. Choose the name of the provisioned cluster.
 3. Go to the **Resource Policy** tab.
 4. Choose **Add authorized integration source**.
 5. Specify the ARN of the source that's the data source for the zero-ETL integration.
-

4.2 Create a zero-ETL integration for Aurora

Step 1: Create a custom DB cluster parameter group

Aurora zero-ETL integrations require specific values for the DB cluster parameters that control replication. Specifically, Aurora MySQL requires *enhanced binlog* (`aurora_enhanced_binlog`), and Aurora PostgreSQL requires *enhanced logical replication* (`aurora.enhanced_logical_replication`).

To configure binary logging or logical replication, you must first create a custom DB cluster parameter group, and then associate it with the source DB cluster.

Aurora MySQL (aurora-mysql8.0 family):

- `aurora_enhanced_binlog=1`
- `binlog_backup=0`
- `binlog_format=ROW`
- `binlog_replication_globaldb=0`
- `binlog_row_image=full`
- `binlog_row_metadata=full`

In addition, make sure that the `binlog_transaction_compression` parameter is *not* set to `ON`, and that the `binlog_row_value_options` parameter is *not* set to `PARTIAL_JSON`.

For more information about Aurora MySQL enhanced binlog, see [Setting up enhanced binlog for Aurora MySQL](#).

Aurora PostgreSQL (aurora-postgresql16 family):

- `rds.logical_replication=1`
- `aurora.enhanced_logical_replication=1`

- `aurora.logical_replication_backup=0`
- `aurora.logical_replication_globaldb=0`

Enabling enhanced logical replication (`aurora.enhanced_logical_replication`) will always write all column values to the write ahead log (WAL) even if `REPLICA IDENTITY FULL` isn't enabled. This might increase the IOPS for your source DB cluster.

Step 2: Select or create a source DB cluster

To create an Aurora DB cluster using the console

1. Sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.
2. In the upper-right corner of the AWS Management Console, choose the AWS Region in which you want to create the DB cluster.

Aurora is not available in all AWS Regions. For a list of AWS Regions where Aurora is available, see [Region availability](#).
3. In the navigation pane, choose **Databases**.
4. Choose **Create database**.
5. For **Choose a database creation method**, choose **Standard create**.
6. For **Engine type**, choose one of the following:
 - **Aurora (MySQL Compatible)**
 - **Aurora (PostgreSQL Compatible)**

Engine options

Engine type [Info](#)

☒ Aurora (MySQL Compatible)


☐ Aurora (PostgreSQL Compatible)


☐ MySQL


☐ MariaDB


☐ PostgreSQL


☐ Oracle


☐ Microsoft SQL Server


☐ IBM Db2


7. Choose the **Engine version**.3.0.5 or above.
8. In **Templates**, choose the template that matches your use case.
9. To enter your master password, do the following:
 - a. In the **Settings** section, expand **Credential Settings**.
 - b. Clear the **Auto generate a password** check box.
 - c. (Optional) Change the **Master username** value and enter the same password in **Master password** and **Confirm password**.

By default, the new DB instance uses an automatically generated password for the master user.

10. In the **Connectivity** section under **VPC security group (firewall)**, if you select **Create new**, a VPC security group is created with an inbound rule that allows your local computer's IP address to access the database.
11. For securing Groups add Aurora/mysql port-no:3306

12. For **Cluster storage configuration**, choose either **Aurora I/O-Optimized** or **Aurora Standard**. For more information, see [Storage configurations for Amazon Aurora DB clusters](#).

13. For the remaining sections, specify your DB cluster settings. For information about each setting, see [Settings for Aurora DB clusters](#).

14. Choose **Create database**.

If you chose to use an automatically generated password, the **View credential details** button appears on the **Databases** page.

To view the master user name and password for the DB cluster, choose **View credential details**.

To connect to the DB instance as the master user, use the user name and password that appear.

15. For **Databases**, choose the name of the new Aurora DB cluster.

On the RDS console, the details for new DB cluster appear. The DB cluster and its DB instance have a status of **creating** until the DB cluster is ready to use.

DB Identifier	Role	Engine	Region & AZ	Size	Status	Actions
database-test1	Regional cluster	Aurora MySQL	us-east-1	1 Instance	Available	-
database-test1-instance-1	Writer Instance	Aurora MySQL	-	db.r6g.large	Creating	-

When the state changes to **available** for both, you can connect to the DB cluster. Depending on the DB instance class and the amount of storage, it can take up to 20 minutes before the new DB cluster is available.

To view the newly created cluster, choose **Databases** from the navigation pane in the Amazon RDS console. Then choose the DB cluster to show the DB cluster details. For more information, see [Viewing an Amazon Aurora DB cluster](#).

The screenshot shows the Amazon RDS console interface for a database cluster named 'database-test1'. The 'Endpoints (2)' section is expanded, displaying a table of endpoints. The 'Writer instance' endpoint is highlighted with a red circle, and its port '3306' is also circled in red.

Endpoint name	Status	Type	Port
database-test1.cluster-ro-123456789012.us-west-1.rds.amazonaws.com	Available	Reader instance	3306
database-test1.cluster-123456789012.us-west-1.rds.amazonaws.com	Available	Writer instance	3306

On the **Connectivity & security** tab, note the port and the endpoint of the writer DB instance. Use the endpoint and port of the cluster in your JDBC and ODBC connection strings for any application that performs write or read operations.

When you create the database, under **Additional configuration**, change the default **DB cluster parameter group** to the custom parameter group that you created in the previous step.

Step 3: Create a target data warehouse

After you create your source DB cluster, you must create and configure a target data warehouse. The data warehouse must meet the following requirements:

- Using an RA3 node type with at least two nodes, or Redshift Serverless.
- Encrypted (if using a provisioned cluster).
- Enable case sensitivity on the data warehouse

Step 4: Creating Aurora zero-ETL integrations with Amazon Redshift

RDS console

To create a zero-ETL integration

1. Sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.
2. In the left navigation pane, choose **Zero-ETL integrations**.
3. Choose **Create zero-ETL integration**.
4. For **Integration identifier**, enter a name for the integration. The name can have up to 63 alphanumeric characters and can include hyphens.

Important

Catalog names are limited to 19 characters in length. Ensure your integration identifier meets this requirement if it will be used as a catalog name.

5. Choose **Next**.
6. For **Source**, select the Aurora DB cluster where the data will originate from.
Modifying DB cluster parameters requires a reboot. Before you can create the integration, the reboot must be complete and the new parameter values must be successfully applied to the cluster.
7. (Optional) Select **Customize data filtering options** and add data filters to your integration. You can use data filters to define the scope of replication to the target data warehouse. For more information, see [Data filtering for Aurora zero-ETL integrations](#).
8. Once your source DB cluster is successfully configured, choose **Next**.
9. For **Target**, do the following:
 - a. (Optional) To use a different AWS account for the Amazon Redshift target, choose **Specify a different account**. Then, enter the ARN of an IAM role with permissions to display your data warehouses. For

instructions to create the IAM role, see [Choosing a target data warehouse in a different account](#).

- b. For **Amazon Redshift data warehouse**, select the target for replicated data from the source DB cluster. You can choose a provisioned Amazon Redshift *cluster* or a Redshift Serverless *namespace* as the target.

Modifying case sensitivity for a *provisioned* Redshift cluster requires a reboot. Before you can create the integration, the reboot must be complete and the new parameter value must be successfully applied to the cluster.

If your selected source and target are in different AWS accounts, then Amazon RDS cannot fix these settings for you. You must navigate to the other account and fix them manually in Amazon Redshift.

10. Once your target data warehouse is configured correctly, choose **Next**.
11. For **Encryption**, specify how you want your integration to be encrypted. By default, RDS encrypts all integrations with an AWS owned key. To choose a customer managed key instead, enable **Customize encryption settings** and choose a KMS key to use for encryption. For more information, see [Encrypting Amazon Aurora resources](#).

Optionally, add an encryption context. For more information, see [Encryption context](#) in the *AWS Key Management Service Developer Guide*.

12. Review your integration settings and choose **Create zero-ETL integration**.

The integration has a status of **Creating** while it's being created, and the target Amazon Redshift data warehouse has a status of **Modifying**. During this time, you can't query the data warehouse or make any configuration changes on it.

Step 5: Add Filters in zero ETL Integration

To add data filters to a zero-ETL integration

1. Sign in to the AWS Management Console and open the Amazon RDS console at <https://console.aws.amazon.com/rds/>.
2. In the navigation pane, choose **Zero-ETL integrations**. Select the integration that you want to add data filters to, and then choose **Modify**.
3. Under **Source**, add one or more **Include** and **Exclude** statements.

The following image shows an example of data filters for a MySQL integration:

Source

Source database

The source database where the data is replicated from. Only databases running the supported versions are available.

my-database

Browse RDS databases

Data filtering options - optional

Info

Include or exclude any existing and future database table that matches your entered list of filter expressions. All tables are included by default.

☒ Customize data filtering options

Choose filter type

Include

Exclude

Filter expression

mydb.mytable, mydb./table_\d+/

Remove

Remove

Each filter expression must be a comma-separated list of patterns. Each pattern can have a maximum of 256 characters. You can include a maximum of 100 total patterns. Filters are evaluated in the order they appear (left to right, top to bottom).

Add filter

- When you're satisfied with the changes, choose **Continue** and **Save changes**.

Filter type	An <code>Include</code> filter type <i>includes</i> all tables that match one of the patterns in the filter expression. An <code>Exclude</code> filter type <i>excludes</i> all tables that match one of the patterns.
-------------	--

Filter logic

If you don't specify any data filters in your integration, Aurora assumes a default filter of `include:*.*`, which replicates all tables to the target data warehouse. However, if you add at least one filter, the default logic switches to `exclude:*.*`, which excludes all tables by default. This lets you explicitly define which databases and tables to include in replication.

For example, if you define the following filter:

```
'include: db.table1, include: db.table2'
```

Page 59 of 164

Aurora evaluates the filter as follows:

```
'exclude: *.* , include: db.table1, include: db.table2'
```

Therefore, Aurora only replicates `table1` and `table2` from the database named `db` to the target data warehouse.

- Include all databases and all tables:

```
'include: *.*'
```

- Include all tables within the `books` database:

```
'include: books.*'
```

- Exclude any tables named `mystery`:

```
'include: *.* , exclude: *.mystery'
```

- Include all tables in the `books` database, except for those containing the substring `mystery`:

```
'include: books.* , exclude: books./.*mystery.*/'
```

- Include all tables in the `books` database, except those starting with `mystery`:

```
'include: books.* , exclude: books./mystery.*/'
```

- Include all tables in the `books` database, except those ending with `mystery`:

```
'include: books.* , exclude: books./.*mystery/'
```

To create Destination Database:

1. In the left navigation pane, choose **Zero-ETL integrations**.
2. From the integration list, choose an integration.
3. If you're using a provisioned cluster, you must first connect to the database. Choose **Connect to database**. You can connect using a recent connection, or by creating a new connection.
4. To create a database from the integration, choose **Create database from integration**.
5. Enter a **Destination database name**. The **Integration ID** and **Data warehouse name** are pre-populated.

For Aurora PostgreSQL, RDS for PostgreSQL, or RDS for Oracle sources, enter the **Source named database** that you specified when creating your zero-ETL integration. In these cases, you can map a maximum of 100 source databases to Amazon Redshift databases.

6. Choose **Create database**.

5 Interacting with Amazon Q generative SQL

To interact with the Amazon Q generative SQL chat to generate SQL

1. In the **Editor** tab of the query editor v2, open a notebook.
2. Choose the  Generative SQL icon, then follow the directions to ask your questions of the Amazon Redshift query editor v2 generative SQL in the chat panel.

You provide questions in a prompt field and Amazon Q generative SQL responds with suggested SQL. Any errors encountered are returned to you in the chat panel.

3. Choose **Add to notebook** to add a Markdown cell with your prompt and a SQL cell with the suggested SQL to your notebook.
4. (Optional) Provide feedback regarding the SQL generated by choosing the  helpful feedback icon or the  not helpful feedback icon. You can categorize not helpful feedback as `Incorrect tables/columns`, `Incorrect predicates/literals/group bys`, `Incorrect SQL structure`, or `Other`. In addition, you can provide some free-form text with your feedback about the accuracy of the SQL.
5. (Optional) Choose **Regenerate SQL** to generate another response for the same prompt. You can choose to **Regenerate SQL** one time for the current prompt.
6. (Optional) In the generative SQL chat panel, choose the  **More** icon, then choose **Refresh database** to refresh the metadata describing your connected database. This metadata includes the definitions of schemas, tables, and columns in your database.

Link -[Amazon Q generative SQL in Amazon Redshift Query Editor Public Preview Demo | Amazon Web Services - YouTube](#)

6 Loading data from Amazon S3

To load data into an existing table

1. Confirm that the table is already created in the database where you want to load data.
2. Confirm that you are connected to the target database in the tree-view panel of query editor v2 before continuing. You can create a connection using the context menu (right-click) to the cluster or workgroup where the data will be loaded.

Choose  **Load data**.

3. For **Data source**, choose **Load from S3 bucket**.
4. In **S3 URIs**, choose **Browse S3** to look for the Amazon S3 bucket that contains the data to load.
5. If the specified Amazon S3 bucket isn't in the same AWS Region as the target table, then choose the **S3 file location** for the AWS Region where the data is located.
6. Choose **This file is a manifest file** if the Amazon S3 file is actually a manifest containing multiple Amazon S3 bucket URIs.
7. Choose the **File format** for the file to be uploaded. The supported data formats are CSV, JSON, DELIMITER, FIXEDWIDTH, SHAPEFILE, AVRO, PARQUET, and ORC. Depending on the specified file format, you can choose the respective **File options**. You can also select **Data is encrypted** if the data is encrypted and enter the Amazon Resource Name (ARN) of the KMS key used to encrypt the data.

If you choose CSV or DELIMITER, you can also choose the **Delimiter character** and whether to **Ignore header rows** if the specified number of rows are actually column names and not data to load.

8. Choose a compression method to compress your file. The default is no compression.
9. (Optional) The **Advanced settings** support various **Data conversion parameters** and **Load operations**. Enter this information as needed for your file.

For more information about data conversion and data load parameters, see [Data conversion parameters](#) and [Data load operations](#) in the *Amazon Redshift Database Developer Guide*.

10. Choose **Next**.
11. Choose **Load existing table**.

12. Confirm or choose the location of the **Target table** including **Cluster or workgroup**, **Database**, **Schema**, and **Table** name where the data is loaded.
13. Choose an **IAM role** that has the required permissions to load data from Amazon S3.
14. (Optional) Choose column names to enter them in **Column mapping** to map columns in the order of the input data file.
15. Choose **Load data** to start the data load.

When the load completes, the query editor displays with the generated COPY command that was used to load your data. The **Result** of the COPY is shown. If successful, you can now use SQL to select data from the loaded table.

To load data into a new table

1. Confirm that you are connected to the target database in the tree-view panel of query editor v2 before continuing. You can create a connection using the context menu (right-click) to the cluster or workgroup where the data will be loaded.
Choose  **Load data**.
2. For **Data source**, choose **Load from S3 bucket**.
3. In **S3 URIs**, choose **Browse S3** to look for the Amazon S3 bucket that contains the data to load.
4. If the specified Amazon S3 bucket isn't in the same AWS Region as the target table, then choose the **S3 file location** for the AWS Region where the data is located.
5. Choose **This file is a manifest file** if the Amazon S3 file is actually a manifest containing multiple Amazon S3 bucket URIs.
6. Choose the **File format** for the file to be uploaded. The supported data formats are CSV, JSON, DELIMITER, FIXEDWIDTH, SHAPEFILE, AVRO, PARQUET, and ORC. Depending on the specified file format, you can choose the respective **File options**. You can also select **Data is encrypted** if the data is encrypted and enter the Amazon Resource Name (ARN) of the KMS key used to encrypt the data.

If you choose CSV or DELIMITER, you can also choose the **Delimiter character** and whether to **Ignore header rows** if the specified number of rows are actually column names and not data to load.

7. Choose a compression method to compress your file. The default is no compression.

8. (Optional) The **Advanced settings** support various **Data conversion parameters** and **Load operations**. Enter this information as needed for your file.

For more information about data conversion and data load parameters, see [Data conversion parameters](#) and [Data load operations](#) in the *Amazon Redshift Database Developer Guide*.

9. Choose **Next**.

10. Choose **Load new table**.

The table columns are inferred from the input data. You can modify the definition of the table schema by adding columns and table details. To revert to the query editor v2 inferred table schema, choose **Restore to defaults**.

11. Confirm or choose the location of the **Target table** including **Cluster or workgroup**, **Database**, and **Schema** where the data is loaded. Enter a **Table** name to be created.

12. Choose an **IAM role** that has the required permissions to load data from Amazon S3.

13. Choose **Create table** to create the table using the definition shown.

A review summary of the table definition is displayed. The table is created in the database. To later delete the table, run a DROP TABLE SQL command. For more information, see [DROP TABLE](#) in the *Amazon Redshift Database Developer Guide*.

14. Choose **Load data** to start the data load.

In this new table we edit the datatype and change the name of column.

Video link -[How to load S3 data to Redshift | Create Redshift table from CSV file in S3](#)

We can also upload from our local file to redshift. we choose data source as local file.

7. Creating a notebook

To create a notebook

1. From the navigator menu, choose the Editor icon ().
2. Choose the plus icon (), and then choose **Notebook**.

By default, a SQL query cell appears in the notebook.

3. In the SQL query cell, do one of the following:

- Enter a query.
 - Paste a query that you copied.
4. (Optionally) Choose the plus icon (), then choose **Markdown** to add a Markdown cell where you can provide descriptive or explanatory text using standard Markdown syntax.
 5. (Optionally) Choose the plus icon (), then choose **SQL** to insert a SQL cell.

You can rename notebooks using the pencil icon ().

From the menu icon (), you can also perform the following operations on a notebook:

-  **Share with my team** – To share the notebook with your team as defined by tags. To share a notebook with your team, make sure that you have the principal tag `sqlworkbench-team` set to the same value as the rest of your team members in your account. For example, an administrator might set the value to `accounting-team` for everyone in the accounting department. For an example, see [Permissions required to use the query editor v2](#).
-  **Export** – To export the notebook to a local file with the `.ipynb` extension.
-  **Import query** – To import a query from a local file into a cell in the notebook. You can import files with `.sql` and `.txt` extensions.
-  **Save version** – To create a version of the notebook. To see versions of a notebook, navigate to your saved notebooks and open **Version history**.
-  **Duplicate** – To create a copy of the notebook and open it in a new notebook tab.
-  **Shortcuts** – To display the shortcuts available when authoring a notebook.

Video Link - <https://youtu.be/GNahyu7j98M>

8 Querying the AWS Glue Data Catalog

Prerequisites & Compatibility

- **Only supported** on:
 - Amazon Redshift **RA3 node type clusters**
 - Amazon Redshift **Serverless**
- By default, the AWS Glue Data Catalog is listed as a query editor v2 database named `awsdatacatalog`.

- Not available in **all AWS Regions** — use `SHOW data_catalog_auto_mount;` to check availability.

Configuration Commands

- **Check if Glue is mounted:** `SHOW datacatalogauto_mount;`
- **Enable Glue auto-mount:** `ALTER SYSTEM SET datacatalogauto_mount = on;`
 - Requires **cluster reboot** or **serverless pause/resume** to take effect.

Exploring Glue Catalog

- **List Glue databases (schemas):** `SHOW SCHEMAS FROM DATABASE awsdatalog;`
- **List tables in a Glue schema:** `SHOW TABLES FROM SCHEMA awsdatalog.myschema;`
- **List columns in a Glue table:** `SHOW COLUMNS FROM TABLE awsdatalog.myglue.mytable;`

Granting Access

- Grant usage to an **IAM user**: `GRANT USAGE ON DATABASE awsdatalog TO "IAM:myIAMUser";`
- Or to an **IAM role**: `GRANT USAGE ON DATABASE awsdatalog TO "IAMR:myIAMRole";`

Authentication Requirements

- **Provisioned cluster:** Use **Temporary credentials** with IAM identity.
- **Serverless workgroup:** Use **Federated user** authentication.

Querying Glue Data

- Use **three-part notation**: `SELECT * FROM awsdatalog.<glue-db>.<glue-table>;`
- Example **join query**:

```
SELECT pn.empid, alias, role, projectname
FROM awsdatalog.empldb.projectnametable pn,
      awsdatalog.empldb.projectaliastable pa
WHERE pn.empid = pa.empid;
```

- Example **create Redshift table from Glue data**:

```
CREATE TABLE dev.public.glue AS

SELECT pn.empid, alias, role, projectname

FROM awsdatacatalog.empldb.projectnametable pn,

     awsdatacatalog.empldb.projectaliastable pa

WHERE pn.empid = pa.empid;
```

9 Scheduled queries with query editor v2

With Amazon Redshift query editor v2, you can automate SQL queries to run on a schedule. Scheduled queries are SQL statements that run automatically at specified times or intervals, letting you efficiently manage recurring data operations and analytics tasks. You might want to schedule queries if you're seeking to streamline batch processing, generate regular reports, or maintain data pipelines within their Amazon Redshift environment.

To create a schedule to run a SQL statement

1. On the **Editor**  view, choose  **Schedule** to create a schedule to run a SQL statement.
2. When you define the schedule, you provide the following information.
 - The IAM role that assumes the required permissions to run the query. This IAM role is also attached to your cluster or workgroup.
 - The authentication values for either AWS Secrets Manager or temporary credentials to authorize access your cluster or workgroup. These authentication methods are supported by the Data API. For more information, see [Authenticating a scheduled query](#).
 - The cluster or workgroup where your database resides.
 - The name of the database that contains the data to be queried.
 - The name of the scheduled query and its description. The query editor v2 prefixes the scheduled query name you provide with "QS2-". The query editor v1 prefixes its scheduled query names with "QS-".
 - The SQL statement to be run on the schedule.
 - The schedule frequency and repeat options or a cron formatted value that defines the schedule. For more information, see [Cron Expressions](#) in the *Amazon CloudWatch Events User Guide*.
 - Optionally, you can enable standard Amazon SNS notifications to monitor the scheduled query. You might need to confirm the email address you provide to the Amazon SNS notification. Check your email

for a link to confirm the email address for the Amazon SNS notification. For more information, see [Email notifications](#) in the *Amazon Simple Notification Service Developer Guide*. If your query is being run but you don't see messages published in your SNS topic, see [My rule runs, but I don't see any messages published into my Amazon SNS topic](#) in the *Amazon EventBridge User Guide*.

3. Choose **Schedule query** to save and activate the schedule and add the schedule to the list of queries in the **Scheduled queries** view.

The **Scheduled queries**  view lists all the scheduled queries for your clusters and workgroups. With this view, you can display schedule query details, activate or deactivate the schedule, edit the schedule, and delete the scheduled query. When you view query details, you can also view the history of running the query with the schedule.

Video link - <https://youtu.be/gTw0XUpO8sw>

For setting up the policy

- The following example shows a permission policy that assumes the IAM role `myRedshiftRole` in account `123456789012`. The IAM role `myRedshiftRole` is also the IAM role that is attached to the cluster or workgroup where the scheduled query runs.
 - JSON

```
• {
•   "Version": "2012-10-17",
•   "Statement": [
•     {
•       "Sid": "AssumeIAMRole",
•       "Effect": "Allow",
•       "Action": "sts:AssumeRole",
•       "Resource": [
•         "arn:aws:iam::123456789012:role/myRedshiftRole"
```

```

    • ]
    • }
    • ]
    • }

```

Update the trust policy of the IAM role used to schedule the query to allow the IAM user to assume it.

```

{
    "Sid": "AssumeRole",
    "Effect": "Allow",
    "Principal": {
        "AWS":
"arn:aws:iam::123456789012:user/myIAMusername"
    },
    "Action": "sts:AssumeRole"
}
]
}

```

For the IAM role that you specify to allow the scheduled query to run, edit the IAM role using the IAM console (<https://console.aws.amazon.com/iam/>).

- Attach the `AmazonRedshiftDataFullAccess` and `AmazonEventBridgeFullAccess` AWS managed policies to the IAM role. The `AmazonRedshiftDataFullAccess` managed policy only allows `redshift-serverless:GetCredentials` permission for Redshift Serverless workgroups that are tagged with the key `RedshiftDataFullAccess`.

10 Amazon Redshift Python connector

To install the Python connector from the Python Package Index (PyPI), you can use pip. To do this, run the following command.

```
>>> pip install redshift_connector
```

You can install the connector within a virtual environment. To do this, run the following command.

```
>>> pip install redshift_connector
```

Optionally, you can install pandas and NumPy with the connector.

```
>>> pip install 'redshift_connector[full]'
```

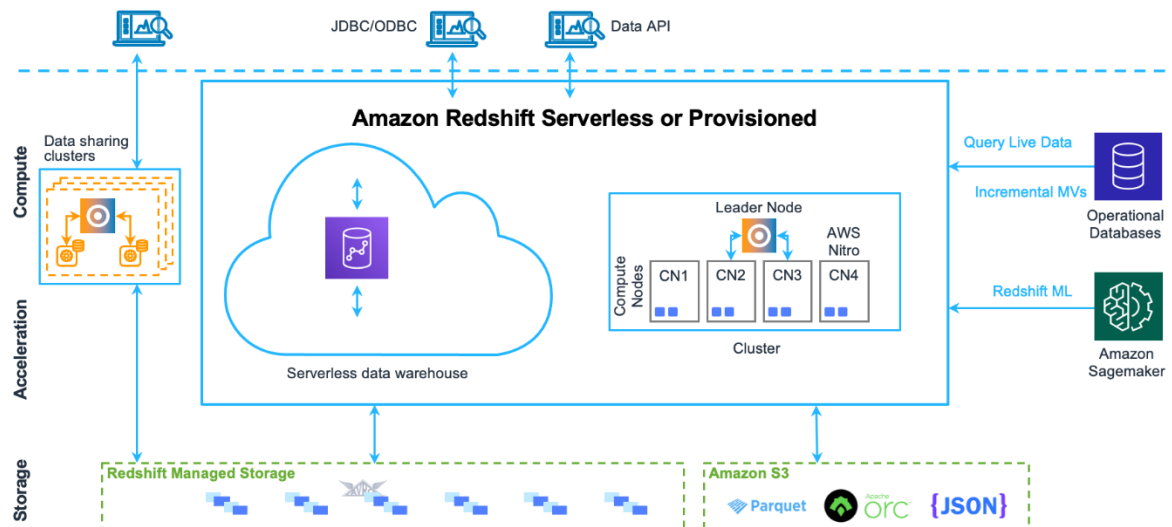
To import the Python connector, run the following command.

```
>>> import redshift_connector
```

To connect to an Amazon Redshift cluster using AWS credentials, run the following command.

```
conn = redshift_connector.connect(  
    host='examplecluster.abc123xyz789.us-west-  
1.redshift.amazonaws.com',  
    port=5439,  
    database='dev',  
    user='awsuser',  
    password='my_password'  
)
```

11 Data warehouse system architecture



◆ Client Applications

- Amazon Redshift supports integration with ETL tools, BI tools, and SQL client applications.
- It's based on **PostgreSQL**, so most SQL clients work with minimal changes.
- There are **important differences** between Redshift SQL and PostgreSQL.

◆ Clusters

- A **cluster** is the core infrastructure, made up of one or more **compute nodes**.
- If there are **2+ compute nodes**, a **leader node** is added to coordinate them.
- Client applications interact **only with the leader node**.

◆ Leader Node

- Manages communication with clients and compute nodes.
- Parses queries, creates execution plans, compiles code, and distributes tasks.
- Executes queries **only on compute nodes** if they reference tables stored there.
- Some SQL functions are **only supported on the leader node**.

◆ Compute Nodes

- Execute compiled code and return intermediate results to the leader node.
- Each has its own **CPU and memory**, based on node type.
- You can scale by adding nodes or upgrading node types.

◆ Redshift Managed Storage (RMS)

- Uses **Amazon S3** for scalable storage.
 - Separates **compute and storage**, allowing independent scaling.
 - Uses **SSD-based cache** and optimizations for performance.
-

◆ Node Slices

- Compute nodes are divided into **slices**, each handling part of the workload.
 - Leader node distributes data and tasks to slices for **parallel processing**.
 - You can define a **distribution key** to optimize data distribution and query performance.
-

◆ Internal Network

- High-speed, private network between leader and compute nodes.
 - Compute nodes are **isolated** from external access.
-

◆ Databases

- A cluster can have **multiple databases**.
 - User data is stored on compute nodes.
 - Redshift is an **RDBMS optimized for analytics**, not just OLTP.
-

12 Amazon Redshift Performance

◆ Massively Parallel Processing (MPP)

- Redshift uses **MPP** to run complex queries quickly on large datasets.
 - Each **compute node core** runs the same query segment on a portion of the data.
 - **Data distribution** across nodes enables parallel processing.
 - Choosing the right **distribution key** helps balance workload and reduce data movement.
-

◆ Parallel Data Loading

- Loading from flat files is **parallelized** across nodes and files.
- This improves speed and efficiency during data ingestion.

◆ Columnar Data Storage

- Tables are stored in **columns**, reducing disk I/O.
- Helps in **fast filtering** when columns are sorted properly.
- Sorting improves query performance by skipping irrelevant data blocks.

◆ Data Compression

- Reduces storage and disk I/O, improving query speed.
- Data is **compressed on disk**, then **uncompressed in memory** during query execution.
- Redshift uses **adaptive compression** based on column data types.
- Best practice: allow Redshift to apply **automatic compression** during data load.

◆ Result Caching

- Redshift caches query results on the **leader node** to reduce runtime.
- Cached results are used if:
 - User has access.
 - Tables/views are unchanged.
 - No volatile functions (e.g., `GETDATE`) are used.
 - No external tables (Redshift Spectrum) are referenced.
 - Query syntax matches exactly.
- **Result caching is on by default**, but can be disabled per session.
- Use `SVL_QLOG` system view to check if a query used cached results.

13 Columnar storage

◆ Traditional Row-Based Storage (OLTP Style)

Imagine a table like this:

ID	Name	Age	Country
1	Alice	30	India
2	Bob	25	USA
3	Carol	28	UK

In **row-based storage**, each row is stored together in a block. So, a block might look like:

Block 1: [1, Alice, 30, India]

Block 2: [2, Bob, 25, USA]

Block 3: [3, Carol, 28, UK]

If you query only the `Age` column, the system still reads all blocks and extracts the `Age` field, which is inefficient.

◆ Columnar Storage in Amazon Redshift

In **columnar storage**, data is stored **column-wise**. So the same table would be stored like:

Block A: [1, 2, 3] → ID column
Block B: [Alice, Bob, Carol] → Name column
Block C: [30, 25, 28] → Age column
Block D: [India, USA, UK] → Country column

Now, if you run a query like:

SQL

```
SELECT AVG(Age) FROM users;
```

Show more lines

Redshift only reads **Block C**, skipping all other blocks. This drastically reduces:

- **Disk I/O**
- **Memory usage**
- **Query time**

◆ Efficiency Gains

Let's say your table has **100 columns** and **1 billion rows**, but your query only needs **5 columns**. With columnar storage:

- Only **5% of the data** is read.
- Compression is more effective because each block contains **similar data types**.
- Redshift uses **1 MB block size**, which is larger than traditional databases (2–32 KB), reducing I/O requests even further.

11 Automatic table optimization

- A **self-tuning feature** that automatically applies **sort keys** and **distribution keys** to tables.
- Designed to **optimize performance** without manual intervention from administrators.

How It Works

- Uses **AI-based analysis** to monitor query patterns and table usage.

- If performance can be improved, Redshift **automatically alters** the table.
- Changes are typically applied **within hours** of cluster creation.

Requirements

- Tables must have **sort key or distribution style set to AUTO**.
- Admins can enable this by:
 - Creating new tables with `AUTO` settings.
 - Altering existing tables to use `AUTO`.

Benefits

- **No manual tuning** needed.
- **Faster performance** for queries.
- **Minimal impact** on ongoing queries during optimization.

Automatic Changes

- If beneficial, Redshift may:
 - Apply a **sort key**.
 - Change distribution style from `AUTO` to `KEY`.

Enabling automatic table optimization

To enable an existing table to be automatically optimized, use the `ALTER` statement options to change the table to `AUTO`. You might choose to define automation for sort keys, but not for distribution keys (and vice versa). If you run an `ALTER` statement to convert a table to be an automated table, existing sort keys and distribution styles are preserved.

```
ALTER TABLE table_name ALTER SORTKEY AUTO;  
  
ALTER TABLE table_name ALTER DISTSTYLE AUTO;
```

Initially, a table has no distribution key or sort key. The distribution style is set to either `EVEN` or `ALL` depending on table size. As the table grows in size, Amazon Redshift applies the optimal distribution keys and sort keys. Optimizations are

applied within hours after a minimum number of queries are run. When determining sort key optimizations, Amazon Redshift attempts to optimize the data blocks read from disk during a table scan. When determining distribution style optimizations, Amazon Redshift tries to optimize the number of bytes transferred between cluster nodes.

Removing automatic table optimization from a table

You can remove a table from automatic optimization. Removing a table from automation involves selecting a sort key or distribution style. To change distribution style, specify a specific distribution style.

```
ALTER TABLE table_name ALTER DISTSTYLE EVEN;  
  
ALTER TABLE table_name ALTER DISTSTYLE ALL;  
  
ALTER TABLE table_name ALTER DISTSTYLE KEY DISTKEY c1;
```

To change a sort key, you can define a sort key or choose none.

```
ALTER TABLE table_name ALTER SORTKEY(c1, c2);  
  
ALTER TABLE table_name ALTER SORTKEY NONE;
```

11.1 Data distribution for query optimization

When you load data into a table, Amazon Redshift distributes the rows of the table to each of the compute nodes according to the table's distribution style. When you run a query, the query optimizer redistributes the rows to the compute nodes as needed to perform any joins and aggregations. The goal in choosing a table distribution style is to minimize the impact of the redistribution step by locating the data where it must be before the query is run.

AUTO Distribution Style – Example

Scenario:

You create a table without specifying a distribution style:

```
CREATE TABLE sales (  
    saleid INT PRIMARY KEY,  
    productid INT,
```

```
sale_date DATE,  
amount DECIMAL(10,2)  
);
```

Since no `DISTSTYLE` is specified, Redshift uses `AUTO`.

What Happens:

- Initially, Redshift sees the table is **small**, so it applies `ALL` distribution.
- As data grows:
 - If `sale_id` is frequently used in joins, Redshift may switch to `KEY` distribution using `sale_id`.
 - If no column is suitable for distribution, it switches to `EVEN`.

This change happens **automatically** and **in the background**.

EVEN Distribution – Example

Scenario:

You create a log table that's not used in joins:

```
CREATE TABLE logs (  
    logid INT,  
    logmessage TEXT,  
    log_time TIMESTAMP  
)
```

`DISTSTYLE EVEN`;

What Happens:

- Rows are distributed **evenly** across all slices.
- Best for **non-join** tables or when no good distribution key exists.

KEY Distribution – Example

Scenario:

You have two tables that are frequently joined:

```
CREATE TABLE orders (
```

```

    orderid INT PRIMARY KEY,
    customerid INT,
    orderdate DATE
)
DISTSTYLE KEY DISTKEY(customerid);

```

```

CREATE TABLE customers (
    customerid INT PRIMARY KEY,
    name TEXT,
    email TEXT
)
DISTSTYLE KEY DISTKEY(customerid);

```

What Happens:

- Both tables are distributed using `customer_id`.
- Redshift **collocates** rows with the same `customer_id` on the same slice.
- This **speeds up joins** between `orders` and `customers`.

ALL Distribution – Example

Scenario:

You have a small, static dimension table:

```

CREATE TABLE productcategories (
    categoryid INT PRIMARY KEY,
    category_name TEXT
)
DISTSTYLE ALL;

```

What Happens:

- A **full copy** of the table is stored on **every node**.
- Ensures **fast joins** with fact tables.
- Best for **small tables** that don't change often.

You can use system views to track and manage distribution:

- **Actions taken by Redshift:** `SELECT * FROM SVLAUTOWORKER_ACTION;`
- **Recommendations for changes:** `SELECT * FROM SVVALTERTABLE_RECOMMENDATIONS;`
- **Current distribution style:** `SELECT * FROM PGCLASSINFO WHERE relname = 'yourtablename';`

We can see the diststyle in To view the distribution style of a table, query the `PG_CLASS_INFO` view or the `SVV_TABLE_INFO` view.

Star Schema Overview

In a star schema:

- **Fact table:** Contains transactional data (e.g., `sales`)
- **Dimension tables:** Contain descriptive data (e.g., `products`, `customers`, `stores`, `time`)

Best Practices with Examples

1. Specify Primary and Foreign Keys

Even though Redshift doesn't enforce them, defining keys helps the **query optimizer**.

```
CREATE TABLE customers (  
    customerid INT PRIMARY KEY,  
    name TEXT,  
    email TEXT  
);
```

```
CREATE TABLE sales (  
    saleid INT PRIMARY KEY,  
    customerid INT REFERENCES customers(customerid),  
    productid INT,  
    saledate DATE,  
    amount DECIMAL(10,2)  
);
```

Your application must ensure key integrity.

2. Distribute Fact Table and Largest Dimension Table on Common Column

Let's say `sales` is the fact table and `customers` is the largest dimension table.

```
CREATE TABLE customers (  
    customerid INT PRIMARY KEY,  
    ...  
)  
DISTSTYLE KEY DISTKEY(customerid);
```

```
CREATE TABLE sales (  
    saleid INT PRIMARY KEY,  
    customerid INT,  
    ...  
)  
DISTSTYLE KEY DISTKEY(customer_id);
```

This ensures **collocation** of rows for faster joins.

3. Choose Largest Dimension Based on Join Participation

Even if `products` is a large table, if `customers` is used more often in joins, it's considered the **largest dimension** for distribution purposes.

4. Designate Distribution Keys for Other Dimension Tables

For example, if `products` joins on `product_id`, use:

```
CREATE TABLE products (  
    productid INT PRIMARY KEY,  
    ...  
)
```


DISTSTYLE KEY DISTKEY(productid);

5. Consider ALL Distribution for Small, Static Dimension Tables

If `stores` is small and rarely updated:

```
CREATE TABLE stores (  
    storeid INT PRIMARY KEY,  
    storename TEXT  
)  
  
DISTSTYLE ALL;
```

This replicates the table on all nodes, improving join performance.

6. Use AUTO Distribution for Remaining Tables

For denormalized or rarely joined tables:

```
CREATE TABLE auditlogs (  
    logid INT,  
    logmessage TEXT,  
    logtime TIMESTAMP  
)  
  
DISTSTYLE AUTO;
```

Redshift will choose the best distribution style over time.

How to Evaluate Distribution Choices

Use the following tools:

- **EXPLAIN plan:** Analyze query performance and distribution impact.
 - **SVV_ALTER_TABLE_RECOMMENDATIONS:** See Redshift's suggestions.
 - **SVL_AUTO_WORKER_ACTION:** Track automatic changes made by Redshift.
-

11.2 Sort keys

- Sort keys define how **data is physically stored** on disk.
- When a table is created with sort keys, rows are stored in **sorted order**.
- Helps the **query planner** optimize performance by skipping unnecessary blocks.
- Redshift stores data in **1 MB blocks**.
- Each block has **min and max values** for sort key columns.
- If a query uses a **range filter**, Redshift can **skip blocks** that don't match.

 Example:

```
CREATE TABLE sales (  
    saleid INT,  
    saledate DATE,  
    amount DECIMAL(10,2)  
)  
COMPOUND SORTKEY (sale_date);
```

If you query:

```
SELECT * FROM sales WHERE sale_date BETWEEN '2025-01-01' AND '2025-01-31';
```

Redshift can **skip blocks** outside this date range, improving performance.

Types of Sort Keys

1. COMPOUND Sort Key

- Default type.
- Sorts data **based on the first column**, then second, and so on.
- Best when queries use **prefix filters** (i.e., first few columns in order).

Example:

```
CREATE TABLE orders (  
    orderid INT,  
    customerid INT,  
    orderdate DATE  
)  
COMPOUND SORTKEY (customerid, order_date);
```

Querying by `customer_id` or both `customer_id` and `order_date` is efficient.

2. INTERLEAVED Sort Key

- Gives **equal weight** to all columns in the sort key.
- Useful when queries filter on **any subset** of the sort key columns.

Example:

```
CREATE TABLE events (
```

```
    eventid INT,
```

```
    location TEXT,
```

```
    eventdate DATE
```

```
)
```

```
INTERLEAVED SORTKEY (location, event_date);
```

Efficient for queries filtering by either `location`, `event_date`, or both.

⚠ Considerations

- **COMPOUND** is better for tables with frequent **INSERT/UPDATE/DELETE**.
- **INTERLEAVED** can use up to **8 columns**, but:
 - Takes longer to **VACUUM REINDEX**.
 - May require more time for **sort and merge** operations.

🔧 Monitoring & Analysis

- Use `EXPLAIN` to see how sort keys affect query plans.
- View sort keys for a table: `SELECT * FROM SVV_TABLE_INFO WHERE table = 'yourtablename';`

12 Loading data in Amazon Redshift

If your input data is not compatible with the table columns that will receive it, the `COPY` command will fail.

Use the following guidelines to help ensure that your input data is valid:

- Your data can only contain UTF-8 characters up to four bytes long.
- Verify that CHAR and VARCHAR strings are no longer than the lengths of the corresponding columns. VARCHAR strings are measured in bytes, not characters, so, for example, a four-character string of Chinese characters that occupy four bytes each requires a VARCHAR(16) column.
- Multibyte characters can only be used with VARCHAR columns. Verify that multibyte characters are no more than four bytes long.
- Verify that data for CHAR columns only contains single-byte characters.
- Do not include any special characters or syntax to indicate the last field in a record. This field can be a delimiter.
- If your data includes null terminators, also referred to as NUL (UTF-8 0000) or binary zero (0x000), you can load these characters as NULLS into CHAR or VARCHAR columns by using the NULL AS option in the COPY command: `null as '\0'` or `null as '\000'`. If you do not use NULL AS, null terminators will cause your COPY to fail.
- If your strings contain special characters, such as delimiters and embedded newlines, use the ESCAPE option with the [COPY](#) command.
- Verify that all single and double quotation marks are appropriately matched.
- Verify that floating-point strings are in either standard floating-point format, such as 12.123, or an exponential format, such as 1.0E4.
- Verify that all timestamp and date strings follow the specifications for [DATEFORMAT and TIMEFORMAT strings](#). The default timestamp format is YYYY-MM-DD hh:mm:ss, and the default date format is YYYY-MM-DD.

12.1 Loading data from Amazon S3

Prerequisites

1. **Amazon Redshift cluster** is up and running.
2. **S3 bucket** contains the data (CSV, JSON, Parquet, etc.).
3. **IAM role** with appropriate permissions is attached to the Redshift cluster.
4. **Target table** in Redshift is already created.

Step-by-Step Process

1. Create the Table in Redshift

CREATE TABLE sales (

saleid INT,

```
productid INT,  
sale_date DATE,  
amount DECIMAL(10,2)  
);
```

2. Prepare Your Data in S3

Let's say your file is: `s3://my-bucket/sales_data.csv`

3. Attach IAM Role to Redshift

Ensure your Redshift cluster has an IAM role with access to S3. Example role ARN: `arn:aws:iam::123456789012:role/MyRedshiftRole`

4. Use the COPY Command

```
COPY sales  
FROM 's3://my-bucket/salesdata.csv'  
IAMROLE 'arn:aws:iam::123456789012:role/MyRedshiftRole'  
CSV  
IGNOREHEADER 1;
```

Explanation:

- `COPY sales`: Loads data into the `sales` table.
 - `FROM`: Specifies the S3 path.
 - `IAM_ROLE`: Grants access to S3.
 - `CSV`: Specifies file format.
 - `IGNOREHEADER 1`: Skips the header row.
-

Optional Enhancements

- **GZIP**: If your file is compressed: `COPY sales FROM 's3://my-bucket/sales_data.csv.gz' GZIP;`
- **DELIMITER**: If using a custom delimiter: `DELIMITER '|'`
- **DATEFORMAT**: For custom date formats: `DATEFORMAT 'YYYY-MM-DD'`

12.2 Loading data from an Amazon DynamoDB table

Step 1: Create the Target Table in Redshift

Before loading, you need a table in Redshift that matches the structure of your DynamoDB data.

Example:

```
CREATE TABLE users (  
    userid VARCHAR(50),  
    name VARCHAR(100),  
    email VARCHAR(100),  
    signupdate DATE  
);
```

Step 2: Catalog DynamoDB Data Using AWS Glue Crawler

- Go to the **AWS Glue Console**.
- Create a **Crawler**:
 - **Source**: Choose DynamoDB.
 - **Target**: AWS Glue Data Catalog.
- Run the crawler to create a **schema** for your DynamoDB table.

This step helps Glue understand the structure of your data.

Step 3: Create an AWS Glue Job

- In Glue Console, create a **new job**.
- Choose:
 - **Source**: DynamoDB table (from the catalog).
 - **Target**: Amazon Redshift (via JDBC connection).
- Configure:
 - **IAM Role**: Must have access to DynamoDB, Redshift, and S3.
 - **Connection**: Set up a Redshift connection with JDBC URL.

Step 4: Transform Data (Optional)

- AWS Glue auto-generates a **PySpark script**.
- You can customize it to:
 - Convert data types.

- Filter or clean data.
- Map fields to Redshift columns.

Step 5: Run the Glue Job

- Execute the job.
- Monitor progress in the Glue Console.
- Once complete, data will be loaded into Redshift.

Alternative: Export to S3 → Load with COPY

1. Export DynamoDB to S3

Use AWS CLI or SDK to export data:

```
aws dynamodb scan --table-name users --output json > users.json
```

```
aws s3 cp users.json s3://my-bucket/users.json
```

2. Load into Redshift

Use the `COPY` command:

```
COPY users
```

```
FROM 's3://my-bucket/users.json'
```

```
IAM_ROLE 'arn:aws:iam::123456789012:role/MyRedshiftRole'
```

```
FORMAT AS JSON 'auto';
```

12.3 S3 event integration to automatically copy files from Amazon S3 buckets

To set up your s3 event integration, confirm the following prerequisites are completed.

- Your Amazon S3 bucket must have a bucket policy that allows several Amazon S3 permissions. For example, the following example policy allows

permissions for the resource bucket `amzn-s3-demo-bucket` that is hosted in `us-east-1`. Both the Amazon S3 bucket and the integration are in the same AWS Region.

- JSON

```
• {
•   "Version": "2012-10-17",
•   "Statement": [
•     {
•       "Sid": "Auto-Copy-Policy-01",
•       "Effect": "Allow",
•       "Principal": {
•         "Service": "redshift.amazonaws.com"
•       },
•       "Action": [
•         "s3:GetBucketNotification",
•         "s3:PutBucketNotification",
•         "s3:GetBucketLocation"
•       ],
•       "Resource": "arn:aws:s3:::amzn-s3-demo-bucket:*",
•       "Condition": {
•         "ArnLike": {
•           "aws:SourceArn": "arn:aws:redshift:us-east-1:111122223333:integration:*"
•         },
•         "StringEquals": {
```



```

•      "aws:SourceAccount": "111122223333"
•      }
•    }
•  }
• ]
• }

```

- Your target Amazon Redshift provisioned cluster or Redshift Serverless namespace must have permission to the bucket. Confirm an IAM role that is associated with your cluster or serverless namespace has a IAM policy that allows the proper permissions. The policy must allow both `s3:GetObject` for a bucket resource such as `amzn-s3-demo-bucket` and `s3:ListBucket` for a bucket resource and its contents such as `amzn-s3-demo-bucket/*`.
 - JSON

```

• {
•   "Version": "2012-10-17",
•   "Statement": [
•     {
•       "Sid": "AutoCopyReadId",
•       "Effect": "Allow",
•       "Action": [
•         "s3:GetObject",
•         "s3:ListBucket"
•       ],
•       "Resource": [
•         "arn:aws:s3:::amzn-s3-demo-bucket",
•         "arn:aws:s3:::amzn-s3-demo-bucket/*"
•       ]
•     }
•   ]
• }

```

```

•      ]
•      }
•      ]
•  }

```

Add your policy to an IAM role that has a trust relationship for the role is as follows.

- JSON

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": [
          "redshift.amazonaws.com"
        ]
      },
      "Action": "sts:AssumeRole"
    }
  ]
}

```

If your target data warehouse is a provisioned cluster, you can associate an IAM role to your provisioned cluster using the Amazon Redshift console, **Cluster permissions** tab in your cluster details. For information

about how to associate a role to your provisioned cluster, see [Associating IAM roles with clusters](#) in the *Amazon Redshift Management Guide*.

If your target data warehouse is Redshift Serverless, you can associate an IAM role to your serverless namespace using the Redshift Serverless console, **Security and encryption** tab in your namespace details. For information about how to associate a role to your serverless namespace, see [Granting permissions to Amazon Redshift Serverless](#) in the *Amazon Redshift Management Guide*.

To create an Amazon S3 event integration on the Amazon Redshift console

1. Sign in to the AWS Management Console and open the Amazon Redshift console at <https://console.aws.amazon.com/redshiftv2/>.
2. In the left navigation pane, choose **S3 event integrations**.
3. Choose **Create Amazon S3 event integration** to open the wizard to create and S3 event integration to use with auto-copy. Your source Amazon S3 bucket and target Amazon Redshift data warehouse must be in the same AWS Region. Specify the following information when going through the steps to create an integration:
 - **Integration name** – Is a unique identifier across all integrations owned by your AWS account in the current AWS Region.
 - **Description** – Is text that describes the Amazon S3 event integration for later reference.
 - **Source S3 bucket** – Is the Amazon S3 bucket in the current AWS account and AWS Region which is the source of ingesting data into Amazon Redshift.
 - **Amazon Redshift data warehouse** – Is the target Amazon Redshift provisioned cluster or Redshift Serverless workgroup that receives the data from the integration.

If your target Amazon Redshift is in the same account, you are able to select the target. If the target is in a different account, you specify the **Amazon Redshift data warehouse ARN**. The target must have a resource policy with authorized principals and integration source. If you do not have the correct resource policies on the target and your target is in the same account, you can select the **Fix it for me** option to automatically apply the resource policies during the create integration process. If your target is in a different AWS account, you need to apply the resource policy on the Amazon Redshift warehouse manually.

4. Enter up to 50 tag **Keys** and with an optional **Value** – To provide additional metadata about the integration.
5. A review page is shown where you can choose **Create S3 event integration**.

Amazon Redshift then creates an S3 event integration with its associated source and target, status, and information about the status of an associated auto-copy job. You can view information about an S3 event integration on the Amazon Redshift console by choosing **S3 event integrations**, and choosing the integration to show its details. Integrations are separated by those created **In my account** and **From other accounts**. The **In my account** list shows integrations where the source and target are in the same account. The **From other accounts** list shows integrations where the source is owned by another account.

If you delete an S3 event integration, the corresponding COPY JOB status changes from 1 (active) to 0 (inactive/pending). However, the corresponding COPY JOB is not automatically dropped. If later you try to create a COPY JOB with the same name, there might be a conflict.

Create and monitor a COPY JOB

After the integration is created, on the **S3 event integration details** page for the integration you created, choose **Create autocopy job** to go to Amazon Redshift query editor v2 where you can create the auto-copy job for the integration. Amazon Redshift matches the bucket in the FROM clause in the COPY JOB CREATE statement to the bucket used in S3 event integration. For information about how to use Amazon Redshift query editor v2, see [Querying a database using the Amazon Redshift query editor v2](#) in the *Amazon Redshift Management Guide*. For example, run the following COPY command in query editor v2 to create an automatic COPY JOB that matches the Amazon S3 bucket `s3://amzn-s3-demo-bucket/staging-folder` to an Amazon S3 event integration.

```
COPY public.target_table
FROM 's3://amzn-s3-demo-bucket/staging-folder'
IAM_ROLE 'arn:aws:iam::123456789012:role/MyLoadRoleName'
JOB CREATE my_copy_job_name
AUTO ON;
```

You define a COPY JOB one time. The same parameters are used for future runs.

To define and manage a COPY JOB, you must have permission. For information about granting and revoking permission on a COPY JOB, see [GRANT](#) and [REVOKE](#). For more information about granting and revoking scoped permissions for a COPY JOB, see [Granting scoped permissions](#) and [Revoking scoped permissions](#).

You manage the load operations using options to CREATE, LIST, SHOW, DROP, ALTER, and RUN jobs. For more information, see [COPY JOB](#).

You can query system views to see the COPY JOB status and progress. Views are provided as follows:

- [SYS_COPY_JOB](#) – contains a row for each currently defined COPY JOB.
- [SYS_COPY_JOB_DETAIL](#) – contains details on pending, error, and ingested files for each COPY JOB.
- [SYS_COPY_JOB_INFO](#) – contains messages logged about a COPY JOB.
- [SYS_LOAD_HISTORY](#) – contains details of COPY commands.
- [SYS_LOAD_ERROR_DETAIL](#) – contains details of COPY command errors.
- [SVV_COPY_JOB_INTEGRATIONS](#) – contains details of S3 event integrations.
- [STL_LOAD_ERRORS](#) – contains errors from COPY commands.
- [STL_LOAD_COMMITS](#) – contains information used to troubleshoot a COPY command data load.

For information about troubleshooting S3 event integration errors, see [Troubleshooting S3 event integration and COPY JOB errors](#).

To get the list of files loaded by a COPY JOB, run the following SQL, but first replace `<job_id>`:

```
SELECT job_id, job_name, data_source, copy_query, filename, status,
curtime

FROM sys_copy_job copyjob

JOIN stl_load_commits loadcommit

ON copyjob.job_id = loadcommit.copy_job_id

WHERE job_id = <job_id>;
```

13 Unloading data into s3

✓ 1. Prepare Your S3 Bucket

- Make sure you have an S3 bucket ready and accessible.
- Note the bucket name and path where you want to store the data.
- Ensure the bucket policy allows Redshift to write to it.

✓ 2. Set Up IAM Role

- Create an **IAM role** with `AmazonS3FullAccess` or a policy that allows `PutObject` on your S3 bucket.
- Attach this IAM role to your Redshift cluster.
- Note the **ARN** of the IAM role (you'll need it in the UNLOAD command).

✓ 3. Connect to Redshift

- Use a SQL client like **Query Editor**, **psql**, or any BI tool connected to Redshift.

✓ 4. Use the UNLOAD Command

The basic syntax:

```
UNLOAD ('SELECT * FROM yourtable')  
TO 's3://your-bucket-name/your-folder/'  
IAMROLE 'arn:aws:iam::account-id:role/yourRedshiftRole'  
FORMAT AS PARQUET  
PARALLEL OFF;
```

Key options:

- `FORMAT AS PARQUET` → Recommended for columnar storage.
- `PARALLEL OFF` → Creates a single file; `ON` creates multiple files.
- You can also use `DELIMITER`, `ADDQUOTES`, `GZIP` for CSV format.

Example for CSV:

```
UNLOAD ('SELECT * FROM sales')  
TO 's3://mybucket/salesdata/'
```

```
IAMROLE 'arn:aws:iam::123456789012:role/MyRedshiftRole'  
DELIMITER ','  
ADDQUOTES  
ALLOWOVERWRITE  
PARALLEL OFF;
```

✓ 5. Verify in S3

- Check your S3 bucket for the unloaded files.
-

14 Scalar SQL function

A scalar SQL UDF incorporates a SQL SELECT clause that runs when the function is called and returns a single value. The [CREATE FUNCTION](#) command defines the following parameters:

- (Optional) Input arguments. Each argument must have a data type.
- One return data type.
- One SQL SELECT clause. In the SELECT clause, refer to the input arguments using \$1, \$2, and so on, according to the order of the arguments in the function definition.

The input and return data types can be any standard Amazon Redshift data type.

Don't include a FROM clause in your SELECT clause. Instead, include the FROM clause in the SQL statement that calls the SQL UDF.

The SELECT clause can't include any of the following types of clauses:

- FROM
- INTO
- WHERE
- GROUP BY
- ORDER BY
- LIMIT
- The following example creates a function that compares two numbers and returns the larger value. For more information, see [CREATE FUNCTION](#).

```
create function f_sql_greater (float, float)
    returns float
stable
as $$
    select case when $1 > $2 then $1
        else $2
    end
$$ language sql;
```

- The following query calls the new f_sql_greater function to query the SALES table and return either COMMISSION or 20 percent of PRICEPAID, whichever is greater.

```
select f_sql_greater(commission, pricepaid*0.20) from sales;
```

15 Materialized View

- **Definition**

- A materialized view stores the **precomputed results of a query** to improve performance and efficiency.

- **Purpose**

- Speeds up **complex, repeated queries** on large tables (e.g., multi-table joins, aggregations).
- Ideal for predictable queries like **dashboard population**.

- **How It Works**

- Queries on materialized views return **precomputed results** without accessing base tables.
- Amazon Redshift can **automatically rewrite queries** to use materialized views for better performance.

- **Chaining Views**

- You can create materialized views **based on other materialized views** to reuse expensive joins and optimize different aggregations.

- **Maintenance**

- Data becomes stale when base tables change.
- Refresh options:
 - **Manual refresh:** REFRESH MATERIALIZED VIEW.
 - **Automatic refresh:** Configure autorefresh or schedule jobs using Redshift Scheduler API.

- **Use Cases**

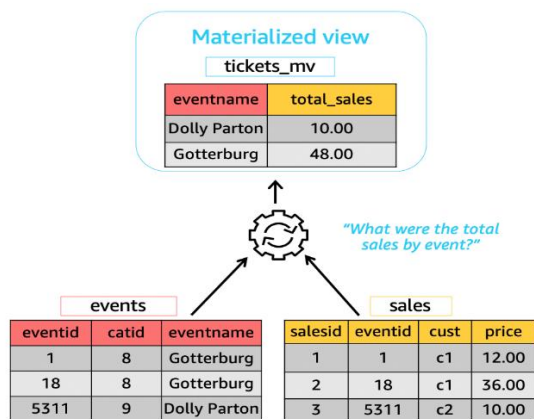
- Dashboards (e.g., Amazon QuickSight).
- Queries with **GROUP BY** or repeated joins on large datasets.

- **Dependencies**

- STV_MV_DEPS table shows dependencies between materialized views.

- **Performance Benefit**

- Saves compute time and resources by avoiding repeated expensive queries.



- ✓ **Default Behavior**

- When you update, insert, or delete data in the base table, the materialized view becomes **stale**.
- It will still return the old precomputed results until refreshed.

- ✓ **Ways to Refresh**

1. **Manual Refresh**

Use: `REFRESH MATERIALIZED VIEW yourviewname`; This updates the view with the latest data from base tables.

2. Automatic Refresh

- You can configure **AUTO REFRESH** when creating the materialized view:

```
CREATE MATERIALIZED VIEW yourviewname AUTO REFRESH YES AS
```

```
SELECT ...
```

- Or schedule refresh jobs using **Amazon Redshift Scheduler API** or **console integration**.

A materialized view can be defined on top of other materialized view(s). To refresh such a materialized view, you must explicitly use the `CASCADE` keyword at the top-most materialized view refresh. For example, assume the following nested materialized view structure:

```
CREATE TABLE t(a INT);

CREATE MATERIALIZED VIEW u AS SELECT * FROM t;

CREATE MATERIALIZED VIEW v AS SELECT * FROM u;

CREATE MATERIALIZED VIEW w AS SELECT * FROM v;

-- w -> v -> u -> t

INSERT INTO t VALUES (1);
```

To bring w fully up to date you have two choices:

- (Recommended) Refresh w using `REFRESH MATERIALIZED VIEW w CASCADE` command. This command executes refresh of all materialized views in a single transaction.
- Refresh u, v, and w as separate commands, in dependency order (first u, then v, then w).

If the `CASCADE` keyword is not explicitly used, the materialized view will be refreshed in `RESTRICT` mode, refreshing only the current materialized view. The following

examples show an informational message when you run `REFRESH MATERIALIZED VIEW` on a materialized view that depends on an out-of-date materialized view.

```
REFRESH MATERIALIZED VIEW w;
```

```
INFO: Materialized view w is already up to date. However, it
depends on another materialized view that is not up to date.
```

```
REFRESH MATERIALIZED VIEW w CASCADE;
```

```
INFO: Materialized view w was incrementally updated successfully.
```

15.1 streaming ingestion from Amazon Kinesis Data Streams

✓ How It Works

- Amazon Redshift supports **streaming ingestion** from **Amazon Kinesis Data Streams** (and Amazon MSK) directly into a **materialized view**.
- Data flows **directly from Kinesis to Redshift**—no intermediate storage like S3.
- The materialized view acts as the landing point for the stream data and can be queried for **near real-time analytics**. docs.aws.amazon.com

✓ Setup Steps

1. Create IAM Role

- Grant permissions for Redshift to read from Kinesis:
 - `kinesis:DescribeStream`, `kinesis:GetRecords`,
`kinesis:GetShardIterator`, etc. tianzhui.cloud

2. Create External Schema

```
CREATE EXTERNAL SCHEMA kinesis_schema
FROM KINESIS
IAM_ROLE 'arn:aws:iam::<account-id>:role/MyRedshiftRole';
```

3. Create Materialized View

- Define the view to parse and store streaming data:

```
CREATE MATERIALIZED VIEW stream_mv AUTO REFRESH YES AS
SELECT approximate_arrival_timestamp,
       JSON_PARSE(kinesis_data) AS data,
```

```
data.dynamodb."Keys"."Id"."S" AS prospectId,
data.dynamodb."NewImage"."Name"."S" AS prospectName
FROM kinesis_schema."my-stream"
WHERE CAN_JSON_PARSE(kinesis_data)
```

- Use `JSON_PARSE` for semi-structured data and **SUPER** type for flexibility. [\[stackoverflow.com\]](https://stackoverflow.com)
- Access directly from stream s

```
CREATE MATERIALIZED VIEW stream_mv AUTO REFRESH YES AS
SELECT approximate_arrival_timestamp,
JSON_PARSE(kinesis_data) AS data,
data.eventType AS event_type,
data.userId AS user_id,
data.timestamp AS event_time
FROM kinesis_schema."my-stream"
WHERE CAN_JSON_PARSE(kinesis_data);
```

4. Refresh Behavior

- `AUTO REFRESH YES` keeps the view updated as new data arrives.
- You can also manually refresh:

```
REFRESH MATERIALIZED VIEW stream_mv;
```

```[4](https://docs.aws.amazon.com/redshift/latest/dg/materialized-view-streaming-ingestion-getting-started.html)

#### ✓ Best Practices

- Use **SUPER type** for JSON data.
- Apply filtering and transformations in the materialized view definition.
- Ensure **network connectivity** from Redshift to Kinesis endpoints (via NAT or IGW). [\[tianzhui.cloud\]](https://tianzhui.cloud/)

---

#### ✓ Use Cases

- Real-time dashboards (e.g., clickstream, IoT telemetry).
  - Near real-time analytics without staging data in S3. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)
-

# 16 federated queries to PostgreSQL

## ✓ How It Works

1. **Create an External Schema** in Redshift that maps to your PostgreSQL database via **Amazon Aurora or RDS**.
2. Use **AWS Glue Data Catalog** or **Redshift Data API** to define metadata.
3. Run SQL queries in Redshift that join local tables with external tables.

---

## Steps to Set Up Federated Query to PostgreSQL

1. **Enable Federated Query** in Redshift:
  - Ensure your Redshift cluster is in **RA3 node type** (required for federated queries).
  - Configure **IAM role** with permissions for RDS/Aurora.
2. **Create a Database Link:**

```
CREATE EXTERNAL SCHEMA postgresschema

FROM POSTGRES

DATABASE 'yourdbname'

SCHEMA 'public'

URI 'your-rds-endpoint'

IAMROLE 'arn:aws:iam::account-id:role/RedshiftRole'
```

3. **Query External Tables:**

```
SELECT c.customername, o.orderid

FROM postgresschema.customers c

JOIN localorders o ON c.customerid = o.customerid;
```

---

## Benefits

- No ETL required.
- Real-time access to PostgreSQL data.
- Combine Redshift analytics with transactional data.

For Mysql Everything same except in from statement FROM 'Mysql'

---

## 17 Redshift Spectrum

**Amazon Redshift Spectrum** is a feature of Amazon Redshift that lets you **run SQL queries directly on data stored in Amazon S3 without loading it into Redshift tables**. It extends Redshift's capabilities beyond its local storage, enabling you to analyze large datasets in S3 using the same Redshift SQL engine.

---

### Key Features

- **Query S3 Data Directly:** No need to copy data into Redshift.
  - **Supports Common Formats:** Parquet, ORC, JSON, CSV, etc.
  - **Integrates with Glue Data Catalog:** For metadata and schema management.
  - **Scalable:** Handles petabytes of data in S3.
- **Independent Layer**
    - Redshift Spectrum runs on dedicated servers separate from your Redshift cluster.
  - **Performance Optimization**
    - Pushes compute-heavy tasks (like filtering and aggregation) to Spectrum layer.
    - Uses less cluster processing power compared to regular queries.
  - **Massive Scalability**
    - Can scale to thousands of instances for parallel processing based on query demand.
  - **External Table Definition**
    - You define Spectrum tables by registering file structures in an external data catalog (AWS Glue, Athena catalog, or Hive metastore).
    - Changes in the external catalog are immediately reflected in Redshift.
  - **Partitioning for Performance**
    - Partitioning external tables improves query performance by skipping irrelevant partitions.
  - **Materialized Views**
    - Materialized views on Spectrum tables can significantly reduce cost and improve performance.

- **Query Behavior**

- Spectrum tables can be queried and joined like normal Redshift tables.
- **No UPDATE operations** on external tables (read-only).
- Same S3 data can be queried from multiple Redshift clusters in the same region.

- **Immediate Data Availability**

- Updates to S3 files are instantly available for queries.

## 17.1 Steps for creating Redshift Spectrum from Glue data catalog

### ✓ 1. Prepare Your Environment

- Ensure you have:
    - An **Amazon Redshift cluster** with an attached **IAM role** that has:
      - `AmazonS3ReadOnlyAccess`
      - `AWSGlueConsoleFullAccess`
    - Data stored in **Amazon S3** in supported formats (Parquet, ORC, CSV, JSON).
    - AWS Glue Data Catalog set up with a database and tables describing your S3 data.
- 

### ✓ 2. Create External Schema in Redshift

Use the `CREATE EXTERNAL SCHEMA` command to link Redshift to the Glue Data Catalog:

```
CREATE EXTERNAL SCHEMA spectrumschema
FROM DATA CATALOG
DATABASE 'mygluedatabase'
IAMROLE 'arn:aws:iam::account-id:role/MyRedshiftRole'
CREATE EXTERNAL DATABASE IF NOT EXISTS;
```

- `DATABASE` → Glue database name.
  - `IAM_ROLE` → Redshift IAM role ARN.
- 

### ✓ 3. Verify External Tables

- Check tables in the external schema:

```
SELECT * FROM SVV_EXTERNAL_TABLES;
```

---

#### ✓ 4. Query S3 Data

Run queries on external tables like normal Redshift tables:

```
SELECT region, SUM(sales)
FROM spectrumschema.salesdata
GROUP BY region;
```

---

#### ✓ 5. Materialized Views (Optional)

- Create materialized views on Spectrum tables for faster queries:

```
CREATE MATERIALIZED VIEW mvsales AS
SELECT region, SUM(sales) AS totalsales
FROM spectrumschema.salesdata
GROUP BY region;
```

## 18 Cross Database Queries

**Cross-database queries in Amazon Redshift** allow you to **query data across multiple Redshift databases within the same cluster** without having to copy or move data. This feature is useful when you have separate databases for different workloads but need to combine data for analytics.

---

#### ✓ Key Points

1. **Same Cluster Only**
  - Works across databases in the same Redshift cluster (not across clusters).
2. **How It Works**
  - You reference objects using **three-part notation**:
  - `database.schema.table`
  - Example

```
SELECT a.customer_id, b.order_id
```



```
FROM sales_db.public.customers a
JOIN marketing_db.public.orders b
ON a.customer_id = b.customer_id;
```

### Permissions

- You need **USAGE** permission on the target database and schema.
- Grant access using:  
GRANT USAGE ON DATABASE marketing\_db TO user\_name;
- **Benefits**
  - No need for data duplication.
  - Simplifies analytics across multiple logical databases.
- **Limitations**
  - Only supported in **RA3 node types**.
  - Cannot use **temporary tables** across databases.
  - External schemas (Spectrum) can also be referenced if permissions allow.

## 19 Semi-structured data in Amazon Redshift

### ✓ Key Features in Redshift

1. **Supported Formats**
  - JSON (including nested JSON)
  - Avro
  - ORC
  - Parquet
2. **How Redshift Handles Semi-Structured Data**
  - Uses the **SUPER data type** to store semi-structured data.
  - Provides **PARTITION BY** and **SORT KEY** for optimization.
  - Supports **nested queries** using **Redshift's PartiQL** (SQL-compatible query language).
3. **SUPER Data Type**
  - Allows storing hierarchical data like JSON.
  - Example:

```
CREATE TABLE events (
```

```
eventid INT,

eventdata SUPER

);
```

#### 4. Querying Semi-Structured Data

- Use **dot notation** or **path expressions**:

```
SELECT eventdata.user.name AS username

FROM events

WHERE event_data.user.age > 25;
```

#### 5. Benefits

- No need to flatten JSON before loading.
- Flexible schema evolution.
- Ideal for IoT, clickstream, and log data.

### ✓ SUPER Data Type

- Stores **schemaless arrays and structures** with Redshift scalars and nested arrays.
- Can natively store **semi-structured formats** like JSON.
- Allows adding new attributes without schema changes.
- Uses **binary format** internally → faster ingestion and less storage than raw JSON.

---

### ✓ Ingesting JSON into SUPER

- Use **COPY command** or **JSON\_PARSE()** function.
- Example: COPY my\_table FROM 's3://bucket/file.json' FORMAT AS JSON 'auto'; Or:
- Supports **strict parsing** by default.
- No schema modification needed for new attributes.

---

### ✓ Querying SUPER with PartiQL

- **PartiQL** = SQL-compatible language for nested/semi-structured data.
- Supports:

- Object and array navigation
  - Unnesting arrays
- Dynamic typing during execution.
- Operates in **lax mode** → type inconsistencies return `NULL` instead of errors.
- Ideal for **ELT workflows**.

Example:

```
SELECT eventdata.user.name AS username
FROM events
WHERE event_data.user.age > 25;
```

---

#### ✓ Integration with Redshift Spectrum

- Spectrum supports **nested data with schema** (e.g., Parquet, JSON).
- SUPER is **schema-less**, unlike Spectrum's schema-based approach.
- SUPER allows flexible arrays and structures without predefined schema.

A **SUPER column** in Amazon Redshift is simply a column in a table that uses the **SUPER data type**. This data type is designed to store **semi-structured, schemaless data** such as JSON, arrays, and nested structures.

---

#### ✓ Key Characteristics of a SUPER Column

- Can hold **hierarchical data** (arrays, objects) without a predefined schema.
- Supports **dynamic attributes** (you can add new fields without altering the table schema).
- Stores data in a **binary format** for efficient storage and faster queries.
- Works with **PartiQL** for querying nested data using SQL-like syntax.

---

#### ✓ Example: Creating a Table with a SUPER Column

```
CREATE TABLE clickstreamevents (
 eventid BIGINT,
 event_data SUPER
);
```

Here, `event_data` is the SUPER column that can store JSON or other semi-structured data.

---

✓ Inserting Data into SUPER Column

```
INSERT INTO clickstreamevents VALUES
```

```
(1, JSON_PARSE({'ip':"192.168.1.1","time":"2025-11-12","customer_id":123}));
```

---

✓ Querying SUPER Column with PartiQL

```
SELECT eventdata.ip, eventdata.customerid
```

```
FROM clickstreamevents
```

```
WHERE eventdata.customerid > 100;
```

---

## 19.1 Using JSON\_PARSE to insert data into SUPER columns

You can insert or update JSON data into a SUPER column using the [JSON\\_PARSE function](#). The function parses data in JSON format and converts it into the SUPER data type, which you can use in INSERT or UPDATE statements.

The following example inserts JSON data into a SUPER column. If the JSON\_PARSE function is missing in the query, Amazon Redshift treats the value as a single string instead of a JSON-formatted string that must be parsed.

```
--Drop the table if it exists.

DROP TABLE IF EXISTS test_json;

--Create the table.

CREATE TABLE test_json (all_data SUPER);

--Populate the table.

INSERT INTO test_json VALUES (JSON_PARSE('
{
 "name": {
 "first_name": "Jake",
```

```

 "last_name": "Smith"

 },

 "age": 30,

 "hobby": "Biking"
}'

));

SELECT * FROM test_json;

all_data

{"name":{"first_name":"Jake","last_name":"Smith"},"age":30,"hobby":"
Biking"}

```

## 19.2 Using COPY to load data into SUPER columns

### Copying data from JSON and Avro

Amazon Redshift provides the following methods to ingest a JSON document using COPY, even with a JSON structure that is fully or partially unknown:

- Store the data deriving from a JSON document into a single SUPER data column using the `noshred` option. This method is useful when the schema isn't known or is expected to change. Thus, this method makes it easier to store the entire tuple in a single SUPER column.
- Shred the JSON document into multiple Amazon Redshift columns using the `auto` or `jsonpaths` option. Attributes can be Amazon Redshift scalars or SUPER values.

You can use these options with the JSON or Avro formats. For more information on JSON options such as `noshred`, `auto`, and `jsonpaths`, see [JSON format for COPY](#).

The maximum size for a JSON object in Amazon Redshift is 4 MB, which applies before any shredding or parsing.

## Method 1: Copying a JSON document into a single SUPER data column using noshred

You can copy entire JSON documents into single SUPER data columns using the `noshred` option in the COPY command. Consider the following example:

1. Create a table with a single SUPER data column.

```
CREATE TABLE region_nations_noshred (rdata SUPER);
```

2. Copy the data from Amazon S3 into the single SUPER data column. To ingest the JSON source data into a single SUPER data column, specify the `noshred` option in the FORMAT JSON clause.

```
3. COPY region_nations_noshred FROM 's3://redshift-downloads/semistructured/tpch-nested/data/json/region_nation'
4. REGION 'us-east-1' IAM_ROLE
'arn:aws:iam::xxxxxxxxxxxx:role/Redshift-S3'
```

```
FORMAT JSON 'noshred';
```

After COPY has successfully ingested the JSON, your table has a `rdata` SUPER data column that contains the data of the entire JSON object. The ingested data maintains all the properties of the JSON hierarchy. However, the leaves are converted to Amazon Redshift scalar types for efficient query processing.

5. Use the following query to retrieve the original JSON string.

```
SELECT rdata FROM region_nations_noshred;
```

## Method 2: Copying a JSON document into multiple SUPER data columns

You can shred a JSON document into multiple columns that can be either SUPER data columns or Amazon Redshift scalar types. Amazon Redshift spreads different portions of the JSON object to different columns. Consider the following example:

1. Create a table with multiple columns to hold the shredded JSON.

```
2. CREATE TABLE region_nations
```

```
3. (
```

```
4. r_regionkey smallint
```

```
5. ,r_name varchar
```

```
6. ,r_comment varchar
```

```
7. ,r_nations super
```

```
);
```

8. To copy JSON into the `region_nations` table, specify the `AUTO` option in the `FORMAT JSON` clause to split the JSON value across multiple columns. `COPY` matches the top-level JSON attributes with column names and allows nested values to be ingested as `SUPER` values, such as JSON arrays and objects.

```
9. COPY region_nations FROM 's3://redshift-
downloads/semistructured/tpch-nested/data/json/region_nation'
```

```
10. REGION 'us-east-1' IAM_ROLE
'arn:aws:iam::xxxxxxxxxxxx:role/Redshift-S3'
```

```
FORMAT JSON 'auto';
```

When the JSON attribute names are in mixed upper and lower cases, specify the `auto ignorecase` option in the `FORMAT JSON` clause. For more information about the `COPY` command, see [Load from JSON data using the 'auto ignorecase' option](#).

In some cases, there is a mismatch between column names and JSON attributes or the attribute to load is nested more than a level deep. If so, use a `jsonpaths` file to manually map JSON attributes to Amazon Redshift columns. Consider the following example:

1. Create a table with multiple columns to hold the shredded JSON.

```
2. CREATE TABLE region_nations
```

```
3. (
```

```
4. r_regionkey smallint
5. ,r_name varchar
6. ,r_comment varchar
7. ,r_nations super
```

```
);
```

8. In this case, the column names don't match the JSON attributes. You can create a `jsonpaths` file that maps the paths of attributes to the table columns by their position in the `jsonpaths` array, like the following:

```
9. {"jsonpaths": [
10. "$.r_regionkey",
11. "$.r_name",
12. "$.r_comment",
13. "$.r_nations
14.]
```

```
}
```

15. Use the location of the `jsonpaths` file as the argument to the `FORMAT JSON` option in `COPY`.

```
16. COPY nations FROM 's3://redshift-
 downloads/semistructured/tpch-nested/data/json/region_nation'
17. REGION 'us-east-1' IAM_ROLE
 'arn:aws:iam::xxxxxxxxxxxx:role/Redshift-S3'
```

```
FORMAT JSON 's3://redshift-downloads/semistructured/tpch-
 nested/data/jsonpaths/nations_jsonpaths.json';
```

18. Use the following query to access the table that shows data spread to multiple columns. The `SUPER` data columns are printed using the `JSON` format.



```
SELECT r_regionkey,r_name,r_comment,r_nations[0].n_nationkey
FROM region_nations ORDER BY 1,2,3 LIMIT 1;
```

`jsonpaths` files map fields in the JSON document to table columns. You can extract additional columns, such as distribution and sort keys, while still loading the complete document as a SUPER column. The following query loads the complete document to the `nations` column. The `name` column is the sort key and the `regionkey` column is the distribution key. Consider the following example:

## Copying data from text and CSV

Amazon Redshift represents SUPER columns in text and CSV formats as serialized JSON. Valid JSON formatting is required for SUPER columns to load with the correct type information. Unquote objects, arrays, numbers, booleans, and null values. Wrap string values in double quotes. SUPER columns use standard escaping rules for text and CSV formats.

When copying from CSV, delimiters are escaped according to the CSV standard. Consider the following example:

```
CREATE TABLE region_nations
(
 r_regionkey smallint
 ,r_name varchar
 ,r_comment varchar
 ,r_nations super
);

COPY region_nations FROM 's3://redshift-
downloads/semistructured/tpch-nested/data/csv/region_nation'
REGION 'us-east-1' IAM_ROLE
'arn:aws:iam::xxxxxxxxxxx:role/Redshift-S3'
FORMAT CSV;
```

When copying from text, if the chosen delimiter might also appear in a SUPER field, use the ESCAPE option during COPY and UNLOAD. Consider the following example:

```
CREATE TABLE region_nations
(
 r_regionkey smallint
 ,r_name varchar
 ,r_comment varchar
 ,r_nations super
);

COPY region_nations FROM 's3://redshift-
downloads/semistructured/tpch-nested/data/text/region_nation'
REGION 'us-east-1' IAM_ROLE
'arn:aws:iam::xxxxxxxxxxx:role/Redshift-S3'
DELIMITER ','
ESCAPE;
```

## Copying data from columnar-format Parquet and ORC

If your semi-structured or nested data is already available in either Apache Parquet or Apache ORC format, you can use the COPY command to ingest data into Amazon Redshift.

The Amazon Redshift table structure should match the number of columns and the column data types of the Parquet or ORC files. By specifying SERIALIZE TO JSON in the COPY command, you can load any column type in the file that aligns with a SUPER column in the table as SUPER. This includes structure and array types.

The following example uses a Parquet format:

```
COPY region_nations FROM 's3://redshift-
downloads/semistructured/tpch-nested/data/parquet/region_nation'

REGION 'us-east-1' IAM_ROLE
'arn:aws:iam::xxxxxxxxxxxx:role/Redshift-S3'

FORMAT PARQUET SERIALIZETOJSON;
```

The following example uses an ORC format.

```
COPY region_nations FROM 's3://redshift-
downloads/semistructured/tpch-nested/data/orc/region_nation'

IAM_ROLE 'arn:aws:iam::xxxxxxxxxxxx:role/Redshift-S3'

FORMAT ORC SERIALIZETOJSON;
```

When the attributes of the date or time data types are in ORC, Amazon Redshift converts them to varchar upon encoding them in SUPER.

For Unload is same as structured.

## 19.3 Example

### Loading semi-structured data

The following statements create a sample table and load a sample JSON object into the `all_data` SUPER column.

```
DROP TABLE IF EXISTS test_json;

SET enable_case_sensitive_super_attribute TO true;

SET enable_case_sensitive_identifier TO true;
```

```
CREATE TABLE test_json (all_data SUPER);

INSERT INTO test_json VALUES (JSON_PARSE('
{
 "data":{
 "pnr":{
 "type":"pnr",
 "pnrid":"123PQRS-2024-09-20",
 "bookingIdentifier":"123PQRS",
 "version":"5",
 "triggerType":"",
 "events":[
 {
 "eventType":"UPDATED",
 "type":"PART",
 "id":"123PQRS-2024-09-20-HO-1"
 },
 {
 "eventType":"CREATED",
 "type":"ABC",
 "id":"123PQRS-2024-09-20-OT-38"
 }
]
 },
],
```

```
"create":{
 "pnrCreateDate":"2024-09-20T16:56:00Z",
 "officeID":"OFFCID1234",
 "officeIDCategory":"Email"
},
"lastModification":{
 "dateTime":"2024-09-20T17:09:00Z"
},
"PARTDetails":[
 {
 "path":"1",
 "TrainPARTs":[
 {
 "PARTID":"123PQRS-2024-09-20-HO-1",
 "departure":{
 "departureStation":"XYZ",
 "departureTimeLocal":"2024-10-03T06:30:00",
 "departureTimeGMT":"2024-10-03T10:30:00Z"
 },
 "arrival":{
 "arrivalStation":"ABC",
 "arrivalTimeLocal":"2024-10-03T08:20:00",
 "arrivalTimeGMT":"2024-10-03T15:20:00Z"
 }
 }
]
 }
]
```

```
 },
 "marketing":{
 "carrierCode":"XX",
 "TrainNumber":"100"
 },
 "operating":{
 "carrierCode":"YY",
 "TrainNumber":"100-A"
 },
 "status":"ON",
 "aircraft":{
 "code":"222"
 },
 "class":"WC",
 "first":"Y",
 "seating":[

]
}
]
}
],
"commuterInformation":[
```

```

 {
 "commuterID": "2",
 "commuterPNR": "123PQRS-2024-09-20-RO-2",
 "commuterTypeCode": "DOM",
 "firstName": "JOHN",
 "lastName": "MILLER"
 }
],
 etc

```

## Querying nested semi-structured data

The following statement uses PartiQL's dot notation to extract the `pnrid` field, which is nested three levels deep inside the top-level `all_data` object.

```

select all_data.data.pnr.pnrid::varchar from test_json;

```

| pnrid              |
|--------------------|
| 123PQRS-2024-09-20 |

The following statement uses PartiQL's bracket notation to specify and extract only the first element from the `events` array nested inside the top-level object.

```

SELECT
 all_data.data.pnr.events[0]
FROM test_json;

```

| events                                                 |
|--------------------------------------------------------|
| [{"event": "boarding", "time": "2024-09-20T10:00:00"}] |

```

{
 "eventType": "UPDATED",
 "type": "PART",
 "id": "123PQRS-2024-09-20-H0-1"
}

```

The following statement extracts the `eventType` property of only the specified element from the `events` array.

```

SELECT

 all_data.data.pnr.events[0].eventType
FROM test_json;

eventtype

"UPDATED"

```

The following statements

## Using `enable_case_sensitive_identifier` and `enable_case_sensitive_super_attribute` with semi-structured data

The following examples show how the configuration options [enable\\_case\\_sensitive\\_identifier](#) and [enable\\_case\\_sensitive\\_super\\_attribute](#) differ when used for querying semi-structured data. For more information on these configuration options, see [Accessing JSON fields with uppercase and mixed-case field names or attributes](#).

In the following statement, resetting both configuration options to their default of false makes the query return NULL.



```

RESET enable_case_sensitive_identifier;

RESET enable_case_sensitive_super_attribute;

SELECT

 all_data.data.pnr.events[0].eventType
FROM test_json;

eventtype

NULL

```

In following example, the sample query returns the desired result after you wrap the case sensitive attributes in double quotation marks and `set enable_case_sensitive_identifier` to true.

```

RESET enable_case_sensitive_identifier;

RESET enable_case_sensitive_super_attribute;

SELECT

 all_data.data.pnr.events[0]."eventType"
FROM test_json;

eventtype

NULL

```

```
SET enable_case_sensitive_identifier TO true;
```

```
SELECT
```

```
 all_data.data.pnr.events[0].eventType
```

```
FROM test_json;
```

```
eventtype
```

```

```

```
"UPDATED"
```

In the following example, the sample query returns the desired result after you set `enable_case_sensitive_super_attribute` to true without wrapping the case sensitive attributes in double quotation marks.

```
RESET enable_case_sensitive_identifier;
```

```
RESET enable_case_sensitive_super_attribute;
```

```
SELECT
```

```
 all_data.data.pnr.events[0].eventType
```

```
FROM test_json;
```

```
eventtype
```

```

```

```
NULL
```

```
SET enable_case_sensitive_super_attribute TO true;
```

```

SELECT
 all_data.data.pnr.events[0].eventType
FROM test_json;

eventtype

"UPDATED"

```

## Unnesting semi-structured data

The following statement uses PartiQL joins to unnest the `events` array. Note that this join works even when the number of indexes for the array aren't static.

For examples of unnesting semi-structured data using `UNNEST` in the `FROM` clause, see [UNNEST examples](#).

```

SELECT
a.all_data.data.pnr.type::varchar type_info,
a.all_data.data.pnr.pnrId::varchar pnr_id ,
a.all_data.data.pnr.bookingIdentifier::varchar booking_id,
a.all_data.data.pnr.version::varchar version_info,
b.eventType::varchar event_type,
b.id::varchar event_id
FROM test_json a,
 a.all_data.data.pnr.events b;

```

| type_info               | pnr_id                   | booking_id | version_info |
|-------------------------|--------------------------|------------|--------------|
| event_type              | event_id                 |            |              |
| -----+-----+-----+----- |                          |            |              |
| -----+-----             |                          |            |              |
| pnr                     | 123PQRS-2024-09-20       | 123PQRS    | 5            |
| UPDATED                 | 123PQRS-2024-09-20-HO-1  |            |              |
| pnr                     | 123PQRS-2024-09-20       | 123PQRS    | 5            |
| CREATED                 | 123PQRS-2024-09-20-OT-38 |            |              |

## Unnesting nested arrays

The following statement uses PartiQL joins to unnest an array that's nested inside another array.

For examples of unnesting semi-structured data using UNNEST in the FROM clause, see [UNNEST examples](#).

```
SELECT
a.all_data.data.pnr.type::varchar type_info,
a.all_data.data.pnr.pnrid::varchar pnr_id ,
a.all_data.data.pnr.bookingIdentifier::varchar booking_id,
a.all_data.data.pnr.version::varchar version_info,
d.id::varchar email_record_id,
d.contact::varchar email_contact,
e::varchar email_purpose,
f::varchar email_commuter
FROM test_json a,
 a.all_data.data.pnr.contactDetail c,
 c."emailContacts" d,
```

```
d.purpose e,
d.commuter f;
```

## Using semi-structured data in materialized views

The following statement uses the statement from the previous example to create a materialized view. The materialized view automatically refresh the number of pending services when the base table gets new data.

```
CREATE MATERIALIZED VIEW mv_total_pending_service

AUTO REFRESH YES

AS

SELECT

a.all_data.data.pnr.type::varchar type_info,
a.all_data.data.pnr.pnr_id::varchar pnr_id ,
a.all_data.data.pnr.bookingIdentifier::varchar booking_id,
a.all_data.data.pnr.version::varchar version_info,
COUNT(*) AS total_pending_service

FROM test_json a,

 a.all_data.data.pnr.PendingService c

GROUP BY 1,2,3,4;
```

## 20 SQL Commands in Redshift

### 20.1 SQL functions supported on the leader node

The following examples use the sample TICKIT database. For more information on the sample database, go to [Sample database](#).

#### CURRENT\_SCHEMA

The CURRENT\_SCHEMA function is a leader-node only function. In this example, the query does not reference a table, so it runs exclusively on the leader node.

```
select current_schema();
```

```
current_schema
```

```

```

```
public
```

In the next example, the query references a system catalog table, so it runs exclusively on the leader node.

```
select * from pg_table_def
where schemaname = current_schema() limit 1;
```

```
 schemaname | tablename | column | type | encoding | distkey |
sortkey | notnull
```

```
-----+-----+-----+-----+-----+-----+-----
```

```
-----+-----
```

```
public | category | catid | smallint | none | t |
1 | t
```

In the next example, the query references an Amazon Redshift system table that resides on the compute nodes, so it returns an error.

```
select current_schema(), userid from users;
```

INFO: Function "current\_schema()" not supported.

ERROR: Specified types or functions (one per INFO message) not supported on Amazon Redshift tables.

## SUBSTR

SUBSTR is also a leader-node only function. In the following example, the query runs exclusive on the leader node because it does not reference a table.

```
SELECT SUBSTR('amazon', 5);
```

```
+-----+
| substr |
+-----+
| on |
+-----+
```

In the following example, the query references a table that resides on the compute nodes. This results in an error.

```
SELECT SUBSTR(catdesc, 1) FROM category LIMIT 1;
```

ERROR: SUBSTR() function is not supported (Hint: use SUBSTRING instead)

To successfully run the previous query, use [SUBSTRING](#).

```
SELECT SUBSTRING(catdesc, 1) FROM category LIMIT 1;
```

```
+-----+
| substring |
+-----+
```

```
+-----+
| National Basketball Association |
+-----+
```

## FACTORIAL()

FACTORIAL() is a leader-node only function. In the following example, the query runs exclusive on the leader node because it does not reference a table.

```
SELECT FACTORIAL(5);
```

```
factorial
```

```

120
```

In the following example, the query references a table that resides on the compute nodes. This results in an error when run using query editor v2.

```
create table t(a int);
```

```
insert into t values (5);
```

```
select factorial(a) from t;
```

```
ERROR: Specified types or functions (one per INFO message) not
supported on Redshift tables.
```

```
Info: Function "factorial(bigint)" not supported.
```

These PostgreSQL features are not supported in Amazon Redshift.

- The query tool *psql* is unsupported. The [Amazon Redshift RSQL](#) client is supported.
- Table partitioning (range and list partitioning)
- Tablespaces
- Constraints



- Unique
- Foreign key
- Primary key
- Check constraints
- Exclusion constraints

Unique, primary key, and foreign key constraints are permitted, but they are informational only. They are not enforced by the system, but they are used by the query planner.

- Inheritance
- PostgreSQL system columns

Amazon Redshift SQL does not implicitly define system columns. However, the following PostgreSQL system column names cannot be used as names of user-defined columns: `oid`, `tableoid`, `xmin`, `cmin`, `xmax`, `cmax`, and `ctid`. For more information, see <https://www.postgresql.org/docs/8.0/static/ddl-system-columns.html>.

- Indexes
- NULLS clause in Window functions
- Collations

Amazon Redshift does not support locale-specific or user-defined collation sequences. See [Collation sequences](#).

- Value expressions
  - Subscripted expressions
  - Array constructors
  - Row constructors
- Triggers
- Management of External Data (SQL/MED)
- Table functions
- VALUES list used as constant tables
- Sequences
- Full text search
- The RULE and TRIGGER permissions.

Amazon Redshift grants or revokes these permissions when you run GRANT ALL or REVOKE ALL, but the presence or absence of the RULE and TRIGGER permissions doesn't affect the grantee's access permissions in any way.

These PostgreSQL data types are not supported in Amazon Redshift.

- Arrays
- BIT, BIT VARYING
- BYTEA
- Composite Types
- Enumerated Types
- Geometric Types (Amazon Redshift implementation of geometric types differs from PostgreSQL)
- HSTORE
- JSON
- Network Address Types
- Numeric Types
  - SERIAL, BIGSERIAL, SMALLSERIAL
  - MONEY
- Object Identifier Types
- Pseudo-Types
- Range Types
- Special Character Types
  - "char" – A single-byte internal type (where the data type named char is enclosed in quotation marks).
  - name – An internal type for object names.

For more information about these types, see [Special Character Types](#) in the PostgreSQL documentation.

- Text Search Types
- TXID\_SNAPSHOT
- UUID
- XML

These PostgreSQL functions are not supported in Amazon Redshift.

- Access privilege inquiry functions
- Advisory lock functions
- Aggregate functions
  - STRING\_AGG()
  - ARRAY\_AGG()
  - EVERY()
  - XML\_AGG()

- CORR()
  - COVAR\_POP()
  - COVAR\_SAMP()
  - REGR\_AVGX(), REGR\_AVGY()
  - REGR\_COUNT()
  - REGR\_INTERCEPT()
  - REGR\_R2()
  - REGR\_SLOPE()
  - REGR\_SXX(), REGR\_SXY(), REGR\_SYY()
- Array functions and operators
- Backup control functions
- Comment information functions
- Database object location functions
- Database object size functions
- Date/Time functions and operators
  - CLOCK\_TIMESTAMP()
  - JUSTIFY\_DAYS(), JUSTIFY\_HOURS(), JUSTIFY\_INTERVAL()
  - PG\_SLEEP()
  - TRANSACTION\_TIMESTAMP()
- ENUM support functions
- Geometric functions and operators
- Generic file access functions
- IS DISTINCT FROM
- Network address functions and operators
- Mathematical functions
  - DIV()
  - SETSEED()
  - WIDTH\_BUCKET()
- Set returning functions
  - GENERATE\_SERIES()
  - GENERATE\_SUBSCRIPTS()
- Range functions and operators
- Recovery control functions
- Recovery information functions
- ROLLBACK TO SAVEPOINT function
- Schema visibility inquiry functions
- Server signaling functions
- Snapshot synchronization functions

- Sequence manipulation functions
- String functions
  - BIT\_LENGTH()
  - OVERLAY()
  - CONVERT(), CONVERT\_FROM(), CONVERT\_TO()
  - ENCODE()
  - FORMAT()
  - QUOTE\_NULLABLE()
  - REGEXP\_MATCHES()
  - REGEXP\_SPLIT\_TO\_ARRAY()
  - REGEXP\_SPLIT\_TO\_TABLE()
- System catalog information functions
- System information functions
  - CURRENT\_CATALOG CURRENT\_QUERY()
  - INET\_CLIENT\_ADDR()
  - INET\_CLIENT\_PORT()
  - INET\_SERVER\_ADDR() INET\_SERVER\_PORT()
  - PG\_CONF\_LOAD\_TIME()
  - PG\_IS\_OTHER\_TEMP\_SCHEMA()
  - PG\_LISTENING\_CHANNELS()
  - PG\_MY\_TEMP\_SCHEMA()
  - PG\_POSTMASTER\_START\_TIME()
  - PG\_TRIGGER\_DEPTH()
  - SHOW VERSION()
- Text search functions and operators
- Transaction IDs and snapshots functions
- Trigger functions
- XML functions

## 20.2 Names and Identifiers

An **identifier** is the name you give to database objects like:

- Tables
- Columns
- Schemas
- Views
- Functions

Examples:

```
CREATE TABLE employees (
 empid INT,
 empname VARCHAR(50)
);
```

Here, `employees`, `emp_id`, and `emp_name` are identifiers.

---

### ✓ Rules for Identifiers

1. **Length:**
  - Maximum **127 bytes** (usually 127 characters for ASCII).
2. **Allowed Characters:**
  - Letters (a–z, A–Z)
  - Digits (0–9)
  - Underscore (`_`)
  - Cannot start with a digit.
3. **Case Sensitivity:**
  - By default, **unquoted identifiers are folded to lowercase**.
  - Example:

```
CREATE TABLE MyTable (ID INT);
```

Internally becomes: `mytable`, `id`

- If you use **double quotes**, case is preserved: `CREATE TABLE "MyTable" ("ID" INT);`  
Now you must always refer to it as `"MyTable"` and `"ID"`.
4. **Reserved Words:**
    - If you use a keyword (like `SELECT`, `DATE`), you **must quote it**:

```
CREATE TABLE "select" (id INT);
```

## 20.3 Data Types

The following table lists the data types that you can use in Amazon Redshift tables.

| Data type        | Aliases                           | Description                                                |
|------------------|-----------------------------------|------------------------------------------------------------|
| SMALLINT         | INT2                              | Signed two-byte integer                                    |
| INTEGER          | INT, INT4                         | Signed four-byte integer                                   |
| BIGINT           | INT8                              | Signed eight-byte integer                                  |
| DECIMAL          | NUMERIC                           | Exact numeric of selectable precision                      |
| REAL             | FLOAT4                            | Single precision floating-point number                     |
| DOUBLE PRECISION | FLOAT8, FLOAT                     | Double precision floating-point number                     |
| CHAR             | CHARACTER, NCHAR, BPCHAR          | Fixed-length character string                              |
| VARCHAR          | CHARACTER VARYING, NVARCHAR, TEXT | Variable-length character string with a user-defined limit |
| DATE             |                                   | Calendar date (year, month, day)                           |
| TIME             | TIME WITHOUT TIME ZONE            | Time of day                                                |
| TIMETZ           | TIME WITH TIME ZONE               | Time of day with time zone                                 |
| TIMESTAMP        | TIMESTAMP WITHOUT TIME ZONE       | Date and time (without time zone)                          |
| TIMESTAMPTZ      | TIMESTAMP WITH TIME ZONE          | Date and time (with time zone)                             |

| Data type              | Aliases                   | Description                                                                                                                  |
|------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------|
| INTERVAL YEAR TO MONTH |                           | Time duration in year to month order                                                                                         |
| INTERVAL DAY TO SECOND |                           | Time duration in day to second order                                                                                         |
| BOOLEAN                | BOOL                      | Logical Boolean (true/false)                                                                                                 |
| HLLSKETCH              |                           | Type used with HyperLogLog sketches.                                                                                         |
| SUPER                  |                           | A superset data type that encompasses all scalar types of Amazon Redshift including complex types such as ARRAY and STRUCTS. |
| VARBYTE                | VARBINARY, BINARY VARYING | Variable-length binary value                                                                                                 |
| GEOMETRY               |                           | Spatial data                                                                                                                 |
| GEOGRAPHY              |                           | Spatial data                                                                                                                 |

---

## 20.4 SQL Commands

### 1. ABORT

- ABORT Command Purpose**
  - Stops the currently running transaction and discards all updates made by that transaction.
  - Has **no effect** on transactions that are already completed.
- Equivalent to ROLLBACK**
  - ABORT performs the same function as the ROLLBACK command.
- Syntax** ABORT [ WORK | TRANSACTION ]
  - WORK and TRANSACTION are optional and synonyms.
- Use Case**
  - Used when you want to undo all changes made during a transaction and return to the state before the transaction began.

✓ Example

**Step 1: Create a table**

```
CREATE TABLE movie_gross (
 name VARCHAR(30),
 gross BIGINT
);
```

**Step 2: Start a transaction and insert data**

```
BEGIN;
```

```
INSERT INTO moviegross VALUES ('Raiders of the Lost Ark', 23400000);
```

```
INSERT INTO moviegross VALUES ('Star Wars', 10000000);
```

**Step 3: Check inserted data**

```
SELECT * FROM movie_gross;
```

**Output:**

| name                    | gross    |
|-------------------------|----------|
| Raiders of the Lost Ark | 23400000 |
| Star Wars               | 10000000 |

(2 rows)

**Step 4: Abort the transaction**

```
ABORT;
```

**Step 5: Verify rollback**

```
SELECT * FROM movie_gross;
```

**Output:**

| name | gross |
|------|-------|
|------|-------|

(0 rows)



## ALTER DATABASE

ALTER DATABASE *databasename*

```
{
 RENAME TO newname
| OWNER TO newowner
| [CONNECTION LIMIT { limit | UNLIMITED }]
 [COLLATE { CASESENSITIVE | CS | CASEINSENSITIVE | CI }]
 [ISOLATION LEVEL { SNAPSHOT | SERIALIZABLE }]
| INTEGRATION {
 REFRESH { { ALL | INERROR } TABLES [IN SCHEMA schema [, ...]] | TABLE schema.table [, ...] }
 | SET
 [QUERYALLSTATES [=] { TRUE | FALSE }]
 [ACCEPTINVCHARS [=] { TRUE | FALSE }]
 [REFRESHINTERVAL <interval>]
 [TRUNCATECOLUMNS [=] { TRUE | FALSE }]
 [HISTORY_MODE [=] { TRUE | FALSE } [FOR { ALL TABLES [IN SCHEMA ...] | TABLE
schema.table }]]
}
```

- **RENAME TO:** Change database name; superuser or owner with CREATEDB needed; can't rename current or system DBs. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/redshift/latest/dg/r_rename_database.html)
- **OWNER TO:** Transfer ownership; superuser only. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/redshift/latest/dg/r_alter_database_owner.html)
- **CONNECTION LIMIT:** Max concurrent connections (superusers bypass). [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/redshift/latest/dg/r_alter_database_connection_limit.html)
- **COLLATE:** Set case sensitivity of string comparisons. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/redshift/latest/dg/r_alter_database_collate.html)
- **ISOLATION LEVEL:** Define transaction level: SNAPSHOT or SERIALIZABLE; requires superuser/CREATE DATABASE; cannot run inside transactions or on dev DB. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/redshift/latest/dg/r_alter_database_isolation_level.html)
- **INTEGRATION:** For zero-ETL databases; can REFRESH, configure QUERY\_ALL\_STATES, ACCEPTINVCHARS, REFRESH\_INTERVAL, TRUNCATECOLUMNS, and HISTORY\_MODE for syncing from source. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/redshift/latest/dg/r_alter_database_integration.html)

---

## ALTER DATASHARE

ALTER DATASHARE *datasharename*

```
{ ADD | REMOVE } { TABLE schema.table [, ...]
 | SCHEMA schema [, ...]
 | FUNCTION schema.name(argtypes) [, ...]
 | ALL TABLES IN SCHEMA schema [, ...]
 | ALL FUNCTIONS IN SCHEMA schema [, ...] }
```

```
| ALTER DATASHARE datasharename
```

```
[SET PUBLICACCESSIBLE [=] { TRUE | FALSE }]
```

```
[SET INCLUDENEW [=] { TRUE | FALSE } FOR SCHEMA schema]
```

- **ADD / REMOVE:** Include or exclude tables, schemas, or UDFs. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/lambda/latest/dg/lambda-iam-roles.html)
- **SET PUBLICACCESSIBLE:** Allow share to publicly accessible clusters. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/lambda/latest/dg/lambda-iam-roles.html)
- **SET INCLUDENEW:** Automatically include future objects in specified schema. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/lambda/latest/dg/lambda-iam-roles.html)

## ALTER DEFAULT PRIVILEGES

```
ALTER DEFAULT PRIVILEGES
```

```
[FOR USER targetuser [, ...]]
```

```
[IN SCHEMA schemaname [, ...]]
```

```
GRANT|REVOKE { privileges } ON TABLES|FUNCTIONS|PROCEDURES TO|FROM grantees
```

- Sets default privileges for **new** objects created by specified users/users in schemas; doesn't affect existing objects. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/lambda/latest/dg/lambda-iam-roles.html)
- Can grant or revoke `SELECT`, `INSERT`, `EXECUTE`, etc. for tables/functions/procedures. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/lambda/latest/dg/lambda-iam-roles.html)

## ALTER EXTERNAL SCHEMA

```
ALTER EXTERNAL SCHEMA schemaname
```

```
[IAMROLE { default | 'SESSION' | 'arn:...role/...' }]
```

```
[AUTHENTICATION { none | iam | mtls }]
```

```
[AUTHENTICATIONARN 'acm-certificate-arn' | SECRETARN 'secret-arn']
```

```
[URI 'Kafka bootstrap URL']
```

- Modify IAM role, auth method (none/iam/mTLS), certificate secrets, or Kafka URI for streaming schemas. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/lambda/latest/dg/lambda-iam-roles.html)

- Only owner, superuser, or ALTER privileged users can run this. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

---

## ALTER EXTERNAL VIEW

ALTER EXTERNAL VIEW *schemaname.viewname*

[FORCE] { AS (query\_definition) | REMOVE DEFINITION }

- AS: Replace view SQL definition; REMOVE DEFINITION: drop it to recreate later.
- FORCE: Update even if underlying objects differ across engines, but may lead to staleness. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

---

## ALTER FUNCTION

ALTER FUNCTION *functionname(argtypes)*

RENAME TO *newname*;

ALTER FUNCTION *functionname(argtypes)*

OWNER TO { *newowner* | CURRENTUSER | SESSIONUSER };

- Rename or change owner of SQL/Python UDF; owner or superuser required for rename, superuser only for owner change. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

---

## ALTER GROUP

ALTER GROUP *groupname*

ADD USER *username* [, ...]

| ALTER GROUP *groupname*

DROP USER *username* [, ...]

| ALTER GROUP *groupname*

RENAME TO *newname*;

- Add, remove users or rename user group; changes apply cluster-wide. Superuser required. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

---

## ALTER IDENTITY PROVIDER

ALTER IDENTITY PROVIDER

*identityprovidername*

```

[PARAMETERS parameterstring]
[NAMESPACE namespace]
[IAMROLE iamrole]
[AUTOCREATEROLES [TRUE [INCLUDE|EXCLUDE GROUPS LIKE filterpattern] | FALSE]]
[DISABLE | ENABLE];

```

- Replaces all parameters for a federation provider; can assign IAM role, auto-create roles from groups (with filters), and enable/disable it. [docs.aws.amazon.com](https://docs.aws.amazon.com)

---

## ALTER MASKING POLICY

ALTER MASKING POLICY *policyname*

USING (*maskingexpression*);

- Modify dynamic data-masking expression; expression must align with original input types; sys:secadmin or superuser permission is required. [docs.aws.amazon.com](https://docs.aws.amazon.com)

```

ALTER TABLE table_name
{
ADD table_constraint
| DROP CONSTRAINT constraint_name [RESTRICT | CASCADE]
| OWNER TO new_owner
| RENAME TO new_name
| RENAME COLUMN column_name TO new_name
| ALTER COLUMN column_name TYPE updated_varchar_data_type_size
| ALTER COLUMN column_name ENCODE new_encode_type
| ALTER COLUMN column_name ENCODE encode_type,
| ALTER COLUMN column_name ENCODE encode_type,;
| ALTER DISTKEY column_name
| ALTER DISTSTYLE ALL
| ALTER DISTSTYLE EVEN
| ALTER DISTSTYLE KEY DISTKEY column_name
| ALTER DISTSTYLE AUTO
| ALTER [COMPOUND] SORTKEY (column_name [,...])
| ALTER SORTKEY AUTO
| ALTER SORTKEY NONE
| ALTER ENCODE AUTO
| ADD [COLUMN] column_name column_type
 [DEFAULT default_expr]
 [ENCODE encoding]
 [NOT NULL | NULL]
 [COLLATE { CASE_SENSITIVE | CS | CASE_INSENSITIVE | CI }] |
| DROP [COLUMN] column_name [RESTRICT | CASCADE]

```

```
| ROW LEVEL SECURITY { ON | OFF } [CONJUNCTION TYPE { AND | OR }]
[FOR DATASHARES]
| MASKING { ON | OFF } FOR DATASHARES }
```

where *table\_constraint* is:

```
[CONSTRAINT constraint_name]
{ UNIQUE (column_name [, ...])
| PRIMARY KEY (column_name [, ...])
| FOREIGN KEY (column_name [, ...])
 REFERENCES reftable [(refcolumn)] }
```

The following options apply **only to external tables**:

```
SET LOCATION { 's3://bucket/folder/' | 's3://bucket/manifest_file' }
| SET FILE FORMAT format |
| SET TABLE PROPERTIES ('property_name'='property_value')
| PARTITION (partition_column=partition_value [, ...])
 SET LOCATION { 's3://bucket/folder' | 's3://bucket/manifest_file' }
| ADD [IF NOT EXISTS]
 PARTITION (partition_column=partition_value [, ...]) LOCATION
{ 's3://bucket/folder' | 's3://bucket/manifest_file' }
[, ...]

| DROP PARTITION (partition_column=partition_value [, ...])
```

Here's the **full quick summary table** for all the commands you listed, grouped for clarity:

| Command                         | What It Does                              | Basic Syntax                                      |                 |
|---------------------------------|-------------------------------------------|---------------------------------------------------|-----------------|
| <b>ABORT</b>                    | Cancels current transaction               | ABORT;                                            |                 |
| <b>ALTER DATABASE</b>           | Modify database properties                | ALTER DATABASE db_name [SET parameter=value];     |                 |
| <b>ALTER DATASHARE</b>          | Change datashare settings                 | `ALTER DATASHARE share_name ADD                   | REMOVE<br>...;` |
| <b>ALTER DEFAULT PRIVILEGES</b> | Set default privileges for future objects | ALTER DEFAULT PRIVILEGES FOR ROLE role GRANT ...; |                 |
| <b>ALTER EXTERNAL SCHEMA</b>    | Update external schema definition         | ALTER EXTERNAL SCHEMA schema_name SET ...;        |                 |

| Command                        | What It Does                          | Basic Syntax                                        |  |
|--------------------------------|---------------------------------------|-----------------------------------------------------|--|
| <b>ALTER EXTERNAL VIEW</b>     | Modify external view                  | ALTER EXTERNAL VIEW view_name SET ...;              |  |
| <b>ALTER FUNCTION</b>          | Change function definition            | ALTER FUNCTION func_name(arg_types) OWNER TO user;  |  |
| <b>ALTER GROUP</b>             | Modify user group                     | ALTER GROUP group_name ADD USER user_name;          |  |
| <b>ALTER IDENTITY PROVIDER</b> | Update identity provider settings     | ALTER IDENTITY PROVIDER provider_name SET ...;      |  |
| <b>ALTER MASKING POLICY</b>    | Modify masking policy                 | ALTER MASKING POLICY policy_name SET ...;           |  |
| <b>ALTER MATERIALIZED VIEW</b> | Change materialized view              | ALTER MATERIALIZED VIEW view_name SET ...;          |  |
| <b>ALTER RLS POLICY</b>        | Modify row-level security policy      | ALTER RLS POLICY policy_name SET ...;               |  |
| <b>ALTER ROLE</b>              | Update role attributes                | ALTER ROLE role_name [WITH option];                 |  |
| <b>ALTER PROCEDURE</b>         | Modify stored procedure               | ALTER PROCEDURE proc_name(arg_types) OWNER TO user; |  |
| <b>ALTER SCHEMA</b>            | Change schema properties              | ALTER SCHEMA schema_name OWNER TO user;             |  |
| <b>ALTER SYSTEM</b>            | Change system-level settings          | ALTER SYSTEM SET parameter=value;                   |  |
| <b>ALTER TABLE</b>             | Modify table structure                | ALTER TABLE table_name ADD column_name datatype;    |  |
| <b>ALTER TABLE APPEND</b>      | Append data from one table to another | ALTER TABLE target APPEND FROM source;              |  |
| <b>ALTER USER</b>              | Change user attributes                | ALTER USER user_name [WITH option];                 |  |
| <b>ANALYZE</b>                 | Update table statistics               | ANALYZE table_name;                                 |  |

| Command                         | What It Does                     | Basic Syntax                                             |  |
|---------------------------------|----------------------------------|----------------------------------------------------------|--|
| <b>ANALYZE COMPRESSION</b>      | Check compression for columns    | ANALYZE COMPRESSION table_name;                          |  |
| <b>ATTACH MASKING POLICY</b>    | Attach masking policy to table   | ATTACH MASKING POLICY policy_name TO table_name;         |  |
| <b>ATTACH RLS POLICY</b>        | Attach row-level security policy | ATTACH RLS POLICY policy_name TO table_name;             |  |
| <b>BEGIN</b>                    | Start a transaction              | BEGIN;                                                   |  |
| <b>CALL</b>                     | Execute stored procedure         | CALL procedure_name(args);                               |  |
| <b>CANCEL</b>                   | Cancel a running query           | CANCEL query_id;                                         |  |
| <b>CLOSE</b>                    | Close a cursor                   | CLOSE cursor_name;                                       |  |
| <b>COMMENT</b>                  | Add comment to object            | COMMENT ON TABLE table_name IS 'description';            |  |
| <b>COMMIT</b>                   | Save transaction                 | COMMIT;                                                  |  |
| <b>COPY</b>                     | Load data from file              | COPY table_name FROM 's3://bucket/file' CREDENTIALS ...; |  |
| <b>CREATE DATABASE</b>          | Create new database              | CREATE DATABASE db_name;                                 |  |
| <b>CREATE DATASHARE</b>         | Create datashare                 | CREATE DATASHARE share_name;                             |  |
| <b>CREATE EXTERNAL FUNCTION</b> | Define external function         | CREATE EXTERNAL FUNCTION func_name(...) RETURNS ...;     |  |
| <b>CREATE EXTERNAL MODEL</b>    | Register external ML model       | CREATE EXTERNAL MODEL model_name ...;                    |  |
| <b>CREATE EXTERNAL SCHEMA</b>   | Define external schema           | CREATE EXTERNAL SCHEMA schema_name FROM ...;             |  |
| <b>CREATE EXTERNAL TABLE</b>    | Create external table            | CREATE EXTERNAL TABLE table_name (...);                  |  |

| Command                         | What It Does                     | Basic Syntax                                                      |  |
|---------------------------------|----------------------------------|-------------------------------------------------------------------|--|
| <b>CREATE EXTERNAL VIEW</b>     | Create external view             | CREATE EXTERNAL VIEW view_name AS<br>SELECT ...;                  |  |
| <b>CREATE FUNCTION</b>          | Create user-defined function     | CREATE FUNCTION func_name(...)<br>RETURNS ...;                    |  |
| <b>CREATE GROUP</b>             | Create user group                | CREATE GROUP group_name;                                          |  |
| <b>CREATE IDENTITY PROVIDER</b> | Register identity provider       | CREATE IDENTITY PROVIDER<br>provider_name ...;                    |  |
| <b>CREATE LIBRARY</b>           | Register external library        | CREATE LIBRARY lib_name ...;                                      |  |
| <b>CREATE MASKING POLICY</b>    | Define masking policy            | CREATE MASKING POLICY policy_name<br>...;                         |  |
| <b>CREATE MATERIALIZED VIEW</b> | Create materialized view         | CREATE MATERIALIZED VIEW<br>view_name AS SELECT ...;              |  |
| <b>CREATE MODEL</b>             | Create ML model                  | CREATE MODEL model_name ...;                                      |  |
| <b>CREATE PROCEDURE</b>         | Create stored procedure          | CREATE PROCEDURE proc_name(...)<br>LANGUAGE plpgsql AS \$ ... \$; |  |
| <b>CREATE RLS POLICY</b>        | Define row-level security policy | CREATE RLS POLICY policy_name<br>...;                             |  |
| <b>CREATE ROLE</b>              | Create role                      | CREATE ROLE role_name;                                            |  |
| <b>CREATE SCHEMA</b>            | Create schema                    | CREATE SCHEMA schema_name;                                        |  |
| <b>CREATE TABLE</b>             | Create table                     | CREATE TABLE table_name (col<br>datatype, ...);                   |  |
| <b>CREATE TABLE AS</b>          | Create table from query          | CREATE TABLE new_table AS SELECT<br>...;                          |  |
| <b>CREATE USER</b>              | Create user                      | CREATE USER user_name WITH<br>PASSWORD 'pwd';                     |  |
| <b>CREATE VIEW</b>              | Create view                      | CREATE VIEW view_name AS SELECT<br>...;                           |  |
| <b>DEALLOCATE</b>               | Free prepared statement          | DEALLOCATE prep_stmt;                                             |  |



| Command                               | What It Does                      | Basic Syntax                                          |  |
|---------------------------------------|-----------------------------------|-------------------------------------------------------|--|
| <b>DECLARE</b>                        | Declare cursor                    | DECLARE cursor_name CURSOR FOR<br>SELECT ...;         |  |
| <b>DELETE</b>                         | Remove rows                       | DELETE FROM table_name WHERE<br>condition;            |  |
| <b>DESC DATASHARE</b>                 | Show datashare<br>details         | DESC DATASHARE share_name;                            |  |
| <b>DESC IDENTITY<br/>PROVIDER</b>     | Show identity<br>provider details | DESC IDENTITY PROVIDER<br>provider_name;              |  |
| <b>DETACH MASKING<br/>POLICY</b>      | Remove masking<br>policy          | DETACH MASKING POLICY policy_name<br>FROM table_name; |  |
| <b>DETACH RLS<br/>POLICY</b>          | Remove RLS policy                 | DETACH RLS POLICY policy_name<br>FROM table_name;     |  |
| <b>DROP DATABASE</b>                  | Delete database                   | DROP DATABASE db_name;                                |  |
| <b>DROP DATASHARE</b>                 | Delete datashare                  | DROP DATASHARE share_name;                            |  |
| <b>DROP EXTERNAL<br/>VIEW</b>         | Delete external<br>view           | DROP EXTERNAL VIEW view_name;                         |  |
| <b>DROP FUNCTION</b>                  | Delete function                   | DROP FUNCTION<br>func_name(arg_types);                |  |
| <b>DROP GROUP</b>                     | Delete group                      | DROP GROUP group_name;                                |  |
| <b>DROP IDENTITY<br/>PROVIDER</b>     | Delete identity<br>provider       | DROP IDENTITY PROVIDER<br>provider_name;              |  |
| <b>DROP LIBRARY</b>                   | Delete library                    | DROP LIBRARY lib_name;                                |  |
| <b>DROP MASKING<br/>POLICY</b>        | Delete masking<br>policy          | DROP MASKING POLICY policy_name;                      |  |
| <b>DROP MODEL</b>                     | Delete ML model                   | DROP MODEL model_name;                                |  |
| <b>DROP<br/>MATERIALIZED<br/>VIEW</b> | Delete<br>materialized view       | DROP MATERIALIZED VIEW view_name;                     |  |
| <b>DROP<br/>PROCEDURE</b>             | Delete procedure                  | DROP PROCEDURE<br>proc_name(arg_types);               |  |
| <b>DROP RLS POLICY</b>                | Delete RLS policy                 | DROP RLS POLICY policy_name;                          |  |

| Command                          | What It Does              | Basic Syntax                                                              |  |
|----------------------------------|---------------------------|---------------------------------------------------------------------------|--|
| <b>DROP ROLE</b>                 | Delete role               | DROP ROLE role_name;                                                      |  |
| <b>DROP SCHEMA</b>               | Delete schema             | DROP SCHEMA schema_name;                                                  |  |
| <b>DROP TABLE</b>                | Delete table              | DROP TABLE table_name;                                                    |  |
| <b>DROP USER</b>                 | Delete user               | DROP USER user_name;                                                      |  |
| <b>DROP VIEW</b>                 | Delete view               | DROP VIEW view_name;                                                      |  |
| <b>END</b>                       | End transaction           | END;                                                                      |  |
| <b>EXECUTE</b>                   | Run prepared statement    | EXECUTE prep_stmt;                                                        |  |
| <b>EXPLAIN</b>                   | Show query plan           | EXPLAIN SELECT ...;                                                       |  |
| <b>FETCH</b>                     | Retrieve rows from cursor | FETCH NEXT FROM cursor_name;                                              |  |
| <b>GRANT</b>                     | Give privileges           | GRANT privilege ON object TO user;                                        |  |
| <b>INSERT</b>                    | Add rows                  | INSERT INTO table_name VALUES (...);                                      |  |
| <b>LOCK</b>                      | Lock table                | LOCK table_name;                                                          |  |
| <b>MERGE</b>                     | Upsert rows               | MERGE INTO target USING source ON condition WHEN MATCHED THEN UPDATE ...; |  |
| <b>PREPARE</b>                   | Prepare statement         | PREPARE prep_stmt AS SELECT ...;                                          |  |
| <b>REFRESH MATERIALIZED VIEW</b> | Update materialized view  | REFRESH MATERIALIZED VIEW view_name;                                      |  |
| <b>RESET</b>                     | Reset parameter           | RESET parameter;                                                          |  |
| <b>REVOKE</b>                    | Remove privileges         | REVOKE privilege ON object FROM user;                                     |  |
| <b>ROLLBACK</b>                  | Undo transaction          | ROLLBACK;                                                                 |  |
| <b>SELECT</b>                    | Retrieve data             | SELECT columns FROM table_name WHERE condition;                           |  |
| <b>SELECT INTO</b>               | Create table from query   | SELECT ... INTO new_table FROM ...;                                       |  |

| Command                  | What It Does              | Basic Syntax                                                       |  |
|--------------------------|---------------------------|--------------------------------------------------------------------|--|
| <b>SET</b>               | Set parameter             | SET parameter=value;                                               |  |
| <b>SHOW</b>              | Display settings          | SHOW parameter;                                                    |  |
| <b>SHOW TABLES</b>       | List tables               | SHOW TABLES;                                                       |  |
| <b>START TRANSACTION</b> | Begin transaction         | START TRANSACTION;                                                 |  |
| <b>TRUNCATE</b>          | Remove all rows           | TRUNCATE table_name;                                               |  |
| <b>UNLOAD</b>            | Export data to file       | UNLOAD ('SELECT ...') TO<br>'s3://bucket/file' CREDENTIALS<br>...; |  |
| <b>UPDATE</b>            | Modify rows               | UPDATE table_name SET<br>column=value WHERE condition;             |  |
| <b>USE</b>               | Switch database           | USE db_name;                                                       |  |
| <b>VACUUM</b>            | Reclaim space and<br>sort | VACUUM table_name;                                                 |  |

## Example:

- Database setup
- Schema and table creation
- Data loading
- Queries and transformations
- Security policies
- Maintenance tasks
- Cleanup

---

Step 1: Create Database and Schema

-- Create a new database for HR analytics

```
CREATE DATABASE hranalytics;
```

-- Switch to the new database

```
USE hranalytics;
```

-- Create schema for employee data

```
CREATE SCHEMA hr;
```

---

## Step 2: Create Tables and Views

-- Create employees table

```
CREATE TABLE hr.employees (
```

```
 empid INT,
```

```
 name VARCHAR(100),
```

```
 department VARCHAR(50),
```

```
 salary DECIMAL(10,2),
```

```
 hiredate DATE
```

```
);
```

-- Create a view for high-salary employees

```
CREATE VIEW hr.highsalary AS
```

```
SELECT name, department, salary FROM hr.employees WHERE salary > 80000;
```

-- Create materialized view for department-wise salary stats

```
CREATE MATERIALIZED VIEW hr.deptsalarystats AS
```

```
SELECT department, AVG(salary) AS avgsalary FROM hr.employees GROUP BY department;
```

---

## Step 3: Insert and Load Data

-- Insert sample data

```
INSERT INTO hr.employees VALUES (1, 'Alice', 'IT', 90000, '2020-01-15');
```

```
INSERT INTO hr.employees VALUES (2, 'Bob', 'Finance', 75000, '2019-03-10');
```

-- Load bulk data from S3

```
COPY hr.employees FROM 's3://company-data/employees.csv'
```

```
CREDENTIALS 'awsaccesskeyid=xxx;awssecretaccesskey=yyy'
```

```
CSV;
```

---

#### Step 4: Query and Transform

-- Select employees hired after 2020

```
SELECT name, hiredate FROM hr.employees WHERE hiredate > '2020-01-01';
```

-- Update salary for an employee

```
UPDATE hr.employees SET salary = 95000 WHERE empid = 1;
```

-- Delete an employee record

```
DELETE FROM hr.employees WHERE empid = 2;
```

-- Merge new data from staging table

```
MERGE INTO hr.employees USING hr.stagingemployees ON hr.employees.empid =
hr.stagingemployees.empid
```

```
WHEN MATCHED THEN UPDATE SET salary = hr.stagingemployees.salary
```

```
WHEN NOT MATCHED THEN INSERT VALUES (hr.stagingemployees.empid,
hr.stagingemployees.name, hr.stagingemployees.department, hr.stagingemployees.salary,
hr.stagingemployees.hiredate);
```

---

#### Step 5: Security and Policies

-- Create masking policy for salary

```
CREATE MASKING POLICY masksalary AS (salary) RETURNS VARCHAR USING ('****');
```

-- Attach masking policy to employees table

```
ATTACH MASKING POLICY masksalary TO hr.employees;
```

-- Create RLS policy for department-based access

```
CREATE RLS POLICY deptaccess AS (department = CURRENTUSER);
```

-- Attach RLS policy

```
ATTACH RLS POLICY dept_access TO hr.employees;
```

-- Grant SELECT privilege to analyst role

GRANT SELECT ON hr.employees TO analyst;

---

#### Step 6: Maintenance

-- Analyze table for stats

ANALYZE hr.employees;

-- Vacuum table to reclaim space

VACUUM hr.employees;

-- Refresh materialized view

REFRESH MATERIALIZED VIEW hr.deptsalarystats;

-- Explain query plan

EXPLAIN SELECT \* FROM hr.employees WHERE salary > 80000;

---

#### Step 7: Cleanup

-- Detach policies

DETACH MASKING POLICY masksalary FROM hr.employees;

DETACH RLS POLICY deptaccess FROM hr.employees;

-- Drop objects

DROP VIEW hr.highsalary;

DROP MATERIALIZED VIEW hr.deptsalarystats;

DROP TABLE hr.employees;

DROP SCHEMA hr;

DROP DATABASE hranalytics;

## ✓ 1. Masking Policy

**Purpose:** A **masking policy** is used to hide or obfuscate sensitive data (like salary, SSN, credit card numbers) when queried by certain users or roles. It ensures compliance with data privacy regulations.

### How It Works:

- You define a policy specifying which columns to mask and how.
- Attach the policy to tables or columns.
- When a user without full access queries the table, masked values are returned instead of real data.

### Example:

-- Create a masking policy to hide salary

```
CREATE MASKING POLICY masksalary AS (salary) RETURNS VARCHAR USING ('****');
```

-- Attach policy to employees table

```
ATTACH MASKING POLICY masksalary TO hr.employees;
```

**Result:** Users with restricted access will see \*\*\*\* instead of actual salary values.

---

## ✓ 2. Row-Level Security (RLS) Policy

**Purpose:** An **RLS policy** controls which rows a user can access based on conditions (e.g., department, region). It enforces fine-grained access control.

### How It Works:

- Define a policy with a condition (e.g., `department = CURRENT_USER`).
- Attach it to a table.
- When a user queries the table, only rows matching the condition are returned.

### Example:

-- Create RLS policy for department-based access

```
CREATE RLS POLICY deptaccess AS (department = CURRENTUSER);
```

-- Attach policy to employees table

```
ATTACH RLS POLICY dept_access TO hr.employees;
```

**Result:** \ If the current user is `Finance`, they only see rows where `department = 'Finance'`.

---

### ✓ 3. Identity Provider

**Purpose:** \ An **identity provider (IdP)** integrates Redshift with external authentication systems (like AWS IAM, Okta, or SAML). It allows federated login and single sign-on.

**How It Works:**

- You register an IdP in Redshift.
- Configure roles and mappings.
- Users authenticate via the IdP instead of local credentials.

**Example:**

```
-- Create identity provider for SAML
CREATE IDENTITY PROVIDER samlprovider TYPE SAML
WITH SAML-metadata 'https://idp.example.com/metadata.xml';
```

**Result:** \ Users can log in using their corporate credentials via SAML.

---

### ✓ 4. Default Privileges

**Purpose:** \ Set **default privileges** for objects created in a schema (tables, views, functions). This ensures new objects automatically inherit permissions.

**Example:**

```
-- Grant SELECT on all future tables in schema hr to analyst role
ALTER DEFAULT PRIVILEGES IN SCHEMA hr GRANT SELECT ON TABLES TO analyst;
```

## SQL Commands

---



## 20.5 SQL Function

### 20.5.1 Aggregate Function

---

#### ✓ ANY\_VALUE

- **Purpose:** Returns any non-NULL value from a group of rows. Useful when you need a placeholder value without caring which one.

---

#### ✓ APPROXIMATE PERCENTILE\_DISC

- **Purpose:** Calculates an approximate percentile using a discrete distribution. Faster than exact percentile calculations for large datasets.

---

#### ✓ AVG

- **Purpose:** Computes the average (mean) of numeric values in a column.

---

#### ✓ COUNT

- **Purpose:** Counts the number of rows or non-NULL values in a column.

---

#### ✓ LISTAGG

- **Purpose:** Aggregates multiple values into a single string, separated by a specified delimiter.

---

#### ✓ MAX

- **Purpose:** Returns the maximum value from a set of values.

---

#### ✓ MEDIAN

- **Purpose:** Returns the median (middle) value from a set of numeric values.

---

#### ✓ MIN

- **Purpose:** Returns the minimum value from a set of values.
-

## ✓ PERCENTILE\_CONT

- **Purpose:** Calculates a percentile using a continuous distribution, interpolating between values for precision.

---

## ✓ STDDEV\_SAMP and STDDEV\_POP

- **Purpose:**
  - STDDEV\_SAMP: Standard deviation of a sample.
  - STDDEV\_POP: Standard deviation of an entire population.

---

## ✓ SUM

- **Purpose:** Computes the sum of numeric values in a column.

---

## ✓ VAR\_SAMP and VAR\_POP

- **Purpose:**
  - VAR\_SAMP: Variance of a sample.
  - VAR\_POP: Variance of an entire population.

Example:

- -- Calculate average salary
- SELECT AVG(salary) AS avg\_salary FROM employees;
- 
- -- Count total employees
- SELECT COUNT(\*) AS total\_employees FROM employees;
- 
- -- Sum of all salaries
- SELECT SUM(salary) AS total\_salary FROM employees;
- 
- -- Minimum and maximum salary
- SELECT MIN(salary) AS min\_salary, MAX(salary) AS max\_salary FROM employees;
- 
- -- Median salary
- SELECT MEDIAN(salary) AS median\_salary FROM employees;
- 
- -- Exact 75th percentile salary
- SELECT PERCENTILE\_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS p75\_salary FROM employees;
- 
- -- Approximate 90th percentile salary
- SELECT APPROXIMATE PERCENTILE\_DISC(0.9) WITHIN GROUP (ORDER BY salary) AS approx\_p90\_salary FROM employees;
- 
- -- Standard deviation and variance (sample and population)

- `SELECT STDDEV_SAMP(salary) AS stddev_sample,`
- `STDDEV_POP(salary) AS stddev_population,`
- `VAR_SAMP(salary) AS var_sample,`
- `VAR_POP(salary) AS var_population`
- `FROM employees;`
- 
- `-- List all employee names in one string`
- `SELECT LISTAGG(name, ', ') WITHIN GROUP (ORDER BY name) AS`  
`employee_names FROM employees;`
- 
- `-- Get any department name (non-NULL)`
- `SELECT ANY_VALUE(department) AS sample_department FROM employees;`

## 20.5.2 Array Functions

- **ARRAY**: Creates an array from a list of values or query results.
- **ARRAY\_CONCAT**: Combines two or more arrays into one.
- **ARRAY\_FLATTEN**: Converts a nested array into a single-level array.
- **GET\_ARRAY\_LENGTH**: Returns the number of elements in an array.
- **SPLIT\_TO\_ARRAY**: Splits a string into an array based on a delimiter.
- **SUBARRAY**: Extracts a portion of an array starting at a given index for a specified length.

```
CREATE TABLE employees (
 emp_id INT,
 name VARCHAR(50),
 skills VARCHAR(200) -- Comma-separated skills
);
```

```
INSERT INTO employees VALUES
(1, 'Alice', 'SQL,Python,Java'),
(2, 'Bob', 'Python,JavaScript'),
(3, 'Charlie', 'Java,C++,SQL');
```

---

### Examples Using Array Functions

`-- Convert skills string to array`

```
SELECT SPLIT_ARRAY(skills, ',') AS skillarray FROM employees;
```

`-- Get length of array (number of skills per employee)`

```
SELECT GET_ARRAY_LENGTH(SPLITTOARRAY(skills, ',')) AS skillcount FROM employees;
```

-- Create an array manually

```
SELECT ARRAY['SQL', 'Python', 'Java'] AS techstack;
```

-- Concatenate two arrays

```
SELECT ARRAY_CONCAT(ARRAY['SQL', 'Python'], ARRAY['JavaScript']) AS combinedskills;
```

-- Flatten nested arrays

```
SELECT ARRAY_FLATTEN(ARRAY[ARRAY['SQL', 'Python'], ARRAY['Java']]) AS flatskills;
```

-- Extract a subarray (first two skills)

```
SELECT SUBARRAY(SPLITTOARRAY(skills, ','), 1, 2) AS firsttwo_skills FROM employees;
```

---

✅ This scenario demonstrates:

- **SPLIT\_TO\_ARRAY** for converting strings to arrays
  - **GET\_ARRAY\_LENGTH** for counting elements
  - **ARRAY** for creating arrays
  - **ARRAY\_CONCAT** for merging arrays
  - **ARRAY\_FLATTEN** for flattening nested arrays
  - **SUBARRAY** for slicing arrays
- 

### 20.5.3 Contitional Expressions

- **CASE**: Performs conditional logic in SQL, similar to `IF-THEN-ELSE`.
- **DECODE**: Simplifies conditional checks by mapping values to results.
- **GREATEST and LEAST**: Returns the largest or smallest value from a list of expressions.
- **NVL and COALESCE**: Replace `NULL` with a specified value. `COALESCE` can check multiple values.
- **NVL2**: Returns one value if the expression is `NOT NULL`, another if it is `NULL`.
- **NULLIF**: Returns `NULL` if two expressions are equal; otherwise returns the first expression.

We have a table:

```
CREATE TABLE employees (
 emp_id INT,
 name VARCHAR(50),
 department VARCHAR(50),
 salary DECIMAL(10,2),
 bonus DECIMAL(10,2)
);
```

```
INSERT INTO employees VALUES
(1, 'Alice', 'IT', 90000, NULL),
(2, 'Bob', 'Finance', 75000, 5000),
(3, 'Charlie', 'HR', 60000, NULL);
```

---

### Examples Using Conditional Expressions

-- CASE: Assign bonus based on department

```
SELECT name,
 CASE
 WHEN department = 'IT' THEN 10000
 WHEN department = 'Finance' THEN 7000
 ELSE 5000
 END AS calculatedbonus
FROM employees;
```

-- DECODE: Map department to bonus

```
SELECT name,
 DECODE(department, 'IT', 10000, 'Finance', 7000, 5000) AS calculatedbonus
FROM employees;
```

-- GREATEST and LEAST: Find highest and lowest between salary and bonus

```

SELECT name,
 GREATEST(salary, COALESCE(bonus, 0)) AS maxvalue,
 LEAST(salary, COALESCE(bonus, 0)) AS minvalue
FROM employees;

-- NVL: Replace NULL bonus with 0
SELECT name, NVL(bonus, 0) AS bonusvalue FROM employees;

-- COALESCE: Replace NULL with first non-null value from multiple columns
SELECT name, COALESCE(bonus, salary * 0.05, 0) AS effectivebonus FROM employees;

-- NVL2: If bonus is NOT NULL, add 1000; else add 500
SELECT name, NVL2(bonus, bonus + 1000, 500) AS adjustedbonus FROM employees;

-- NULLIF: Avoid division by zero (returns NULL if bonus = 0)
SELECT name, salary / NULLIF(bonus, 0) AS salarytobonusratio FROM employees;

```

---

## 20.5.4 Data type formatting functions

- **CAST**: Converts a value from one data type to another.
- **CONVERT**: Similar to CAST, but often used for specific type conversions (e.g., date/time formats).
- **TEXT\_TO\_INT\_ALT**: Converts text to an integer, handling alternative formats.
- **TEXT\_TO\_NUMERIC\_ALT**: Converts text to a numeric value, handling alternative formats.
- **TO\_CHAR**: Converts a number or date to a string with a specified format.
- **TO\_DATE**: Converts a string to a date using a format mask.
- **TO\_NUMBER**: Converts a string to a numeric value using a format mask.
- **TRY\_CAST**: Attempts to cast a value; returns NULL if conversion fails (safe conversion).

Imagine we have a table `transactions`:

```

CREATE TABLE transactions (
 txnid INT,
 amounttext VARCHAR(20),

```

```
txndatetext VARCHAR(20)
);
```

```
INSERT INTO transactions VALUES
(1, '1,200.50', '2025-11-17'),
(2, '850', '2025-10-01'),
(3, 'invalid', 'not a date');
```

---

### Examples Using Conversion Functions

-- CAST: Convert amounttext to numeric (basic)

```
SELECT CAST(amounttext AS DECIMAL(10,2)) AS amount FROM transactions;
```

-- CONVERT: Convert string to date (alternative syntax)

```
SELECT CONVERT(DATE, txndatetext) AS txndate FROM transactions;
```

-- TEXTTOINTALT: Convert text to integer, handling commas

```
SELECT TEXTTOINTALT(amounttext) AS amountint FROM transactions;
```

-- TEXTTONUMERICALT: Convert text to numeric, handling commas and decimals

```
SELECT TEXTTONUMERICALT(amounttext) AS amountnum FROM transactions;
```

-- TOCHAR: Format numeric as currency

```
SELECT TOCHAR(1200.5, 'FM$999,999.00') AS formattedamount;
```

-- TODATE: Convert string to date with format mask

```
SELECT TODATE(txndatetext, 'YYYY-MM-DD') AS txndate FROM transactions;
```

-- TONUMBER: Convert formatted string to number

```
SELECT TONUMBER('1,200.50', '999,999.99') AS amountnum;
```

-- TRYCAST: Safely convert text to numeric (returns NULL if invalid)

```
SELECT TRYCAST(amounttext AS DECIMAL(10,2)) AS safeamount FROM transactions;
```

---

## 20.5.5 Date Function

- **+** (**Concatenation**): Joins two strings together.
  - **ADD\_MONTHS**: Adds a specified number of months to a date.
  - **AT TIME\_ZONE**: Converts a timestamp to a specified time zone.
  - **CONVERT\_TIMEZONE**: Converts a timestamp from one time zone to another.
  - **CURRENT\_DATE / SYSDATE / GETDATE / TIMEOFDAY**: Returns the current date or timestamp.
  - **DATE\_CMP, DATE\_CMP\_TIMESTAMP, DATE\_CMP\_TIMESTAMP\_TZ**: Compares two dates or timestamps.
  - **DATEADD**: Adds an interval (days, months, etc.) to a date.
  - **DATEDIFF**: Returns the difference between two dates in specified units.
  - **DATE\_PART / DATE\_PART\_YEAR / EXTRACT**: Extracts a specific part (year, month, day) from a date or timestamp.
  - **DATE\_TRUNC**: Truncates a date or timestamp to a specified precision (month, year).
  - **INTERVAL\_CMP**: Compares two intervals.
  - **LAST\_DAY**: Returns the last day of the month for a given date.
  - **MONTHS\_BETWEEN**: Returns the number of months between two dates.
  - **NEXT\_DAY**: Returns the next occurrence of a specified weekday after a given date.
  - **TIMEZONE**: Retrieves or sets the current time zone.
  - **TO\_TIMESTAMP**: Converts a string to a timestamp.
  - **TRUNC**: Truncates a numeric or date value to a specified precision.
- 

Imagine we have a table `payroll`:

```
CREATE TABLE payroll (
```

```
 empid INT,
```

```
 name VARCHAR(50),
```

```
 joindate DATE,
```

```
 last_payment TIMESTAMP,
```

```
 salary DECIMAL(10,2)
```

```
);
```

```
INSERT INTO payroll VALUES
```

```
(1, 'Alice', '2023-01-15', '2025-11-01 10:30:00', 90000),
```



```
(2, 'Bob', '2024-03-10', '2025-10-25 09:00:00', 75000);
```

---

### Examples Using Date/Time Functions

-- Concatenate name and department

```
SELECT name + ' - Payroll' AS employeelabel FROM payroll;
```

-- Add 3 months to joindate

```
SELECT ADDMONTHS(joindate, 3) AS probationend FROM payroll;
```

-- Convert timestamp to UTC

```
SELECT lastpayment AT TIME ZONE 'UTC' AS paymentutc FROM payroll;
```

-- Convert timestamp from PST to EST

```
SELECT CONVERTTIMEZONE('US/Pacific', 'US/Eastern', lastpayment) AS paymentest FROM payroll;
```

-- Current date and time

```
SELECT CURRENTDATE AS today, SYSDATE AS systemtime, GETDATE() AS gettime, TIMEOFDAY() AS timeofday;
```

-- Compare two dates

```
SELECT DATECMP(joindate, CURRENTDATE) AS cmpresult FROM payroll;
```

-- Add 10 days to last payment date

```
SELECT DATEADD(day, 10, lastpayment) AS nextpayment FROM payroll;
```

-- Difference in days between joindate and today

```
SELECT DATEDIFF(day, joindate, CURRENTDATE) AS daysworked FROM payroll;
```

-- Extract year and month from joindate

```
SELECT DATEPART('year', joindate) AS joinyear, DATEPART('month', joindate) AS joinmonth FROM payroll;
```

-- Truncate date to month

```
SELECT DATETRUNC('month', lastpayment) AS paymentmonth FROM payroll;
```

-- Last day of the month for joindate

```
SELECT LASTDAY(joindate) AS monthend FROM payroll;
```

-- Months between joindate and today

```
SELECT MONTHSBETWEEN(CURRENTDATE, joindate) AS monthsworked FROM payroll;
```

-- Next Monday after last payment

```
SELECT NEXTDAY(lastpayment, 'Monday') AS nextmonday FROM payroll;
```

-- Convert string to timestamp

```
SELECT TOTIMESTAMP('2025-11-17 14:30:00', 'YYYY-MM-DD HH24:MI:SS') AS
convertedtimestamp;
```

-- Truncate salary to nearest thousand

```
SELECT TRUNC(salary, -3) AS roundedsalary FROM payroll;
```

---

## 20.5.6 JSON functions

- **JSON\_PARSE**: Converts a valid JSON string into a Redshift SUPER data type for semi-structured data handling.
- **CAN\_JSON\_PARSE**: Checks if a string can be parsed as valid JSON (returns TRUE/FALSE).
- **JSON\_SERIALIZE**: Converts a SUPER value back into a JSON string.
- **JSON\_SERIALIZE\_TO\_VARBYTE**: Converts a SUPER value into a VARBYTE representation of JSON.
- **IS\_VALID\_JSON**: Returns TRUE if the input string is valid JSON.
- **IS\_VALID\_JSON\_ARRAY**: Returns TRUE if the input string is a valid JSON array.
- **JSON\_ARRAY\_LENGTH**: Returns the number of elements in a JSON array.
- **JSON\_EXTRACT\_ARRAY\_ELEMENT\_TEXT**: Extracts a specific element from a JSON array as text.
- **JSON\_EXTRACT\_PATH\_TEXT**: Extracts the value of a specific key from a JSON object as text.

Imagine we have a table `employee_data`:

```
CREATE TABLE employeedata (
 empid INT,
 details SUPER
);
```

```
INSERT INTO employeedata VALUES
(1, JSONPARSE({'name':"Alice","skills":["SQL","Python"],"salary":90000})),
(2, JSON_PARSE({'name':"Bob","skills":["Java","C++"],"salary":75000}));
```

---

### Examples Using JSON Functions

-- Check if a string is valid JSON

```
SELECT ISVALIDJSON({'name':"Alice"}) AS validjson;
```

-- Check if a string is a valid JSON array

```
SELECT ISVALIDJSONARRAY(['SQL',"Python"]) AS validjsonarray;
```

-- Verify if a string can be parsed as JSON

```
SELECT CANJSONPARSE({'name':"Alice"}) AS canparse;
```

-- Parse JSON string into SUPER type

```
SELECT JSONPARSE({'department':"IT","location":"NY"}) AS parsedjson;
```

-- Serialize SUPER back to JSON string

```
SELECT JSONSERIALIZE(details) AS jsonstring FROM employeedata;
```

-- Serialize SUPER to VARBYTE

```
SELECT JSONSERIALIZETOVARBYTE(details) AS jsonvarbyte FROM employeedata;
```

-- Get length of JSON array (skills)

```
SELECT JSONARRAYLENGTH(details.skills) AS skillcount FROM employeeedata;
```

```
-- Extract specific skill from array
```

```
SELECT JSONEXTRACTARRAYELEMENTTEXT(details.skills, 1) AS firstskill FROM employeeedata;
```

```
-- Extract name from JSON object
```

```
SELECT JSONEXTRACTPATHTEXT(details, 'name') AS employeeename FROM employeeedata;
```

---